



การพัฒนาระบบต้นแบบของสภาพแวดล้อมอัจฉริยะ สำหรับผู้ใช้อิธรถเข็น

โดย

นายวรพล แสงสระศรี

นายศุภวิชญ์ อัสวลาทยอง

โครงการนี้เป็นส่วนหนึ่งของการศึกษาตามหลักสูตร
วิศวกรรมศาสตรบัณฑิต
สาขาวิศวกรรมไฟฟ้าและคอมพิวเตอร์
คณะวิศวกรรมศาสตร์ มหาวิทยาลัยธรรมศาสตร์
ปีการศึกษา 2568
ลิขสิทธิ์ของมหาวิทยาลัยธรรมศาสตร์

การพัฒนาระบบต้นแบบของสภาพแวดล้อมอัจฉริยะ สำหรับผู้ใช้งานรถเข็น

โดย

นายวรพล แสงสระศรี

นายศุภวิชญ์ อัครลาภทอง

โครงการนี้เป็นส่วนหนึ่งของการศึกษาตามหลักสูตร
วิศวกรรมศาสตรบัณฑิต
สาขาวิศวกรรมไฟฟ้าและคอมพิวเตอร์
คณะวิศวกรรมศาสตร์ มหาวิทยาลัยธรรมศาสตร์
ปีการศึกษา 2568
ลิขสิทธิ์ของมหาวิทยาลัยธรรมศาสตร์

Prototyping Development of Smart Environment for Wheelchair User

BY

Mr. Worapon Saengsrasi

Mr. Supawit Ausawalaithong

A PROJECT SUBMITTED IN PARTIAL FULFILLMENT OF THE
REQUIREMENTS FOR THE DEGREE OF BACHELOR OF ENGINEERING
IN ELECTRICAL AND COMPUTER ENGINEERING
FACULTY OF ENGINEERING
THAMMASAT UNIVERSITY
ACADEMIC YEAR 2025
COPYRIGHT OF THAMMASAT UNIVERSITY

มหาวิทยาลัยธรรมศาสตร์

คณะวิศวกรรมศาสตร์

โครงการ

ของ

นายวรพล แสงสระศรี

นายศุภวิชญ์ อัครฉายทอง

เรื่อง

การพัฒนาระบบต้นแบบของสภาพแวดล้อมอัจฉริยะสำหรับผู้ใช้อัจฉริยะ

ได้รับการตรวจสอบและอนุมัติ ให้เป็นส่วนหนึ่งของการศึกษาตามหลักสูตร
วิศวกรรมศาสตรบัณฑิต

เมื่อวันที่ 1 พฤษภาคม พ.ศ. 2569

อาจารย์ที่ปรึกษาโครงการ

(ผู้ช่วยศาสตราจารย์ ดร. ศุภชัย วรพจน์พิศุทธิ์)

หัวหน้าภาควิชาวิศวกรรมไฟฟ้าและคอมพิวเตอร์

(ผู้ช่วยศาสตราจารย์ ดร. ศุภชัย วรพจน์พิศุทธิ์)

หัวข้อโครงการ	การพัฒนาระบบต้นแบบของสภาพแวดล้อมอัจฉริยะสำหรับผู้ใช้อัจฉริยะ
ชื่อผู้เขียน	นายวรพล แสงสระศรี นายศุภวิชญ์ อัสวลาทยอง
ชื่อปริญญา	วิศวกรรมศาสตรบัณฑิต
สาขาวิชา/คณะ/มหาวิทยาลัย	วิศวกรรมไฟฟ้าและคอมพิวเตอร์ คณะวิศวกรรมศาสตร์ มหาวิทยาลัยธรรมศาสตร์
อาจารย์ที่ปรึกษาโครงการ	ผู้ช่วยศาสตราจารย์ ดร. ศุภชัย วรพจน์พิศุทธิ์
ปีการศึกษา	2568

บทคัดย่อ

โครงการนี้เสนอแพลตฟอร์ม WheelSense ระบบต้นแบบสภาพแวดล้อมอัจฉริยะสำหรับผู้ใช้อัจฉริยะในสถานดูแลผู้สูงอายุ โดยบูรณาการการรับรู้การเคลื่อนไหว การระบุตำแหน่งภายในอาคาร การบริหารงานเชิงปฏิบัติการ และการสนับสนุนการตัดสินใจของผู้ดูแล เข้าเป็นสายงานข้อมูลครบวงจร ภายใต้โครงสร้างพื้นฐานที่ติดตั้งและดูแลรักษาได้จริง เพื่อแก้ปัญหาความปลอดภัย การสื่อสารข้ามกะ และการบันทึกข้อมูลแบบกระต่ายในบ้านพักคนชราของประเทศไทย

สถาปัตยกรรมของระบบประกอบด้วยสี่ชั้นที่ทำงานประสานกัน ได้แก่ ชั้นอุปกรณ์ภาคสนามสำหรับตรวจจับการเคลื่อนที่และวัดสัญญาณชีพพื้นฐาน ชั้นโหนดภายในอาคารสำหรับแกนสัญญาณวิทยุ (BLE beacon) และเสริมภาพบริบท ชั้นเซิร์ฟเวอร์ภายในสถานดูแลสำหรับรับเข้า จัดเก็บ และให้บริการข้อมูลผ่าน API ที่มีการยืนยันตัวตน และชั้นส่วนติดต่อผู้ใช้ ประกอบด้วยแดชบอร์ดเว็บตามบทบาทและแอปพลิเคชันบนสมาร์ตโฟน ระบบระบุตำแหน่งใช้แนวทางลายนิ้วมือสัญญาณวิทยุ (RSSI fingerprinting) ร่วมกับอัลกอริทึมเพื่อนบ้านใกล้สุด (K-Nearest Neighbors) ในระดับโหนดและห้อง ส่วนผู้ช่วยปัญญาประดิษฐ์ (AI) แบบประมวลผลภายในเครื่อง (on-device) ถูกบูรณาการผ่าน Model Context Protocol (MCP) ภายใต้กระบวนการเสนอ-ยืนยัน-ดำเนินการ เพื่อให้การกระทำที่เปลี่ยนสถานะของระบบต้องได้รับการยืนยันก่อนเสมอ และเพื่อรักษาความเป็นส่วนตัวของข้อมูลผู้พักอาศัยภายในสถานดูแล

การประเมินระบบต้นแบบดำเนินการที่มหาวิทยาลัยธรรมศาสตร์ครอบคลุมสี่มิติ ได้แก่ ความทนทานของการจำแนกตำแหน่งจากชุดข้อมูลภาคสนาม; ความต่อเนื่องและเวลาแฝงของสายงานข้อมูลครบวงจร ความสอดคล้องในการเลือกเครื่องมือและเวลาแฝงของผู้ช่วย AI จากชุดสถานการณ์

ทดสอบ และประสบการณ์ผู้ใช้ตามบทบาทจากแบบสอบถามออนไลน์ นอกจากนี้ โครงการได้นำเสนอระบบต้นแบบต่อผู้บริหารและผู้ดูแลของบ้านพักคนชราवासนะเวศม์ เพื่อเก็บข้อเสนอแนะเชิงการใช้งานจริง โดยไม่มีการติดตั้งระบบ ณ สถานที่ดังกล่าว

ผลการพัฒนาสะท้อนความเป็นไปได้ของการบูรณาการหลายชั้นเทคโนโลยีเข้าเป็นสายงานเดียว ภายใต้ข้อจำกัดด้านต้นทุน ความซับซ้อนของการติดตั้ง และความเป็นส่วนตัวของข้อมูลประเด็นที่ต้องพัฒนาต่อยอด ได้แก่ การสอบเทียบเวลาระหว่างองค์ประกอบย่อย การขยายชุดกรณีทดสอบให้สะท้อนสภาพงานจริงมากยิ่งขึ้น และการปรับปรุงประสบการณ์ใช้งานให้เหมาะสมกับผู้ดูแลในบริบทที่หลากหลาย ซึ่งเป็นแนวทางสำคัญสำหรับการนำระบบไปประยุกต์ใช้ในที่พักอาศัย โรงพยาบาล และสถานดูแลผู้สูงอายุของประเทศไทยในระยะถัดไป

คำสำคัญ: สภาพแวดล้อมอัจฉริยะ, ผู้ใช้เก้าอี้รถเข็น, บลูทูธพลังงานต่ำ, ระบบระบุตำแหน่งในอาคาร, อินเทอร์เน็ตของสรรพสิ่ง

Title	Prototyping Development of Smart Environment for Wheelchair User
Author	Mr. Worapon Saengsrasi Mr. Supawit Ausawalaithong
Degree	Bachelor of Engineering
Major Field/Faculty/University	Electrical and Computer Engineering Faculty of Engineering Thammasat University
Advisor	Asst. Prof. Dr. Supachai Worapotphisut
Academic Year	2025

ABSTRACT

This project presents WheelSense, a prototype smart environment for wheelchair users in elderly care facilities. The system integrates motion sensing, indoor positioning, operational management, and caregiver decision support into an end-to-end data pipeline under deployable infrastructure. It aims to address safety limitations, cross-shift communication gaps, and paper-based recording practices prevalent in Thai nursing homes.

The architecture comprises four coordinated layers: (1) edge devices for motion detection and vital sign sensing; (2) indoor nodes for BLE beacon scanning and contextual enhancement; (3) a care-facility server for data ingestion, storage, and authenticated APIs; and (4) user interfaces including role-based web dashboards and mobile applications. Indoor positioning employs RSSI fingerprinting with K-Nearest Neighbors (KNN) for zone and room-level inference. An on-device AI assistant is integrated via the Model Context Protocol (MCP) under a propose-confirm-execute workflow, ensuring state-changing actions require prior confirmation while preserving resident data privacy.

System evaluation at Thammasat University covers four dimensions: positioning robustness from field data; end-to-end pipeline continuity and latency; AI tool-selection consistency and latency from template-based test scenarios; and

role-based user experience from online questionnaires. The prototype was also presented to administrators and caregivers at Vasanthawet Elderly Home to collect practical feedback without on-site installation.

Development results demonstrate the feasibility of integrating multi-layer technologies into a unified pipeline under cost, installation complexity, and data privacy constraints. Future work includes timestamp alignment across subsystems, expanding test cases to reflect real-world conditions, and improving user experience for caregivers in diverse contexts. These approaches are critical for deploying the system in residences, hospitals, and elderly care facilities in Thailand.

keyword: Smart environment, wheelchair user, Bluetooth Low Energy, indoor positioning, Internet of Things

กิตติกรรมประกาศ

คณะผู้จัดทำขอขอบพระคุณ ผู้ช่วยศาสตราจารย์ ดร. ศุภชัย วรพจน์พิศุทธิ์ อาจารย์ที่ปรึกษาโครงการ ที่ได้ให้คำแนะนำ ข้อเสนอแนะ และคำปรึกษาทางวิชาการอย่างต่อเนื่องตลอดการดำเนินงาน ทั้งในด้านการออกแบบระบบ การเลือกใช้เทคโนโลยี และการแก้ไขปัญหาทางเทคนิค จนโครงการสามารถพัฒนาได้อย่างเป็นระบบ

คณะผู้จัดทำขอขอบพระคุณ รองศาสตราจารย์ ดร. สายรัก สอาดไพร คณาจารย์ และเพื่อนร่วมงานจากคณะสหเวชศาสตร์ มหาวิทยาลัยธรรมศาสตร์ ที่ให้ความอนุเคราะห์ด้านข้อมูล คำแนะนำ และมุมมองการใช้งานจริงของผู้ใช้เก่าอีรธเซ็น ซึ่งเป็นประโยชน์อย่างยิ่งต่อการปรับปรุงระบบต้นแบบให้เหมาะสมและปลอดภัยมากยิ่งขึ้น

คณะผู้จัดทำขอขอบคุณทุกท่านที่ให้การสนับสนุนตลอดระยะเวลาการดำเนินโครงการ และหวังเป็นอย่างยิ่งว่าโครงการนี้จะเป็นส่วนหนึ่งในการพัฒนาเทคโนโลยีเพื่อยกระดับคุณภาพชีวิตของผู้ใช้เก่าอีรธเซ็นและผู้พิการในอนาคต

สารบัญ

	หน้า
บทคัดย่อภาษาไทย	(1)
บทคัดย่อภาษาอังกฤษ	(3)
กิตติกรรมประกาศ	(5)
สารบัญตาราง	(10)
สารบัญรูป	(12)
สัญลักษณ์และคำย่อ	16
บทที่ 1 บทนำ	17
1.1 ที่มาและความสำคัญ	17
1.2 วัตถุประสงค์	19
1.3 ขอบเขตงานวิจัย	19
1.4 ประโยชน์ที่คาดว่าจะได้รับ	20
1.5 แผนการดำเนินงาน	21
บทที่ 2 วรณกรรมและทฤษฎีที่เกี่ยวข้อง	22
2.1 บริบทการดูแลผู้สูงอายุและความท้าทายในสถานดูแล	22
2.2 แนวคิด IoT, Smart Environment และ Ambient Assisted Living	23
2.3 ทฤษฎีระบบระบุตำแหน่งภายในอาคาร	24
2.3.1 เทคนิคเชิงระยะทาง (Range-based Techniques)	24
2.3.2 เทคนิคการสร้างแผนที่สัญญาณ (Fingerprinting)	26
2.4 เทคโนโลยีเซ็นเซอร์สำหรับการตรวจวัดกิจกรรม	27
2.4.1 Photoplethysmography (PPG)	27
2.4.2 Inertial Measurement Unit (IMU)	28
2.5 อัลกอริทึม K-Nearest Neighbors (KNN)	30
2.6 อุปกรณ์ฮาร์ดแวร์และเครื่องมือที่เกี่ยวข้อง	31
2.6.1 M5StickC Plus2	31
2.6.2 Lilygo T-SimCam	32
2.6.3 Polar Verity Sense	33
2.6.4 Raspberry Pi 5	34
2.7 เทคโนโลยีซอฟต์แวร์และโครงสร้างพื้นฐานที่เกี่ยวข้อง	35
2.7.1 FastAPI (Backend Framework)	36
2.7.2 React (UI Library)	37

2.7.3	Next.js (Frontend Framework)	38
2.7.4	Flutter และ Dart (Mobile Development Framework)	38
2.7.5	PostgreSQL (Database Management System)	39
2.7.6	Docker (Containerization)	40
2.7.7	Ollama และ Large Language Models (LLM)	41
2.7.8	Model Context Protocol (MCP)	41
2.7.9	MQTT (Message Queuing Telemetry Transport)	42
2.7.10	PlatformIO และการพัฒนา Firmware	43
2.7.11	Home Assistant และการควบคุมอุปกรณ์ IoT	43
2.8	งานวิจัยที่เกี่ยวข้อง	44
บทที่ 3	วิธีการศึกษา	45
3.1	การออกแบบระบบ	45
3.2	การออกแบบเซ็นเซอร์บนบอร์ดอาร์ดูইน	47
3.2.1	ฮาร์ดแวร์	47
3.2.2	โครงข่ายข้อมูล	48
3.2.3	การประมวลผลข้อมูล	49
3.3	อุปกรณ์สวมใส่และสมาร์ทโฟน	51
3.3.1	ฮาร์ดแวร์	51
3.3.2	โครงข่ายข้อมูล	51
3.3.3	การประมวลผลข้อมูล	52
3.4	การจำแนกตำแหน่ง	53
3.4.1	ฮาร์ดแวร์	53
3.4.2	การประมวลผลข้อมูล	55
3.5	เซิร์ฟเวอร์	56
3.5.1	สถาปัตยกรรมโดยรวม	56
3.5.2	แอปพลิเคชันแบ็กเอนด์ FastAPI	57
3.5.3	ช่องทางสื่อสารและการรับข้อมูล	58
3.5.4	ฐานข้อมูล PostgreSQL	59
3.5.5	ระบบควบคุมอุปกรณ์ Home Assistant	60
3.5.6	ระบบรับรองความถูกต้องและการควบคุมสิทธิ์	61
3.6	ระบบปัญญาประดิษฐ์ (AI)	62
3.6.1	สถาปัตยกรรม	62
3.6.2	Prompt	64
3.6.3	องค์ประกอบอื่นๆ	66
3.7	ส่วนติดต่อผู้ใช้งานและ HMI	67
บทที่ 4	ผลการวิจัย	72
4.1	บทนำและภาพรวมการประเมินผล	72
4.2	ผลการประเมินระบบเซ็นเซอร์บนบอร์ดอาร์ดูইน	72
4.2.1	การทดสอบฮาร์ดแวร์และการอ่านค่า	72
4.2.2	การทดสอบโครงข่ายและการเชื่อมโยงข้อมูล	73

4.2.3	ผลการประมวลผลข้อมูล	73
4.3	ผลการประเมินระบบจำแนกตำแหน่ง	74
4.3.1	การทดสอบฮาร์ดแวร์และ Beacon	74
4.3.2	ผลการทดสอบอัลกอริทึม	74
4.4	ผลการประเมินอุปกรณ์สวมใส่และสมาร์ทโฟน	76
4.4.1	การทดสอบฮาร์ดแวร์ Polar Verity Sense	76
4.4.2	การทดสอบการเชื่อมต่อสมาร์ทโฟนและ Gateway	77
4.4.3	ผลการประมวลผลข้อมูลสัญญาณชีพ	77
4.5	ผลการประเมินเซิร์ฟเวอร์ส่วนกลาง	79
4.5.1	ผลการทดสอบสถาปัตยกรรมและการ Deploy	79
4.5.2	ผลการทดสอบช่องทางสื่อสาร	81
4.5.3	ผลการทดสอบฐานข้อมูล	81
4.5.4	ผลการทดสอบ Home Assistant Integration	81
4.5.5	ผลการทดสอบความปลอดภัย	83
4.6	ผลการประเมินระบบผู้ช่วยอัจฉริยะ	83
4.6.1	ผลการทดสอบสถาปัตยกรรม AI	83
4.6.2	ผลการทดสอบ Prompt และ Intent Classification	83
4.6.3	ผลการทดสอบ Propose-Confirm-Execute Flow	87
4.7	ผลการประเมินส่วนติดต่อผู้ใช้งาน	89
4.7.1	ผลการทดสอบแดชบอร์ดเว็บ	89
4.7.2	ผลการทดสอบแอปพลิเคชันบนอุปกรณ์เคลื่อนที่	91
4.8	การทดสอบระบบแบบปลายถึงปลายและกรณีศึกษา	92
4.8.1	สถานการณ์ทดสอบแบบปลายถึงปลาย	92
4.8.2	กรณีศึกษาการใช้งานจริง	93
บทที่ 5	สรุปผลการวิจัย และข้อเสนอแนะ	94
5.1	อภิปรายผลงานวิจัย	94
5.2	สรุปผลงานวิจัย	96
5.3	ข้อจำกัดของงานวิจัย	97
5.4	ข้อเสนอแนะในการวิจัย	97
	รายการอ้างอิง	98
	ภาคผนวก	102
	รายละเอียดบริบทโครงการ	103
ก.1	ปัญหาทางวิศวกรรมที่ซับซ้อน	104
ก.2	องค์ความรู้และเครื่องมืออุปกรณ์ที่ใช้	104
ก.3	มาตรฐาน กฎหมาย และข้อบังคับที่เกี่ยวข้อง	105
ก.4	ผู้มีส่วนได้ส่วนเสีย ผลกระทบ และความยั่งยืน	105
ก.5	การบริหารงานวิศวกรรมและโครงการ	105
	คลังซอร์สโค้ด	107
	แบบประเมินและข้อมูลประกอบการใช้งาน	108

ภาพประกอบการทดลองและการติดตั้ง	109
ข้อมูลประกอบการทดลองระบบ	110
หลักฐานการประเมิน AI และ MCP	111
ข้อมูลประกอบเพิ่มเติมจากชุดทดลอง Paper IEEE	112

สารบัญตาราง

	หน้า
1.1	แผนการดำเนินงานตลอดโครงการวิจัย21
2.1	บทบาทหน้าที่และความรับผิดชอบของบุคลากรในสถานดูแลผู้สูงอายุ22
2.2	การเปรียบเทียบลักษณะการใช้งานและคุณลักษณะหลักของฮาร์ดแวร์ที่เกี่ยวข้อง31
2.3	การทบทวนเทคโนโลยีซอฟต์แวร์และโครงสร้างพื้นฐานพื้นฐานที่สำคัญ35
3.1	คุณสมบัติของไมโครคอนโทรลเลอร์ M5StickC Plus248
3.2	โครงสร้างหัวข้อ MQTT ในระบบ WheelSense49
3.3	คุณสมบัติของอุปกรณ์สวมใส่และสมาร์ทโฟน51
3.4	ตารางการสื่อสารระหว่าง Polar Verity Sense สมาร์ทโฟน และเซิร์ฟเวอร์52
3.5	ข้อกำหนดและแนวทางการติดตั้ง Node Tsimcam54
3.6	มาตรการรองรับกรณีสัญญาณ Beacon ขาดหาย55
3.7	บริการหลักในระบบ WheelSense57
3.8	เปรียบเทียบช่องทางการสื่อสารในระบบ59
3.9	AI Pipeline 5 ชั้นของระบบ WheelSense62
3.10	เครื่องมือ MCP แบ่งตามหมวดหมู่ในระบบ66
3.11	สิทธิ์การเข้าถึงเครื่องมือ AI ตามบทบาทผู้ใช้ (Policy Engine)67
3.12	ขั้นตอนการใช้งานหลักของ dashboard ตามบทบาทผู้ใช้71
4.1	ผลการทำงานของอุปกรณ์บนเก้าอี้รถเข็นและอุปกรณ์สวมใส่ (n=100)73
4.2	ผลการทดสอบแบบแจกแจงตามสถานะเครือข่าย (n=50 ต่อสถานการณ์)73
4.3	ผลการตรวจจับสถานะการเคลื่อนไหว (n=120 นาที)73
4.4	ผลการเปรียบเทียบวิธีการจำแนกตำแหน่งภายในอาคาร (n=100 รอบ)76
4.5	เมตริกซ์ความสับสนแบบย่อของผลการจำแนกตำแหน่ง (KNN $k = 5$)76
4.6	ผลการทดสอบความแม่นยำของอุปกรณ์ Polar Verity Sense (n=2 คน x 30 นาที)77

4.7	ผลการทดสอบการเชื่อมต่อสมาร์ทโฟน Gateway (n=50)	77
4.8	ผลการประมวลผลข้อมูลสัญญาณชีพจากอุปกรณ์สวมใส่ (n=500 ชุด)	77
4.9	ผลความพร้อมใช้งานของบริการหลักหลัง deployment	79
4.10	latency ของเส้นทางข้อมูลหลักในบริการส่วนกลาง (n=1000 คำขอ)	81
4.11	ผลการทดสอบประสิทธิภาพฐานข้อมูล (n=5000 query)	81
4.12	ผลการทดสอบการควบคุมอุปกรณ์ผ่าน Home Assistant (n=100 คำสั่ง)	82
4.13	ผลการทดสอบความปลอดภัยของระบบ	83
4.14	ผลการประเมินประสิทธิภาพของไปป์ไลน์ EaseAI แบ่งตามชั้น (n=36 prompt)	86
4.15	ผลการประเมินความสอดคล้องของการเลือกเครื่องมือ MCP	86
4.16	ผลการประเมินประสบการณ์ผู้ใช้แดชบอร์ด (n=5 ผู้ดูแลจริง + 10 จำลอง)	89
4.17	ผลการทดสอบจำแนกตามภารกิจของ HMI	91
4.18	ผลการทดสอบแบบปลายถึงปลายของระบบ (n=8 สถานการณ์)	92
ก.1	การจัดเข้ากลุ่มปัญหาทางวิศวกรรมที่ซับซ้อนตามกรอบ IEA	104
ค.1	หัวข้อที่ใช้ในแบบประเมิน HMI	108
ง.1	บริบทภาพประกอบที่ใช้ตรวจสอบการติดตั้ง	109
จ.1	ชุดข้อมูลประกอบที่ใช้ตรวจสอบผลการทดลอง	110
ฉ.1	ผลรวมการเลือกเครื่องมือของ AI เทียบกับคำตอบอ้างอิง	111
ฉ.2	เวลาแฝงของ AI pipeline	111
ช.1	จำนวนข้อมูลที่ใช้ในชุดทดลอง RSSI/KNN	112
ช.2	ผลเปรียบเทียบ KNN และ XGBoost ในชุดทดลองเดียวกัน	112
ช.3	Confusion matrix ของ KNN ใน Case 1	112
ช.4	Confusion matrix ของ KNN ใน Case 2	113
ช.5	Confusion matrix รายตำแหน่งของ KNN ใน Case 3	113

สารบัญรูป

	หน้า
1.1 บริบทการดูแลผู้สูงอายุและผู้พิการที่ใช้เก้าอี้รถเข็นในสถานดูแลของประเทศไทย	17
1.2 อาคารและพื้นที่บริการหลักของบ้านพักคนชราวาสนะเวศม์ที่ใช้เป็นบริบทศึกษา	18
1.3 การใช้กระดานบันทึกข้อมูลแบบแอนะล็อกที่พบในหน่วยงานจริง	18
2.1 สถาปัตยกรรม 4 ชั้นของระบบ Ambient Assisted Living (AAL)	23
2.2 หลักการระบุตำแหน่งแบบ Trilateration จากจุดอ้างอิง 3 จุด	24
2.3 ความสัมพันธ์ระหว่างค่า RSSI และระยะทางตามแบบจำลอง Path Loss	25
2.4 ขั้นตอนการระบุตำแหน่งด้วยเทคนิค Fingerprinting	26
2.5 หลัก การ ทำงาน ของ เซ็นเซอร์ PPG แบบ สะท้อน แสง (Reflection Mode) และส่วนประกอบของสัญญาณที่วัดได้ ซึ่งประกอบด้วยส่วนคงที่ (DC) จากเนื้อเยื่อและเลือดดำ และส่วนที่แปรผันตามจังหวะการเต้นของหัวใจ (AC) จากเลือดแดงในหลอดเลือดฝอย	27
2.6 แกนพิกัดและทิศทางการหมุนของ IMU	28
2.7 หลัก การ ของ อัล กอ ริ ทึม KNN ใน การ จำแนก ข้อมูล ตาม ระยะ ห่าง ใน พื้นที่ คุณสมบัติ	30
2.8 ลักษณะของอุปกรณ์ M5StickC Plus2 ที่มีจอแสดงผลและเซ็นเซอร์ภายใน	32
2.9 ลักษณะภายนอกของโหนดกล้อง Lilygo T-SimCam ที่รองรับการประมวลผลภาพเบื้องต้น	33
2.10 ลักษณะเชิงแนวคิดของ Polar Verity Sense และสายรัดสำหรับการสวมใส่	33
2.11 Raspberry Pi 5 ที่ใช้เป็นเซิร์ฟเวอร์ส่วนกลางในระบบ	34
2.12 กระบวนการทำงานของ FastAPI ที่เชื่อมการตรวจสอบข้อมูลเข้ากับเอกสาร API อัตโนมัติ	36
2.13 แนวคิดการทำงานของ React Virtual DOM สำหรับการอัปเดตส่วนติดต่อผู้ใช้แบบมีประสิทธิภาพ	37

2.14	แนวทางการเรนเดอร์และการนำทางของ Next.js สำหรับเว็บแอปพลิเคชันเชิงปฏิบัติการ	38
2.15	แนวทางการใช้ Flutter เป็น mobile gateway สำหรับรวมข้อมูล BLE และส่งข้อมูลขึ้น server	39
2.16	แนวทางการจัดเก็บข้อมูลแบบผสมผสานระหว่างข้อมูลเชิงโครงสร้างและกึ่งโครงสร้าง	40
2.17	การจัดวางบริการหลักของระบบในสภาพแวดล้อมคอนเทนเนอร์	40
2.18	แนวทางการให้บริการโมเดลภาษาขนาดใหญ่ภายในระบบผ่าน Ollama	41
2.19	บทบาทของ MCP ในการเชื่อมโยง LLM เข้ากับแหล่งข้อมูลและเครื่องมือ	42
2.20	รูปแบบการสื่อสารแบบ Publish/Subscribe ของ MQTT ในระบบ IoT	42
2.21	กระบวนการพัฒนาและติดตั้งเฟิร์มแวร์ผ่าน PlatformIO	43
2.22	หน้า จอ หลัก ของ Home Assistant แสดง การ จัดการ อุปกรณ์ IoT และ Automation	44
3.1	โครงสร้างเชิงชั้นของระบบต้นแบบแสดงการไหลของข้อมูลระหว่างชั้นต่างๆ	45
3.2	สถาปัตยกรรมระบบ WheelSense โดยรวมแสดงการเชื่อมโยงระหว่างทุกชั้น	47
3.3	ตำแหน่งการติดตั้งไมโครคอนโทรลเลอร์ M5StickC Plus2 บนเก้าอี้รถเข็น	48
3.4	เส้น ทาง ข้อมูล จาก M5StickC Plus2 ผ่าน Wi-Fi และ MQTT Broker สู่ Backend API	49
3.5	แผนผังแสดงวงรอบการทำงานหลักของเฟิร์มแวร์ M5StickC Plus2	50
3.6	การติดตั้ง Polar Verity Sense และตำแหน่งใช้งานสมาร์ทโฟน	52
3.7	เส้น ทาง ข้อมูล จาก Polar Verity Sense ผ่าน สมาร์ท โฟน สู่ MQTT Broker และ Backend API	52
3.8	แผนผังแสดงวงรอบการทำงานของแอปพลิเคชันบนสมาร์ทโฟน	53
3.9	แผนผังการจัดวาง BLE beacon (12 โซน/ห้อง)	54
3.10	Flowchart การประมวลผลข้อมูลสำหรับการจำแนกตำแหน่ง	56
3.11	สถาปัตยกรรมเชิงหน้าที่ของบริการส่วนกลางแสดงการเชื่อมโยงระหว่างบริการ	57
3.12	แผนผังแสดงกระบวนการรับรองและประมวลผลข้อมูลฝั่งเซิร์ฟเวอร์	59

3.13	แผนภาพความสัมพันธ์ของเอนทิตี (ER Diagram) เบื้องต้นในฐานข้อมูล	60
3.14	แผนผังแสดงกระบวนการประมวลผล 5 ชั้นของ AI Pipeline	63
3.15	workflow การ ใช้ งาน HMI ของ แต่ละ บทบาท ผ่าน frontend, FastAPI server, policy/visibility layer และ service ที่เกี่ยวข้อง	67
3.16	หน้าจอเริ่มต้นของ HMI สำหรับแยกบทบาทผู้ใช้และหน้าจอภาพรวมของผู้ดูแลระบบ	68
3.17	หน้าจอหลักของผู้ดูแลและผู้พักอาศัยที่ใช้ข้อมูลคนละระดับตามบทบาท	69
3.18	หน้าจอ HMI ที่ใช้ตอบสนองเหตุการณ์และตรวจสอบอุปกรณ์ภาคสนาม	69
3.19	หน้าจอที่ใช้เชื่อมบริบทพื้นที่กับ gateway บนโทรศัพท์สำหรับตรวจสอบสถานะ BLE/Polar และการส่งข้อมูลขึ้น server	70
4.1	การติดตั้ง BLE beacon ตามจุดอ้างอิงในอาคาร	74
4.2	หน้าจอแสดงตำแหน่งปัจจุบันของผู้ใช้บนแดชบอร์ด	75
4.3	การเชื่อมต่อ Polar Verity Sense กับ mobile app สำหรับตรวจสอบการทำงาน	78
4.4	หน้าจอ monitoring ของเซิร์ฟเวอร์แสดงสถานะบริการต่างๆ	80
4.5	รายการ container services ใน Docker Compose	80
4.6	แผนผังการเชื่อมต่อระหว่าง WheelSense MCP และ Home Assistant	82
4.7	หน้าตาสนทนากับผู้ช่วย AI บนแดชบอร์ด	84
4.8	ตัวอย่างการเลือกเครื่องมือโดย AI เพื่อตอบคำถาม	85
4.9	กระบวนการยืนยันก่อนดำเนินการ (propose-confirm-execute)	88
4.10	การสอนใช้งาน	89
4.11	หน้าจอแอปพลิเคชันบนอุปกรณ์เคลื่อนที่ (Observer view)	90
ข.1	หน้าจอลังจอร์สโค้ด GitHub ของโครงการ WheelSense	107

สัญลักษณ์และคำย่อ

สัญลักษณ์/คำย่อ	ความหมาย
AAL	Ambient Assisted Living (สภาพแวดล้อมช่วยเหลือนการดำรงชีวิต)
ABAC	Attribute-Based Access Control (การควบคุมการเข้าถึงตามคุณลักษณะ)
ACID	Atomicity, Consistency, Isolation, Durability
ACL	Access Control List (รายการควบคุมการเข้าถึง)
API	Application Programming Interface
BLE	Bluetooth Low Energy (บลูทูธพลังงานต่ำ)
HTTP	Hypertext Transfer Protocol
IMU	Inertial Measurement Unit (หน่วยวัดความเร่ง-หมุน)
IoT	Internet of Things (อินเทอร์เน็ตของสรรพสิ่ง)
IRB	Institutional Review Board (คณะกรรมการจริยธรรมการวิจัย)
IRR	Inter-Rater Reliability (ความสอดคล้องระหว่างผู้ประเมิน)
JSON	JavaScript Object Notation
JSONB	JSON Binary (ของ PostgreSQL)
JWT	JSON Web Token
KNN / k -NN	k -Nearest Neighbors (เพื่อนบ้านใกล้สุด k ตัว)
LLM	Large Language Model (แบบจำลองภาษาขนาดใหญ่)
MCP	Model Context Protocol (โพรโทคอลบริบทสำหรับเครื่องมือของโมเดล)
MQTT	Message Queuing Telemetry Transport
NASA-TLX	NASA Task Load Index
OAuth	Open Authorization (มาตรฐานการอนุญาตแบบเปิด)
PPG	Photoplethysmography (การตรวจวัดสัญญาณชีพด้วยแสง)
QoS	Quality of Service (คุณภาพการบริการของ MQTT)
RBAC	Role-Based Access Control (การควบคุมการเข้าถึงตามบทบาท)
REST	Representational State Transfer
RSSI	Received Signal Strength Indicator (ค่าความแรงสัญญาณที่ได้รับ)
SSE	Server-Sent Events
SUS	System Usability Scale (มาตรวัดความสามารถในการใช้งาน)
TLS	Transport Layer Security

บทที่ 1

บทนำ

1.1 ที่มาและความสำคัญ

จากสถานการณ์คนพิการในประเทศไทย ปี พ.ศ. 2567 [1] พบว่ามีประชากรผู้พิการกว่า 2.2 ล้านคน โดยเฉพาะผู้พิการทางการเคลื่อนไหว ภาครัฐจึงได้มีนโยบายส่งเสริมคุณภาพชีวิตหลายด้าน ทั้งการสร้างอาชีพผ่านรายงานการจ้างงานคนพิการ [2] การสนับสนุนสิทธิและการใช้ชีวิตอิสระตามอนุสัญญาว่าด้วยสิทธิคนพิการ (CRPD) [3] รวมถึงการประยุกต์ใช้เทคโนโลยีเพื่อการดำรงชีวิตอิสระสำหรับผู้สูงอายุและผู้พิการ (AAL) [4] เพื่อลดอุปสรรคในการดำเนินชีวิต นอกจากนี้ยังมีการส่งเสริมการมีส่วนร่วมทางสังคมตามแผนพัฒนาการกีฬาแห่งชาติ ฉบับที่ 6 [5] และการสร้างสภาพแวดล้อมที่ปราศจากอุปสรรคตามแผนพัฒนาคุณภาพชีวิตคนพิการแห่งชาติ ฉบับที่ 5 [6]

ผู้ใช้เก้าอี้รถเข็นและผู้สูงอายุในสถานดูแล เช่น บ้านพักคนชราवासนะเวคม ซึ่งเป็นบริบทศึกษาของโครงการนี้ ต้องพึ่งพาผู้ดูแลและเผชิญข้อจำกัดด้านการเคลื่อนที่และความปลอดภัย



รูปที่ 1.1: บริบทการดูแลผู้สูงอายุและผู้พิการที่ใช้เก้าอี้รถเข็นในสถานดูแลของประเทศไทย

จากการลงพื้นที่สำรวจอาคารและพื้นที่บริการหลัก พบว่ามีผู้พักอาศัยกระจายอยู่ในหลายอาคาร ทำให้ผู้ดูแลไม่สามารถทราบตำแหน่งและสภาพของผู้ใช้เก้าอี้รถเข็นทุกคนได้ทันเวลา ซึ่งอาจนำไปสู่อุบัติเหตุ เช่น การหกล้ม [7]



รูปที่ 1.2: อาคารและพื้นที่บริการหลักของบ้านพักคนชราวาสนะเวชม์ที่ใช้เป็นบริบทศึกษา

นอกจากนี้ การทำงานของผู้ดูแลยังมีข้อจำกัดจากการบันทึกข้อมูลแบบแอนะล็อก เช่น กระดานหรือกระดาษ ซึ่งเสี่ยงต่อการสูญหายและไม่สามารถนำมาวิเคราะห์แนวโน้มสุขภาพได้ ปริมาณงานที่มากยังนำไปสู่ความล้าจากการแจ้งเตือน (alert fatigue) [8]

รายการอาหาร		สถิติผู้พัก		เดือน	บุคคลที่	วันที่
อาหาร	จำนวน	จำนวน	รายชื่อ			
อาหารเช้า	11 (A)	กลุ่ม 1	55			
ข้าวต้ม	9 (A)					
ผลไม้	9 (A)					
ข้าวต้ม	13 (B)	กลุ่ม 2	19			
ผลไม้	8 (B)					
ผลไม้	6 (B)					
ผลไม้	9 (A)	กลุ่ม 3	15			
ผลไม้	10 (A)					
ผลไม้	65 (C)					
ผลไม้	11 (A)	กลุ่ม 4	15			
ผลไม้	15 (A)					
ผลไม้	9 (A)					
ผลไม้	176					
ผลไม้	19					
ผลไม้	25					
ผลไม้	1 = 26					

รูปที่ 1.3: การใช้กระดานบันทึกข้อมูลแบบแอนะล็อกที่พบในหน่วยงานจริง

แม้ปัจจุบันจะมีระบบอัตโนมัติในบ้านและระบบระบุตำแหน่งภายในอาคาร [9], [10], [11] แต่มักไม่สอดคล้องกับพฤติกรรมของผู้ใช้เก้าอี้รถเข็น มีความซับซ้อน หรือมีต้นทุนสูง โครงการนี้จึงเสนอการพัฒนาแพลตฟอร์ม WheelSense ซึ่งเป็นระบบสภาพแวดล้อมอัจฉริยะแบบบูรณาการ ที่เชื่อมโยงอุปกรณ์บนเก้าอี้รถเข็น อุปกรณ์สวมใส่ โหนดภายในอาคาร และเซิร์ฟเวอร์ขอบ เพื่อสนับสนุนการทำงานของผู้นั่งและเพิ่มความปลอดภัยให้แก่ผู้ใช้งานอย่างมีประสิทธิภาพ

1.2 วัตถุประสงค์

1. เพื่อพัฒนาระบบต้นแบบสภาพแวดล้อมอัจฉริยะสำหรับผู้ใช้อัจฉริยะที่สามารถรับข้อมูลการเคลื่อนไหว สัญญาณชีพ และบริบทของพื้นที่ภายในอาคารได้อย่างต่อเนื่อง
2. เพื่อพัฒนาระบบระบุตำแหน่งภายในอาคารระดับโซนหรือห้อง โดยอาศัยสัญญาณ BLE และการประมวลผล บนเซิร์ฟเวอร์
3. เพื่อพัฒนาแพลตฟอร์มซอฟต์แวร์สำหรับจัดเก็บ ประมวลผล และแสดงผลข้อมูลแก่ผู้ใช้งานตามบทบาท ผ่านเว็บ Dashboard และแอปพลิเคชันบนอุปกรณ์เคลื่อนที่
4. เพื่อบูรณาการผู้ช่วยปัญญาประดิษฐ์ภายใต้กรอบเสนอ-ยืนยัน-ดำเนินการ เพื่อช่วยสรุปข้อมูลและ สนับสนุนการตัดสินใจของผู้ดูแลอย่างปลอดภัย
5. เพื่อประเมินระบบต้นแบบในด้านความถูกต้องของการระบุตำแหน่ง ความเสถียรของการไหลของข้อมูล ประสิทธิภาพของส่วนบริการ และประสบการณ์ผู้ใช้ของผู้ดูแล

1.3 ขอบเขตงานวิจัย

1. พัฒนาชุดอุปกรณ์ภาคสนามสำหรับติดตั้งบนเก้าอี้รถเข็นและอุปกรณ์สวมใส่ที่ใช้เก็บข้อมูลการเคลื่อนไหว และสัญญาณชีพพื้นฐาน
2. พัฒนาระบบระบุตำแหน่งภายในอาคารระดับโซนหรือห้อง โดยใช้ข้อมูล BLE RSSI และข้อมูลอ้างอิงจาก พื้นที่ที่กำหนด
3. พัฒนาเว็บ Dashboard และแอปพลิเคชันบนอุปกรณ์เคลื่อนที่สำหรับการใช้งานตามบทบาทของผู้ดูแล ผู้ปฏิบัติงาน และผู้เกี่ยวข้อง
4. บูรณาการผู้ช่วยปัญญาประดิษฐ์แบบภายในระบบ โดยจำกัดการกระทำที่มีผลเปลี่ยนสถานะให้อยู่ภายใต้ ขั้นตอนยืนยันก่อนดำเนินการ
5. ประเมินระบบต้นแบบภายใต้สภาพแวดล้อมทดลองและการใช้งานสาธิตในบริบทสถานดูแลโดยไม่มุ่งหมาย เป็นการทดลองทางคลินิกหรือการรับรองเป็นเครื่องมือแพทย์

1.4 ประโยชน์ที่คาดว่าจะได้รับ

ประโยชน์ต่อสังคม

1. ช่วยยกระดับคุณภาพการดูแลผู้สูงอายุและผู้ใช้เก้าอี้รถเข็นในสถานดูแล โดยเพิ่มความสามารถในการเฝ้าระวัง ความปลอดภัย การติดตามตำแหน่ง และการรับรู้เหตุผิดปกติได้รวดเร็วยิ่งขึ้น
2. ช่วยลดภาระงานซ้ำซ้อนของผู้ดูแลและสนับสนุนการประสานงานข้ามเวรด้วยข้อมูลที่ค้นย้อนกลับได้ ซึ่งเอื้อต่อการจัดการทรัพยากรบุคคลอย่างมีประสิทธิภาพมากขึ้น

องค์ความรู้ที่ได้รับ

1. ได้องค์ความรู้ด้านการออกแบบระบบสภาพแวดล้อมอัจฉริยะที่บูรณาการอุปกรณ์ภาคสนาม บริการส่วนกลาง และส่วนติดต่อผู้ใช้เข้าด้วยกันในบริบทการดูแล
2. ได้องค์ความรู้ด้านการประยุกต์ใช้ผู้ช่วยปัญญาประดิษฐ์อย่างปลอดภัยในงานเชิงปฏิบัติการที่เกี่ยวข้องกับ ข้อมูลผู้พักอาศัยและการตัดสินใจของผู้ดูแล

1.5 แผนการดำเนินงาน

การดำเนินงานวิจัยครอบคลุมระยะเวลา 12 เดือน โดยแบ่งเป็นช่วงการวิเคราะห์ปัญหา การศึกษาเอกสารที่เกี่ยวข้อง การออกแบบและสร้างระบบต้นแบบ การบูรณาการและทดสอบ ตลอดจนการวิเคราะห์ผลและจัดทำเล่มวิทยานิพนธ์ ดังรายละเอียดในตารางที่ 1.1

ตารางที่ 1.1: แผนการดำเนินงานตลอดโครงการวิจัย

กิจกรรม	เดือนที่											
	1	2	3	4	5	6	7	8	9	10	11	12
1. วิเคราะห์ปัญหาและเก็บบริบททำงาน	•	•										
2. ศึกษาวรรณกรรมและทฤษฎีที่เกี่ยวข้อง	•	•	•									
3. ออกแบบสถาปัตยกรรมระบบ		•	•	•								
4. พัฒนาอุปกรณ์บนบอร์ดเซ็นเซอร์และอุปกรณ์ สวมใส่			•	•	•	•						
5. พัฒนาโครงข่ายระบุตำแหน่งภายในอาคาร				•	•	•	•					
6. พัฒนาเซิร์ฟเวอร์และฐานข้อมูล				•	•	•	•	•				
7. พัฒนา HMI และแดชบอร์ด					•	•	•	•	•			
8. บูรณาการ AI และกลไกความปลอดภัย						•	•	•	•			
9. ทดสอบและประเมินผลระบบ								•	•	•		
10. วิเคราะห์ผลและสรุปข้อค้นพบ									•	•	•	
11. จัดทำและปรับปรุงวิทยานิพนธ์										•	•	•

หมายเหตุ: •หมายถึงทำกิจกรรมในเดือนนั้น

บทที่ 2

วรรณกรรมและทฤษฎีที่เกี่ยวข้อง

2.1 บริบทการดูแลผู้สูงอายุและความท้าทายในสถานดูแล

การเปลี่ยนผ่านสู่สังคมสูงวัย (Aging Society) ส่งผลกระทบโดยตรงต่อระบบการจัดการในสถานดูแลผู้สูงอายุ (Nursing Homes หรือ Care Facilities) ซึ่งต้องรองรับผู้รับบริการที่มีความเปราะบางและมีความต้องการที่หลากหลายมากขึ้น [12] ในบริบทของสถานดูแลขนาดกลางถึงใหญ่ เช่น บ้านพักคนชราวาระสนะเวศม์ ซึ่งมีพื้นที่กว้างขวางและมีอาคารพักอาศัยหลายหลัง ความท้าทายที่สำคัญคืออัตราส่วนระหว่างบุคลากรและผู้สูงอายุ โดยในขณะที่สถานดูแลระดับพรีเมียมอาจมีอัตราส่วนที่ 1:1 ถึง 1:4 แต่ในสถานดูแลของรัฐหรือสถานดูแลขนาดใหญ่ทั่วไป อัตราส่วนผู้ดูแลต่อผู้สูงอายุอาจสูงถึง 1:20 โดยเฉพาะในช่วงเวรกลางคืน [13], [14] ซึ่งเป็นตัวเลขที่สะท้อนถึงวิกฤตการขาดแคลนแรงงานในภาคส่วนการดูแลผู้สูงอายุทั้งในระดับประเทศและระดับโลก การมีผู้ดูแลเพียง 1 คนต่อผู้รับบริการ 20 ท่าน ทำให้เกิด "ช่องว่างในการเฝ้าระวัง" (Monitoring Gap) ที่รุนแรง และความเสี่ยงต่อการเกิดอุบัติเหตุหรือภาวะฉุกเฉินที่ไม่ได้รับการตอบสนองทันทั่วถึง

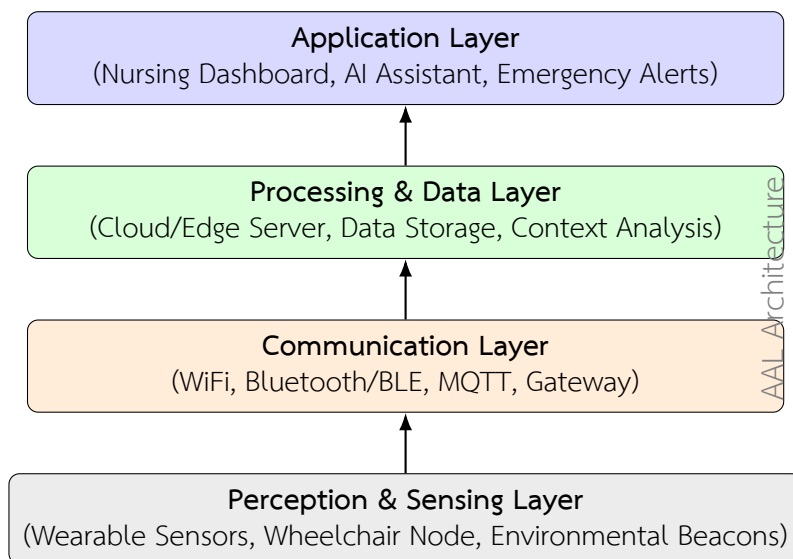
ในการปฏิบัติงานจริง ภายในสถานดูแลมีการแบ่งบทบาทหน้าที่ของบุคลากรอย่างเป็นระบบ โดยหน้าที่หลักสรุปได้ดังตารางที่ 2.1 โดยมีการกำหนดให้ข้อความขีดซ้ายเพื่อให้อ่านง่ายและลดความผิดพลาดในการสื่อสาร:

ตารางที่ 2.1: บทบาทหน้าที่และความรับผิดชอบของบุคลากรในสถานดูแลผู้สูงอายุ

ตำแหน่ง	หน้าที่และความรับผิดชอบหลัก
พยาบาลวิชาชีพ	ประเมินอาการทางคลินิก วางแผนการพยาบาล จัดการยา และตัดสินใจในภาวะฉุกเฉิน
ผู้ช่วยพยาบาล / ผู้ดูแล	ดูแลกิจวัตรประจำวัน (ADL) เช่น การอาบน้ำ การป้อนอาหาร การเคลื่อนย้าย และสังเกตอาการเบื้องต้น
นักกายภาพบำบัด	ฟื้นฟูสมรรถภาพการเคลื่อนไหว ฝึกการเดิน และดูแลการใช้อุปกรณ์ช่วยเดินหรือเก้าอี้รถเข็น
เจ้าหน้าที่ธุรการ / Supervisor	จัดการข้อมูลผู้รับบริการ ตารางเวรบุคลากร และประสานงานระหว่างครอบครัวกับทีมดูแล

2.2 แนวคิด IoT, Smart Environment และ Ambient Assisted Living

เทคโนโลยีสารสนเทศถูกนำมาประยุกต์ใช้เพื่อปิดช่องว่างในการดูแล โดยมีแนวคิดพื้นฐาน 3 ระดับที่เกี่ยวข้องกัน ดังแสดงในรูปที่ 2.1



รูปที่ 2.1: สถาปัตยกรรม 4 ชั้นของระบบ Ambient Assisted Living (AAL)

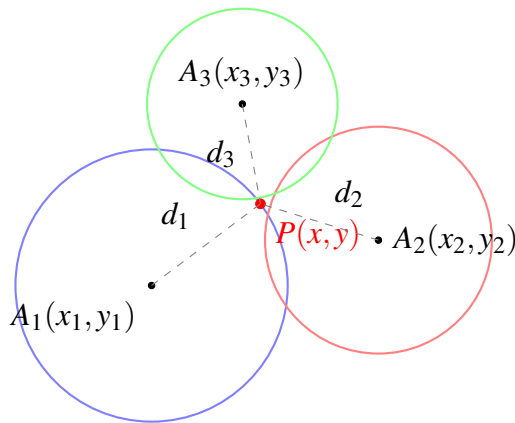
แนวคิด **Ambient Assisted Living (AAL)** คือการบูรณาการเทคโนโลยี **Internet of Things (IoT)** และ **Smart Environment (สภาพแวดล้อมอัจฉริยะ)** เข้าด้วยกัน เพื่อสร้างระบบสนับสนุนการใช้ชีวิตของผู้สูงอายุ โดย IoT ทำหน้าที่เป็นโครงสร้างพื้นฐานในการตรวจวัดและสื่อสารข้อมูลจากอุปกรณ์สวมใส่ [15] ในขณะที่สภาพแวดล้อมอัจฉริยะจะช่วยประมวลผลให้พื้นที่นั้นสามารถรับรู้และตอบสนองต่อพฤติกรรมของมนุษย์ได้อย่างแนบเนียน [16] การผสมผสานรวมกันนี้ทำให้ระบบ AAL สามารถมุ่งเน้นที่ความปลอดภัย ความเป็นส่วนตัว และการเฝ้าระวังแบบไม่ล่วงละเมิดความเป็นส่วนตัว (Non-intrusive) ซึ่งถือเป็นหัวใจสำคัญในการเปลี่ยนผ่านจากการดูแลเชิงรับ (Reactive Care) ไปสู่การดูแลเชิงรุก (Proactive Care) ที่ตอบสนองต่อเหตุฉุกเฉินหรือความผิดปกติได้อย่างทันท่วงที

2.3 ทฤษฎีระบบระบุตำแหน่งภายในอาคาร

เนื่องจากระบบระบุตำแหน่งผ่านดาวเทียม (GPS) มีข้อจำกัดในการใช้งานภายในอาคารเนื่องจากสัญญาณถูกบดบังด้วยโครงสร้างอาคาร ระบบระบุตำแหน่งภายในอาคาร (Indoor Positioning System: IPS) จึงถูกพัฒนาขึ้นเพื่อระบุพิกัดในพื้นที่ปิด เทคนิคการระบุตำแหน่งพื้นฐานที่แพร่หลายมีดังนี้:

2.3.1 เทคนิคเชิงระยะทาง (Range-based Techniques)

อาศัยการคำนวณระยะทางระหว่างตัวรับและตัวส่งสัญญาณ โดยเทคนิคที่แพร่หลายที่สุดคือ **Trilateration** ซึ่งคำนวณจุดตัดของวงกลม 3 วงที่เกิดจากจุดอ้างอิงอ้างอิง 3 จุด ดังรูปที่ 2.2



รูปที่ 2.2: หลักการระบุตำแหน่งแบบ Trilateration จากจุดอ้างอิง 3 จุด

หลักการของ Trilateration คือการหาจุดตัดของวงกลม 3 วง โดยพิกัดเป้าหมาย (x, y) สามารถคำนวณได้จากความสัมพันธ์ระหว่างพิกัดจุดอ้างอิง (x_i, y_i) และระยะทาง d_i ดังสมการ:

$$\text{พิกัด } P(x, y) \text{ คำนวณได้จากระบบสมการ: } \begin{cases} (x - x_1)^2 + (y - y_1)^2 = d_1^2 \\ (x - x_2)^2 + (y - y_2)^2 = d_2^2 \\ (x - x_3)^2 + (y - y_3)^2 = d_3^2 \end{cases} \quad (2.1)$$

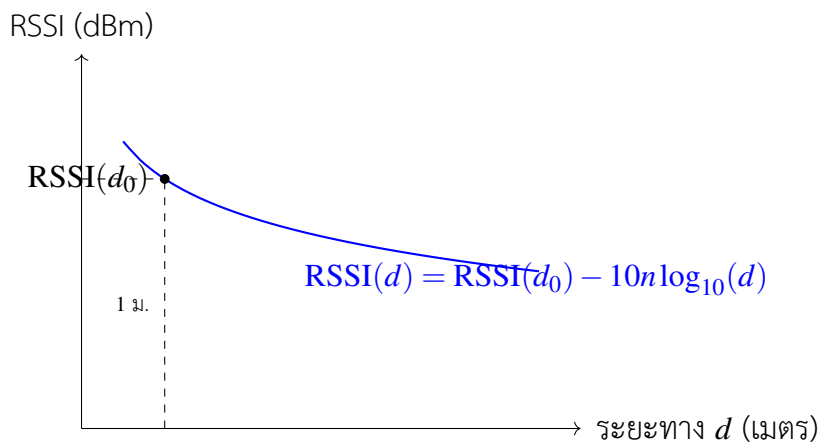
เมื่อ (x_i, y_i) คือพิกัดของจุดอ้างอิงที่ i และ d_i คือระยะทางที่วัดได้จากจุดอ้างอิงนั้นๆ การแก้สมการเพื่อหาพิกัด (x, y) สามารถทำได้โดยการนำสมการที่ 1 ไปลบออกจากสมการที่ 2

และ 3 เพื่อกำจัดตัวแปรยกกำลังสอง จะได้ระบบสมการเชิงเส้นในรูปแบบเมทริกซ์ $AX = B$ ดังสมการ (2.2):

$$\begin{bmatrix} 2(x_2 - x_1) & 2(y_2 - y_1) \\ 2(x_3 - x_1) & 2(y_3 - y_1) \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} (x_2^2 - x_1^2) + (y_2^2 - y_1^2) + (d_1^2 - d_2^2) \\ (x_3^2 - x_1^2) + (y_3^2 - y_1^2) + (d_1^2 - d_3^2) \end{bmatrix} \quad (2.2)$$

ซึ่งฟังก์ชันเป้าหมาย $P(x,y)$ จะคำนวณได้จากการหาอินเวอร์สของเมทริกซ์ A ต่อไป

สำหรับการหาระยะทาง (d) จากความแรงของสัญญาณวิทยุ มักใช้แบบจำลองการลดทอนของสัญญาณ (Log-distance Path Loss Model) ดังรูปที่ 2.3 และสมการ (2.3):



รูปที่ 2.3: ความสัมพันธ์ระหว่างค่า RSSI และระยะทางตามแบบจำลอง Path Loss

$$RSSI(d) = RSSI(d_0) - 10n \log_{10} \left(\frac{d}{d_0} \right) + X_\sigma \quad (2.3)$$

เมื่อ $RSSI(d)$ คือกำลังสัญญาณที่รับที่ระยะ d , $RSSI(d_0)$ คือกำลังสัญญาณอ้างอิงที่ระยะ 1 เมตร, n คือค่าสัมประสิทธิ์การสูญเสียสัญญาณในพื้นที่ และ X_σ คือสัญญาณรบกวนที่มีการกระจายตัวแบบปกติ [17]

2.3.2 เทคนิคการสร้างแผนที่สัญญาณ (Fingerprinting)

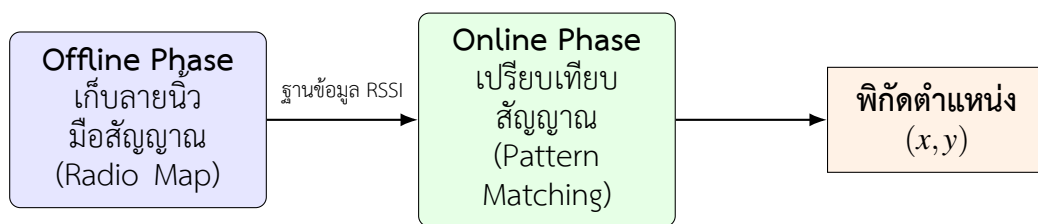
เป็นวิธีที่นิยมใช้ในอาคารที่มีโครงสร้างซับซ้อน แบ่งเป็น 2 ขั้นตอนหลัก ดังแสดงกระบวนการในรูปที่ 2.4:

1. **Offline Phase:** เก็บข้อมูลความแรงสัญญาณ (RSSI) จากตัวส่งสัญญาณทุกจุดในแต่ละฟกััดพื้นที่ เพื่อสร้างเป็น "ลายนิ้วมือสัญญาณ" (Radio Map)
2. **Online Phase:** นำสัญญาณที่วัดได้จริงเปรียบเทียบกับ Radio Map โดยใช้อัลกอริทึม เช่น K-Nearest Neighbors (KNN) เพื่อหาตำแหน่งที่ใกล้เคียงที่สุด

สำหรับการคำนวณความใกล้เคียงระหว่างสัญญาณที่วัดได้แบบเรียลไทม์กับลายนิ้วมือสัญญาณในฐานข้อมูล มักใช้การหาระยะห่างแบบยูคลิด (Euclidean Distance) ดังสมการ [17]:

$$D_j = \sqrt{\sum_{i=1}^N (\text{RSSI}_i - \text{RSSI}_{i,j})^2} \quad (2.4)$$

เมื่อ D_j คือระยะห่างแบบยูคลิดระหว่างสัญญาณที่วัดได้กับลายนิ้วมือสัญญาณที่จุดอ้างอิง j , N คือจำนวนจุดส่งสัญญาณ (Beacon) ทั้งหมดที่ครอบคลุม, RSSI_i คือค่าความแรงสัญญาณจาก Beacon ตัวที่ i ที่วัดได้ในขณะนั้น และ $\text{RSSI}_{i,j}$ คือค่าความแรงสัญญาณจาก Beacon ตัวที่ i ณ จุดอ้างอิง j ในฐานข้อมูล โดยระบบจะเลือกตำแหน่งอ้างอิงที่มีค่า D_j ต่ำที่สุด หรือเฉลี่ยจาก K จุดที่ต่ำที่สุด เพื่อใช้เป็นพิกัดเป้าหมาย



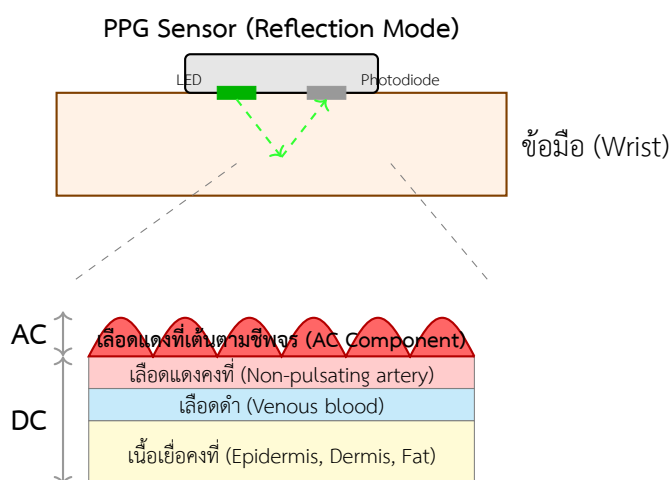
รูปที่ 2.4: ขั้นตอนการระบุตำแหน่งด้วยเทคนิค Fingerprinting

2.4 เทคโนโลยีเซ็นเซอร์สำหรับการตรวจวัดกิจกรรม

เทคโนโลยีเซ็นเซอร์แบบสวมใส่เป็นองค์ประกอบสำคัญในระบบติดตามและดูแลสุขภาพ แบ่งออกเป็นเซ็นเซอร์ทางสรีรวิทยา (Physiological Sensors) และเซ็นเซอร์ทางกายภาพ (Inertial Sensors)

2.4.1 Photoplethysmography (PPG)

เซ็นเซอร์ PPG ตรวจวัดการเปลี่ยนแปลงของปริมาตรเลือดในหลอดเลือดฝอยด้วยการฉายแสงและวัดการสะท้อน ข้อมูลหลักที่ได้คืออนุกรมเวลาของช่วงระหว่างจังหวะหัวใจ (RR-interval) ซึ่งนำไปคำนวณค่าต่างๆ ดังนี้ [18]



รูปที่ 2.5: หลักการทำงานของเซ็นเซอร์ PPG แบบสะท้อนแสง (Reflection Mode) และส่วนประกอบของสัญญาณที่วัดได้ ซึ่งประกอบด้วยส่วนคงที่ (DC) จากเนื้อเยื่อและเลือดดำ และส่วนที่แปรผันตามจังหวะการเต้นของหัวใจ (AC) จากเลือดแดงในหลอดเลือดฝอย

(1) อัตราการเต้นของหัวใจ (Heart Rate):

$$HR = \frac{60}{\overline{RR}} \quad (2.5)$$

เมื่อ $\overline{RR}$ คือค่าเฉลี่ยของช่วง RR-interval (วินาที) และ HR มีหน่วยเป็น bpm (ครั้งต่อนาที)

(2) ความแปรปรวนของอัตราการเต้นของหัวใจ (HRV) ด้วย SDNN:

$$SDNN = \sqrt{\frac{1}{N-1} \sum_{i=1}^N (RR_i - \overline{RR})^2} \quad (2.6)$$

เมื่อ N คือจำนวนช่วง RR ทั้งหมด และ RR_i คือช่วง RR ที่ i ค่า SDNN ที่ต่ำลงบ่งบอกถึงระบบประสาทอัตโนมัติที่ตอบสนองน้อยลง

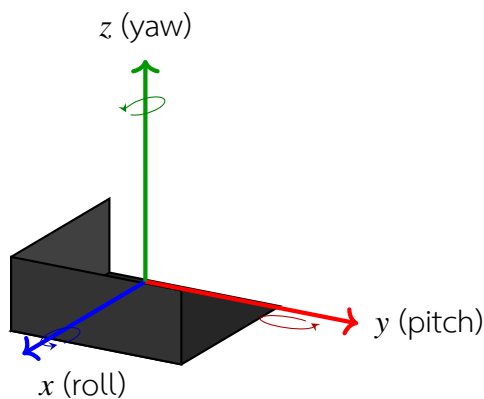
(3) RMSSD สำหรับประเมินกิจกรรมประสาทพาราซิมพาเทติก:

$$\text{RMSSD} = \sqrt{\frac{1}{N-1} \sum_{i=1}^{N-1} (RR_{i+1} - RR_i)^2} \quad (2.7)$$

ค่า RMSSD นิยมใช้ตรวจจับความเครียดทางกายภาพ (Physical Stress) เพราะสะท้อนการควบคุมอัตราการเต้นของหัวใจในช่วงสั้น

2.4.2 Inertial Measurement Unit (IMU)

เซ็นเซอร์ IMU ประกอบด้วย Accelerometer วัดความเร่งในสามแกน (a_x, a_y, a_z) และ Gyroscope วัดอัตราการหมุน ($\omega_x, \omega_y, \omega_z$) อุปกรณ์ที่มีชิป IMU เช่น M5StickC Plus2 ซึ่งใช้ชิป MPU6886 (6-axis) สามารถนำมาติดตั้งบนล้อรถเข็น เพื่อให้ Gyroscope แกน Z (ω_z) วัดอัตราการหมุนได้โดยตรง ดังแสดงในรูปที่ 2.6 [19]



รูปที่ 2.6: แกนพิกัดและทิศทางการหมุนของ IMU

ข้อมูลจาก IMU สามารถนำไปประมวลผลได้ดังนี้

(1) ขนาดของความเร่งสุทธิ (Resultant Acceleration):

$$|\vec{a}| = \sqrt{a_x^2 + a_y^2 + a_z^2} \quad (2.8)$$

ค่านี้ใช้เป็นฟีเจอร์หลักสำหรับการตรวจจับการล้ม เนื่องจากการล้มจะทำให้เกิดการเปลี่ยนแปลงของ $|\vec{a}|$ อย่างรวดเร็ว

(2) การลด Noise จากแรงโน้มถ่วงด้วย Low-pass Filter (EMA):

$$\hat{\omega}_{z,k} = \alpha \cdot \omega_{z,k} + (1 - \alpha) \cdot \hat{\omega}_{z,k-1} \quad (2.9)$$

เมื่อ $\omega_{z,k}$ คืออัตราการหมุนดิบจาก Gyroscope ณ เวลา k , $\hat{\omega}_{z,k}$ คืออัตราการหมุนที่ผ่านการกรองแล้ว และ α คือค่าคงที่ของ Exponential Moving Average (EMA) โดยทั่วไปอาจกำหนด $\alpha = 0.3$ เพื่อลด noise ของ MPU6886 ในแนวแกน Z ซึ่งมีผลจากแรงโน้มถ่วงเมื่อทิศทางของเซ็นเซอร์เปลี่ยนไปตามการหมุนของล้อ นอกจากนี้ยังมีการกำหนด Deadband ที่ $|\hat{\omega}_z| < 5$ dps เพื่อตัด drift ขณะหยุดนิ่ง

(3) การคำนวณระยะทางจาก Arc Length ของล้อ:

$$\Delta s_k = |\hat{\omega}_{z,k} \cdot \Delta t| \cdot r \quad (2.10)$$

เมื่อ Δs_k คือระยะทางที่ล้อเคลื่อนที่ในช่วงเวลา Δt (วินาที), r คือรัศมีล้อรถเข็น (เช่น $r = 0.30$ เมตร สำหรับรถเข็นทั่วไป) และระยะทางสะสมทั้งหมดคือ $s = \sum_k \Delta s_k$ สมการนี้ใช้หลักการ Arc Length ($s = r\theta$) โดยที่มุมการหมุน $\Delta\theta_k = \hat{\omega}_{z,k} \cdot \Delta t$

(4) การคำนวณความเร็วด้วย Sliding Window:

$$v = \frac{\sum_{k \in W} \Delta s_k}{T_W} \quad (2.11)$$

เมื่อ W คือช่วงหน้าต่างเวลา (Sliding Window) ขนาด $T_W = 500$ มิลลิวินาที และ v คือความเร็วเฉลี่ยในหน้าต่างนั้น วิธีนี้ช่วยลดผลของ noise ที่สะสมจากการ integrate โดยตรง

(5) การตรวจจับการล้ม (Fall Detection) ด้วย Threshold:

$$\text{Fall} = \begin{cases} 1 & \text{ถ้า } |\vec{a}| > T_{\text{fall}} \text{ และ } |\vec{a}| < T_{\text{impact}} \text{ ภายหลัง} \\ 0 & \text{อื่นๆ} \end{cases} \quad (2.12)$$

โดย T_{fall} คือเกณฑ์ความเร่งระหว่างการร่วงหล่น (Free-fall threshold, เช่น $< 0.5g$) และ T_{impact} คือเกณฑ์ความเร่งขณะกระทบ (Impact threshold, เช่น $> 2g$) การตรวจพบทั้งสองเหตุการณ์ภายในช่วงเวลาสั้นจะถูกวินิจฉัยว่าเป็นการล้ม

2.5 อัลกอริทึม K-Nearest Neighbors (KNN)

อัลกอริทึม K-Nearest Neighbors (KNN) เป็นหนึ่งในเทคนิคการเรียนรู้แบบมีการสอน (Supervised Learning) ที่เรียบง่ายและมีประสิทธิภาพสูงสำหรับการจำแนกประเภท (Classification) และการประมาณค่า (Regression) [20] โดยหลักการทำงานพื้นฐานคือการคำนวณความคล้ายคลึงระหว่างข้อมูลใหม่และข้อมูลในฐานข้อมูลผ่านฟังก์ชันระยะทาง (Distance Function) และพิจารณา "เพื่อนบ้าน" ที่อยู่ใกล้เคียงที่สุดจำนวน K ตัวในพื้นที่ของคุณลักษณะ (Feature Space) เพื่อตัดสินใจเลือกคลาสที่มีจำนวนมากที่สุด ดังแสดงในรูปที่ 2.7

ขั้นตอนการคำนวณของ KNN ประกอบด้วยสมการหลักดังนี้:

(1) การคำนวณระยะทางระหว่างข้อมูล (Distance Calculation): มักนิยมใช้ระยะทางแบบยูคลิด (Euclidean Distance) เพื่อวัดความห่างระหว่างจุดข้อมูลใหม่ X และจุดข้อมูลในฐานข้อมูล Y ในพื้นที่ n มิติ:

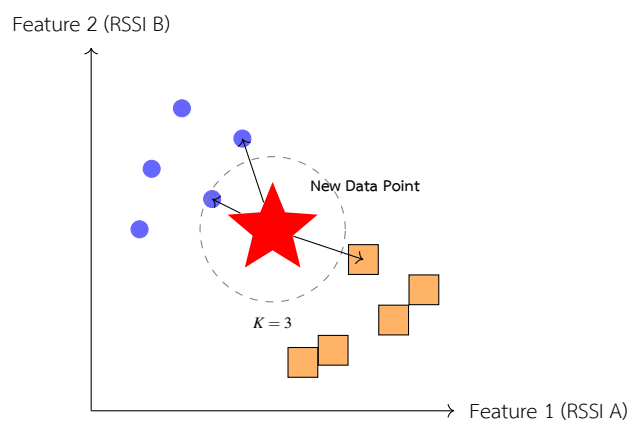
$$d(X, Y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2} \quad (2.13)$$

เมื่อ x_i และ y_i คือค่าของคุณลักษณะ (Feature) ตัวที่ i ของข้อมูล X และ Y ตามลำดับ

(2) การจำแนกประเภทข้อมูล (Classification Rule): หลังจากหาระยะทางและเลือกเพื่อนบ้านที่ใกล้ที่สุด K อันดับแรก (N_K) ระบบจะทำการตัดสินใจเลือกคลาส C โดยใช้เสียงส่วนใหญ่ (Majority Voting):

$$\hat{y} = \text{mode}\{y_i \mid i \in N_K\} \quad (2.14)$$

เมื่อ $\hat{y}$ คือคลาสที่ทำนายได้ และ y_i คือคลาสของเพื่อนบ้านลำดับที่ i



รูปที่ 2.7: หลักการของอัลกอริทึม KNN ในการจำแนกข้อมูลตามระยะทางในพื้นที่คุณลักษณะ

ในการประยุกต์ใช้สำหรับการระบุตำแหน่งแบบ Fingerprinting อัลกอริทึม KNN จะเปรียบเทียบค่า RSSI ที่วัดได้จริงกับข้อมูลใน Radio Map โดยใช้การคำนวณระยะทางแบบยูคลิด (Euclidean Distance) ตามสมการที่ (2.4) เพื่อเลือกจุดอ้างอิงที่มีความคล้ายคลึงของสัญญาณมากที่สุด K จุด และประมาณพิกัดตำแหน่งเป็นค่าเฉลี่ยของจุดเหล่านั้น (Weighted KNN)

2.6 อุปกรณ์ฮาร์ดแวร์และเครื่องมือที่เกี่ยวข้อง

ในการพัฒนาเทคโนโลยีเพื่อการติดตามและดูแลพฤติกรรม การเลือกใช้อุปกรณ์ฮาร์ดแวร์ที่มีความเหมาะสมทั้งในด้านสมรรถนะการประมวลผล การใช้พลังงาน และขนาดของอุปกรณ์ถือเป็นปัจจัยสำคัญ ในปัจจุบันอุปกรณ์ประเภทไมโครคอนโทรลเลอร์ที่มีความสามารถสูงและการประมวลผลที่ขอบเครือข่าย (Edge Computing) เริ่มมีบทบาทมากขึ้นในการลดภาระของเซิร์ฟเวอร์ส่วนกลาง ในส่วนนี้จะเป็นการทบทวนคุณลักษณะและขีดความสามารถของอุปกรณ์ฮาร์ดแวร์หลักที่มักถูกนำมาประยุกต์ใช้ในงานด้าน IoT และการติดตามสภาวะสุขภาพ

ตารางที่ 2.2: การเปรียบเทียบลักษณะการใช้งานและคุณลักษณะหลักของฮาร์ดแวร์ที่เกี่ยวข้อง

อุปกรณ์	ลักษณะการใช้งานทั่วไป	คุณลักษณะเด่น	จุดเด่นเชิงเทคนิค
M5StickC Plus2	อุปกรณ์ควบคุมขนาดเล็กจิ๋ว (Mini IoT Kit) และโน้ตเซ็นเซอร์เคลื่อนที่	ESP32-PICO-V3-02, จอสี LCD, มี IMU 6 แกนในตัว, แบตเตอรี่ในตัว	ขนาดกะทัดรัด พร้อมใช้งานทันที (All-in-one design)
Lilygo T-SimCam	โน้ตกล้องประมวลผลที่ขอบเครือข่าย (Edge AI Camera)	ESP32-S3, กล้อง 2MP, มี PSRAM ขนาดใหญ่, รองรับ Wi-Fi/BLE 5	มีชิปที่รองรับชุดคำสั่งเร่งความเร็วสำหรับงาน AI
Polar Verity Sense	เซ็นเซอร์สวมใส่สำหรับติดตามอัตราการเต้นหัวใจ	วัดด้วยแสง (PPG), กันน้ำ, รองรับ BLE/ANT+, มีหน่วยความจำในตัว	ความแม่นยำระดับมาตรฐานการกีฬาและการแพทย์

2.6.1 M5StickC Plus2

M5StickC Plus2 เป็นชุดพัฒนา IoT ขนาดจิ๋วจากตระกูล M5Stack ที่รวมองค์ประกอบพื้นฐานที่จำเป็นสำหรับการสร้างอุปกรณ์สวมใส่หรือโน้ตเซ็นเซอร์ไร้สายไว้ในตัวเดียว ข้อมูลทางเทคนิคระบุว่าอุปกรณ์ใช้ชิป ESP32-PICO-V3-02 (Dual-core 240 MHz) มีหน่วยความจำ Flash 8 MB, PSRAM 2 MB และจอภาพ TFT ขนาด 1.14 นิ้ว [19] จุดเด่นสำคัญคือการติดตั้งเซ็นเซอร์วัดการเคลื่อนไหวแบบ 6 แกน (6-axis IMU) รุ่น MPU6886 มาให้ในบอร์ด พร้อมปุ่มกด ลำโพงขนาดเล็ก และแบตเตอรี่ Li-Po ในตัว

ด้วยลักษณะที่กะทัดรัดและรองรับการสื่อสารทั้ง Wi-Fi และ Bluetooth อุปกรณ์นี้จึงนิยมใช้ในงานต้นแบบระบบติดตามตำแหน่ง การตรวจวัดกิจกรรมทางกายภาพ และการควบคุมอุปกรณ์อัจฉริยะจากระยะไกล นอกจากนี้ยังมีพอร์ตขยาย (Grove port) ที่ช่วยให้สามารถเชื่อมต่อกับเซ็นเซอร์ภายนอกเพิ่มเติมได้ง่าย ทำให้เป็นเครื่องมือที่มีความยืดหยุ่นสูงสำหรับการพัฒนาแอปพลิเคชัน IoT ที่ต้องการการตอบสนองแบบเรียลไทม์

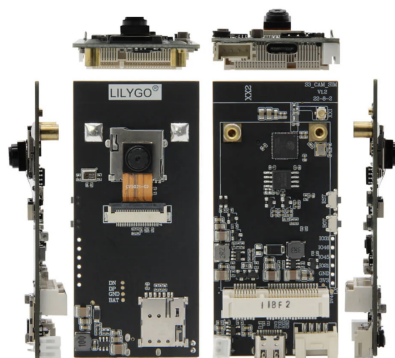


รูปที่ 2.8: ลักษณะของอุปกรณ์ M5StickC Plus2 ที่มีจอแสดงผลและเซ็นเซอร์ภายใน

2.6.2 Lilygo T-SimCam

Lilygo T-SimCam เป็นบอร์ดไมโครคอนโทรลเลอร์ที่ออกแบบมาเฉพาะสำหรับการประมวลผลภาพที่ขอบเครือข่าย (Visual Edge Computing) โดยใช้ชิปหลักเป็น ESP32-S3R8 ซึ่งมีความโดดเด่นในการรองรับชุดคำสั่งสำหรับการคำนวณแบบเวกเตอร์เพื่อเร่งความเร็วงานด้าน AI อุปกรณ์ประกอบด้วยหน่วยความจำ PSRAM ขนาด 8 MB และ Flash 16 MB พร้อมกล้อง OV2640 ความละเอียด 2 ล้านพิกเซล [21]

คุณลักษณะเด่นของอุปกรณ์คือความสามารถในการบันทึกภาพและการสื่อสารไร้สายความเร็วสูง (Wi-Fi และ Bluetooth 5) ทำให้สามารถทำหน้าที่เป็นโหนดเฝ้าระวังอัจฉริยะที่สามารถคัดกรองเหตุการณ์จากภาพ (Image-based triggering) ได้ในตัวโดยไม่ต้องส่งข้อมูลวิดีโอทั้งหมดไปยังเซิร์ฟเวอร์ นอกจากนี้ยังมีช่องเสียบการ์ดหน่วยความจำและการรองรับไมโครโฟนดิจิทัล ทำให้เป็นฮาร์ดแวร์ที่เหมาะสมสำหรับการพัฒนาระบบรักษาความปลอดภัย หรือระบบตรวจจับวัตถุเบื้องต้นที่มีข้อจำกัดด้านพลังงานและพื้นที่



รูปที่ 2.9: ลักษณะภายนอกของโมดูลกล้อง Lilygo T-SimCam ที่รองรับการประมวลผลภาพเบื้องต้น

2.6.3 Polar Verity Sense

Polar Verity Sense เป็นเซ็นเซอร์วัดอัตราการเต้นของหัวใจแบบออปติคัล (Optical Heart Rate Sensor) ที่ใช้เทคโนโลยี PPG (Photoplethysmography) ที่มีความแม่นยำสูง อุปกรณ์ถูกออกแบบมาให้เป็นแขนรัดหรือสวมใส่ในตำแหน่งที่ใกล้กับเส้นเลือดฝอยบนผิวหนัง รองรับการสื่อสารผ่าน Bluetooth Low Energy (BLE) และ ANT+ ได้พร้อมกัน และมีความสามารถในการทำงานต่อเนื่องสูงสุดถึง 30 ชั่วโมง [18], [22]

จุดเด่นของอุปกรณ์รุ่นนี้คือความน่าเชื่อถือของข้อมูลชีวสัญญาณและการออกแบบให้มีความทนทาน กันน้ำ และสามารถเก็บข้อมูลได้ในตัวแม้ไม่มีการเชื่อมต่อกับโทรศัพท์สมาร์ทโฟนในขณะนั้น นอกจากนี้ยังมีเซ็นเซอร์วัดการเคลื่อนไหว (Inertial Sensors) ภายในเพื่อใช้ชดเชยสัญญาณรบกวนจากการเคลื่อนที่ (Motion Artifact) ทำให้ผลลัพธ์ของอัตราการเต้นของหัวใจมีความเสถียรมากขึ้น ในขณะที่ผู้ใช้งานมีการเคลื่อนไหวร่างกายอย่างต่อเนื่อง อุปกรณ์นี้จึงถูกนำไปใช้แพร่หลายทั้งในการวิจัยทางการกีฬาและการติดตามสภาวะสุขภาพทางไกล



รูปที่ 2.10: ลักษณะเชิงแนวคิดของ Polar Verity Sense และสายรัดสำหรับการสวมใส่

2.6.4 Raspberry Pi 5

Raspberry Pi 5 เป็นคอมพิวเตอร์บอร์ดขนาดเล็ก (Single Board Computer) รุ่นล่าสุดจาก Raspberry Pi Foundation ที่ใช้ชิป ARM Cortex-A76 Quad-core ความถี่ 2.4 GHz มีหน่วยความจำ RAM 8 GB และรองรับการเชื่อมต่อผ่าน Gigabit Ethernet, Wi-Fi 5, Bluetooth 5.0 และ USB 3.0 [23] จุดเด่นของรุ่นนี้คือประสิทธิภาพการประมวลผลที่เพิ่มขึ้นอย่างมากเมื่อเทียบกับรุ่นก่อนหน้า ทำให้เหมาะสำหรับการใช้งานเป็นเซิร์ฟเวอร์ขนาดเล็กที่ต้องการการประมวลผลข้อมูลแบบเรียลไทม์และบริการต่างๆ พร้อมกัน

ด้วยขนาดที่กะทัดรัดและการใช้พลังงานที่ต่ำ Raspberry Pi 5 จึงเป็นทางเลือกที่น่าสนใจสำหรับการทำระบบเซิร์ฟเวอร์ส่วนกลางในโครงการ IoT ที่ต้องการความสามารถในการประมวลผลที่สูง แต่ยังคงความยืดหยุ่นในการติดตั้งและบำรุงรักษา อุปกรณ์นี้สามารถรันบริการต่างๆ เช่น MQTT Broker, Database Server, API Server และ AI Runtime ได้อย่างมีประสิทธิภาพในบอร์ดเดียว



รูปที่ 2.11: Raspberry Pi 5 ที่ใช้เป็นเซิร์ฟเวอร์ส่วนกลางในระบบ

2.7 เทคโนโลยีซอฟต์แวร์และโครงสร้างพื้นฐานที่เกี่ยวข้อง

ความสำเร็จของการพัฒนาระบบ IoT และการติดตามพฤติกรรมไม่ได้ขึ้นอยู่กับฮาร์ดแวร์เพียงอย่างเดียว แต่ยังรวมถึงการบูรณาการซอฟต์แวร์ในหลายชั้น ตั้งแต่ส่วนติดต่อผู้ใช้ที่ช่วยสื่อสารข้อมูลไปยังผู้ดูแล การประมวลผลข้อมูลแบบเรียลไทม์ การจัดเก็บข้อมูลที่มีความน่าเชื่อถือ ไปจนถึงการนำเทคโนโลยีปัญญาประดิษฐ์สมัยใหม่มาใช้ในการตีความสถานการณ์ ในส่วนนี้จะเป็นการทบทวนเทคโนโลยีและเครื่องมือซอฟต์แวร์หลักที่ถูกนำมาใช้เป็นพื้นฐานในการพัฒนาระบบ

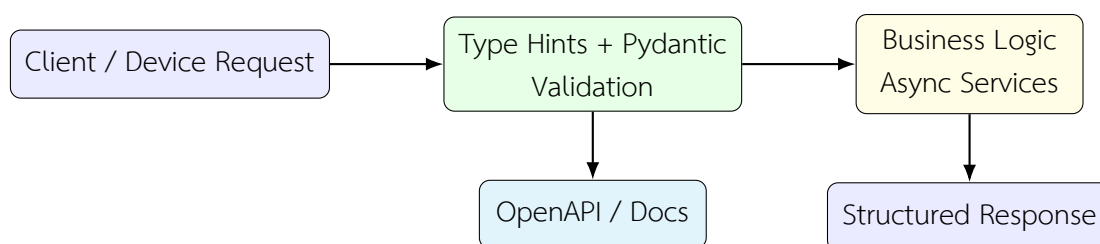
ตารางที่ 2.3: การทบทวนเทคโนโลยีซอฟต์แวร์และโครงสร้างพื้นฐานพื้นฐานที่สำคัญ

ชั้นเทคโนโลยี	เทคโนโลยีหลัก	บทบาทและหน้าที่สำคัญ
ส่วนติดต่อผู้ใช้	UX/UI, Web Application, React, Next.js	การแสดงผลแดชบอร์ดและการโต้ตอบกับผู้ใช้แบบเรียลไทม์
บริการแอปพลิเคชัน	FastAPI	การจัดการตรรกะทางธุรกิจ (Business Logic) และการให้บริการ API
การจัดเก็บข้อมูล	PostgreSQL	การจัดเก็บข้อมูลเชิงสัมพันธ์และข้อมูลกึ่งโครงสร้าง (JSONB) อย่างมีประสิทธิภาพ
การสื่อสารอุปกรณ์	MQTT	โปรโตคอลน้ำหนักเบาสำหรับการรับส่งข้อมูลระหว่างอุปกรณ์ IoT และเซิร์ฟเวอร์
สภาพแวดล้อมรันไทม์	Docker	การสร้างสภาพแวดล้อมจำลอง (Containerization) เพื่อความสม่ำเสมอของระบบ
ปัญญาประดิษฐ์	Ollama, gemma4:e2b, MCP	การประมวลผลโมเดลภาษาขนาดใหญ่ (LLM) ในระดับขอบเครือข่ายหรือเซิร์ฟเวอร์ท้องถิ่น
การพัฒนาเฟิร์มแวร์	PlatformIO	เครื่องมือสำหรับการจัดการวงจรการพัฒนาระบบของไมโครคอนโทรลเลอร์

2.7.1 FastAPI (Backend Framework)

FastAPI เป็นกรอบงานพัฒนาเว็บ API ที่เน้นประสิทธิภาพ ความชัดเจนของชนิดข้อมูล และการทำงานบนมาตรฐานสมัยใหม่ โดยอาศัยมาตรฐาน OpenAPI และ JSON Schema สำหรับ นิยาม API และสนับสนุนการสร้างเอกสารอัตโนมัติผ่าน Swagger UI และ ReDoc [24] คุณสมบัติ นี้มีประโยชน์โดยตรงต่อการพัฒนาระบบขนาดใหญ่ที่ต้องมีการเชื่อมต่อระหว่าง Mobile App, Firmware Gateway และบริการอื่น ๆ ร่วมกันอย่างสอดคล้อง

อีกประเด็นสำคัญคือ FastAPI อาศัย Python Type Hints และ Pydantic ในการตรวจสอบข้อมูล ทำให้สามารถกำหนดสัญญา (Contract) ระหว่าง Input และ Output ได้ชัดเจนตั้งแต่ ระดับโค้ดจนถึงเอกสาร API [24], [25] สำหรับระบบ IoT ที่ต้องรับข้อมูลจากแหล่งที่หลากหลาย ความสามารถในการตรวจสอบความถูกต้องของข้อมูลตั้งแต่ขอบเขตของระบบ (System Boundary) ช่วยลดโอกาสเกิดความผิดพลาดในการประมวลผลข้อมูลที่ผิดรูปแบบได้

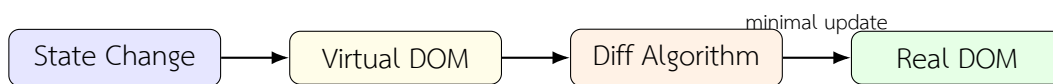


รูปที่ 2.12: กระบวนการทำงานของ FastAPI ที่เชื่อมการตรวจสอบข้อมูลเข้ากับเอกสาร API อัตโนมัติ

2.7.2 React (UI Library)

React เป็นไลบรารีจาวาสคริปต์ (JavaScript Library) แบบโอเพนซอร์สที่ใช้สำหรับการสร้างส่วนติดต่อผู้ใช้ (User Interface) โดยอาศัยสถาปัตยกรรมแบบคอมโพเนนต์ (Component-based Architecture) [26] แนวคิดหลักของ React คือการแบ่งหน้าจอ UI ออกเป็นส่วนย่อย ๆ ที่สามารถนำกลับมาใช้ใหม่ได้ (Reusable Components) และมีการจัดการสถานะ (State Management) ที่แยกเป็นอิสระต่อกัน

คุณสมบัติที่สำคัญของ React คือการใช้ Virtual DOM ซึ่งเป็นตัวแทนของ DOM (Document Object Model) ในหน่วยความจำ เมื่อมีการเปลี่ยนแปลงสถานะข้อมูล React จะคำนวณความแตกต่างระหว่าง Virtual DOM เดิมและโครงสร้างใหม่ แล้วอัปเดตเฉพาะส่วนที่เปลี่ยนแปลงไปยัง DOM จริงบนเบราว์เซอร์ ซึ่งช่วยเพิ่มประสิทธิภาพในการแสดงผลข้อมูลที่มีการเปลี่ยนแปลงบ่อยครั้ง เช่น แดชบอร์ดติดตามสถานะแบบเรียลไทม์ ดังแสดงในรูปที่ 2.13

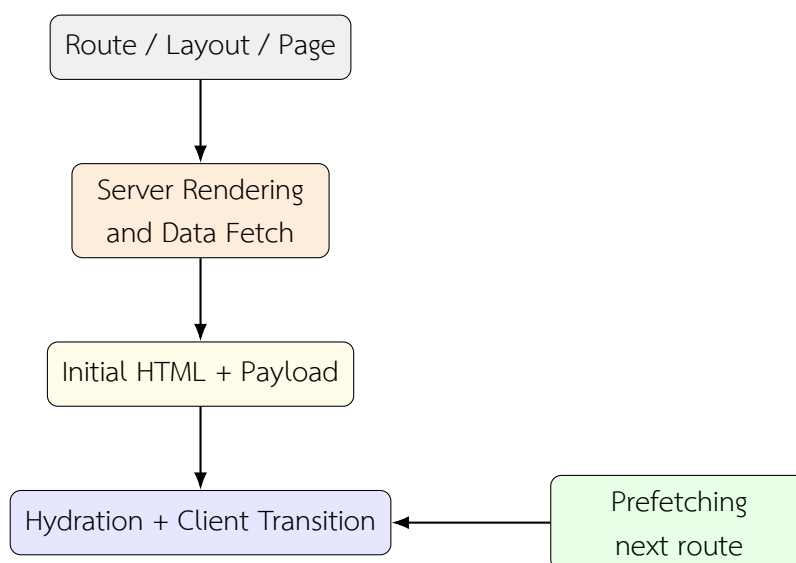


รูปที่ 2.13: แนวคิดการทำงานของ React Virtual DOM สำหรับการอัปเดตส่วนติดต่อผู้ใช้แบบมีประสิทธิภาพ

2.7.3 Next.js (Frontend Framework)

Next.js เป็นเฟรมเวิร์กบน React ที่เหมาะกับการพัฒนาเว็บแอปพลิเคชันสมัยใหม่ที่ต้องการทั้งประสิทธิภาพและความยืดหยุ่น โดยเฉพาะในส่วนของ App Router ที่มีการใช้ React Server Components เป็นพื้นฐาน พร้อมรองรับความสามารถด้าน prefetching, streaming และ client-side transitions ที่ทำให้การสลับมุมมองหน้าจามีความรวดเร็วและลื่นไหล [27], [28]

คุณลักษณะเหล่านี้มีความสำคัญต่อการแสดงผลข้อมูลที่มีความซับซ้อนและเปลี่ยนแปลงตลอดเวลา การใช้ Server Rendering ช่วยลดระยะเวลาการแสดงผลครั้งแรกของผู้ใช้ (Time to First Meaningful Paint) ขณะที่ระบบการนำทางที่รวดเร็วช่วยให้ผู้ใช้งานสามารถเข้าถึงข้อมูลเชิงลึกจากหน้าภาพรวมได้อย่างทันท่วงที ซึ่งเป็นหัวใจสำคัญของระบบแดชบอร์ดแบบปฏิบัติการ (Operational Dashboard)



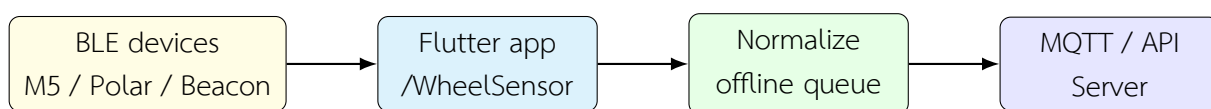
รูปที่ 2.14: แนวคิดการเรนเดอร์และการนำทางของ Next.js สำหรับเว็บแอปพลิเคชันเชิงปฏิบัติการ

2.7.4 Flutter และ Dart (Mobile Development Framework)

แอปพลิเคชันบนอุปกรณ์เคลื่อนที่ในระบบนี้ใช้ Flutter และภาษา Dart เป็นฐานหลัก เนื่องจากต้องทำงานเป็น mobile gateway สำหรับอ่านสถานะ BLE ของ M5StickC Plus2, Polar Verity Sense และ BLE beacons ก่อนส่งข้อมูลขึ้น server ผ่าน MQTT หรือ API Flutter ช่วยให้พัฒนา UI และ logic สำหรับ Android ได้ค่อนข้างเร็ว ใช้ codebase เดียว และควบคุมหน้าจอ /

WheelSensor ให้เหมาะกับการใช้งานภาคสนาม เช่น ตรวจสอบสถานะการ pair อุปกรณ์ ดูค่า sensor ล่าสุด และตรวจว่าข้อมูลถูกส่งขึ้น server แล้วหรือยัง

ในเชิงทฤษฎี Flutter ทำงานผ่าน widget tree และ rendering engine ของตนเอง ทำให้ UI แสดงผลค่อนข้างสม่ำเสมอระหว่างอุปกรณ์ ส่วน Dart เหมาะกับการทำงาน asynchronous เช่น BLE scan, MQTT publish, offline queue และ reconnect state เพราะสามารถจัดการ event stream ได้ชัดเจน แนวทางนี้สอดคล้องกับบทบาทของโทรศัพท์ในระบบที่ไม่ได้เป็นเพียงหน้าจอแสดงผล แต่เป็น gateway หลักที่รวมข้อมูลจากอุปกรณ์ใกล้เคียงตัวก่อนส่งให้ backend ตรวจสอบและบันทึกต่อไป

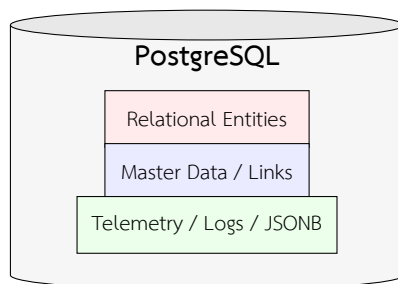


รูปที่ 2.15: แนวคิดการใช้ Flutter เป็น mobile gateway สำหรับรวมข้อมูล BLE และส่งข้อมูลขึ้น server

2.7.5 PostgreSQL (Database Management System)

PostgreSQL เป็นระบบจัดการฐานข้อมูลเชิงสัมพันธ์ (Relational Database) ที่มีจุดเด่นในการรองรับธุรกรรมแบบ ACID และการจัดเก็บข้อมูลที่มีโครงสร้างซับซ้อนร่วมกับข้อมูลกึ่งโครงสร้างในรูปแบบ JSONB [29], [30] ความสามารถนี้มีความสำคัญอย่างยิ่งต่อระบบ IoT ที่ต้องจัดการทั้งข้อมูลหลักที่มีความสัมพันธ์ชัดเจน (เช่น ข้อมูลผู้ใช้และอุปกรณ์) และข้อมูลจากเซ็นเซอร์ที่มีโครงสร้างยืดหยุ่นหรือเปลี่ยนแปลงได้ตามรุ่นของฮาร์ดแวร์

การใช้ฐานข้อมูลที่รองรับทั้ง Relational และ JSONB ช่วยให้นักพัฒนาสามารถรักษาความสอดคล้องของข้อมูลหลักผ่านระบบ Schema ที่เข้มงวด ขณะเดียวกันก็สามารถขยายขีดความสามารถในการเก็บ Metadata หรือข้อมูลดิบจากเซ็นเซอร์ได้โดยไม่ต้องเปลี่ยนโครงสร้างฐานข้อมูลบ่อยครั้ง นำไปสู่การออกแบบระบบที่มีความเสถียรและยืดหยุ่นต่อการขยายผลในอนาคต



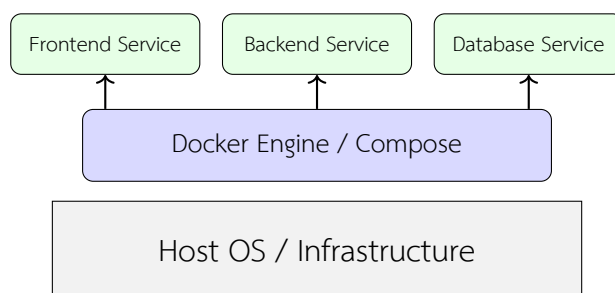
ธุรกรรม, ความสอดคล้อง, และการสอบทานย้อนหลัง

รูปที่ 2.16: แนวคิดการจัดเก็บข้อมูลแบบผสมผสานระหว่างข้อมูลเชิงโครงสร้างและกึ่งโครงสร้าง

2.7.6 Docker (Containerization)

Docker มีบทบาทสำคัญในการสร้างความสม่ำเสมอของสภาพแวดล้อมการทำงาน (Runtime Environment) โดยการใช้คอนเทนเนอร์เพื่อแยกแต่ละองค์ประกอบของแอปพลิเคชันออกจากกัน [31] แนวคิดนี้ช่วยแก้ปัญหาคอมpatibility ความแตกต่างของ Dependency ระหว่างเครื่องนักพัฒนาและเครื่องเซิร์ฟเวอร์จริง ทำให้การปรับใช้ระบบ (Deployment) สามารถทำได้อย่างรวดเร็วและลดโอกาสเกิดข้อผิดพลาดจากสภาพแวดล้อมที่ไม่ตรงกัน

นอกจากนี้ การใช้ Docker Compose ยังช่วยให้การจัดการระบบที่มีหลายบริการทำงานร่วมกัน (Multi-service architecture) เช่น เว็บแอปพลิเคชัน ฐานข้อมูล และตัวรับส่งข้อมูล เป็นไปอย่างเป็นระบบผ่านไฟล์กำหนดค่าชุดเดียว [32] ทำให้การเริ่มต้นทำงานของระบบทั้งหมดเป็นกระบวนการที่ทำซ้ำได้และมีความน่าเชื่อถือสูง

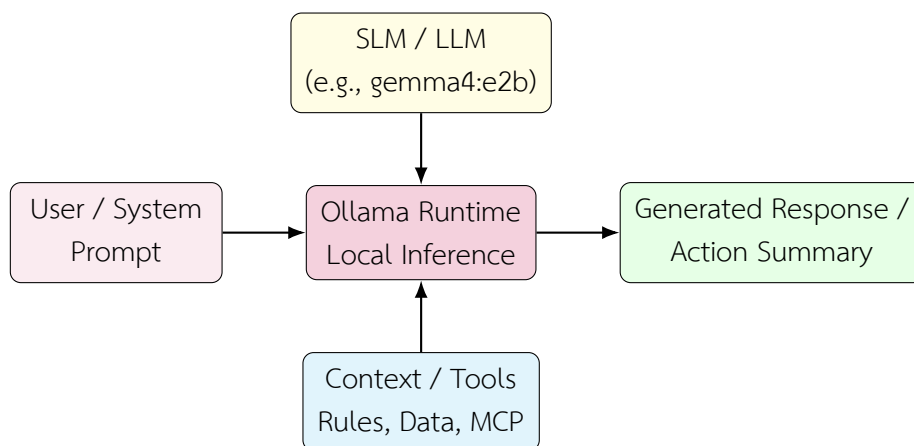


รูปที่ 2.17: การจัดวางบริการหลักของระบบในสภาพแวดล้อมคอนเทนเนอร์

2.7.7 Ollama และ Large Language Models (LLM)

การนำโมเดลภาษาขนาดใหญ่ (LLM) มาใช้ในระบบติดตามพฤติกรรมมีวัตถุประสงค์เพื่อช่วยสรุปเหตุการณ์และสนับสนุนการโต้ตอบเชิงภาษาธรรมชาติ อย่างไรก็ตาม ระบบลักษณะนี้มีข้อกำหนดด้านความเป็นส่วนตัวสูง จึงมีความจำเป็นต้องใช้ Local Inference มากกว่าบริการภายนอก **Ollama** เป็นแพลตฟอร์มที่ช่วยให้การรันโมเดล LLM ภายในโครงสร้างพื้นฐานขององค์กรทำได้สะดวก พร้อมเปิด API ที่แอปพลิเคชันอื่นเรียกใช้ได้โดยตรง [33], [34]

ในเชิงวิชาการ โมเดลที่เหมาะสมสำหรับการรันบนอุปกรณ์ที่มีทรัพยากรจำกัดคือกลุ่มโมเดลขนาดเล็ก (Small Language Models) เช่น **gemma4:e2b** ซึ่งถูกออกแบบให้มีพารามิเตอร์ที่มีประสิทธิภาพสำหรับการทำงานระดับ Edge [35] โดยมีความสามารถครอบคลุมทั้งงานด้าน Reasoning, Coding และการทำความเข้าใจสื่อหลายรูปแบบ (Multimodal Understanding) การใช้ LLM ในระบบเฝ้าระวังจึงควรเน้นบทบาทในการเป็นตัวสนับสนุนการตัดสินใจ (Decision Support) ภายใต้กฎเกณฑ์เชิงระบบที่ตรวจสอบได้



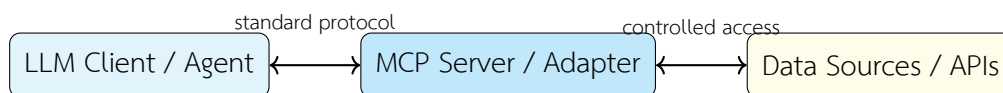
รูปที่ 2.18: แนวคิดการให้บริการโมเดลภาษาขนาดใหญ่ภายในระบบผ่าน Ollama

2.7.8 Model Context Protocol (MCP)

Model Context Protocol (MCP) เป็นมาตรฐานเปิดที่ออกแบบมาเพื่อให้แอปพลิเคชันสามารถจัดเตรียมบริบท (Context) และเครื่องมือ (Tools) ให้กับ LLM ได้อย่างสม่ำเสมอ [36], [37] มาตรฐานนี้ทำหน้าที่เสมือนตัวเชื่อมต่อที่เป็นสากล ช่วยให้แบบจำลองสามารถเข้าถึงแหล่งข้อมูลที่หลากหลายได้ภายใต้โครงสร้างเดียวกัน

เมื่อมีการนำ MCP มาใช้ ระบบจะสามารถแยกส่วนของโมเดลออกจากส่วนของข้อมูล

และเครื่องมือได้อย่างชัดเจน ส่งผลดีต่อความปลอดภัยและการควบคุมสิทธิ์การเข้าถึงข้อมูลที่สำคัญ (เช่น ข้อมูลสุขภาพ) โดยที่ตัวโมเดลยังสามารถทำหน้าที่ประมวลผลข้อมูลและเลือกใช้เครื่องมือที่เหมาะสมเพื่อตอบสนองต่อคำร้องขอของผู้ใช้ได้อย่างมีประสิทธิภาพ

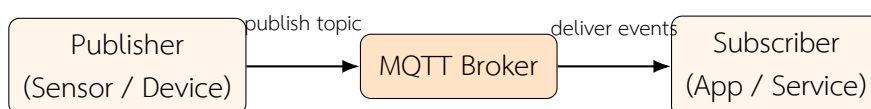


รูปที่ 2.19: บทบาทของ MCP ในการเชื่อมโยง LLM เข้ากับแหล่งข้อมูลและเครื่องมือ

2.7.9 MQTT (Message Queuing Telemetry Transport)

MQTT เป็นโพรโทคอลสื่อสารแบบ Publish/Subscribe ที่ออกแบบมาสำหรับบริบท IoT และเครือข่ายที่มีทรัพยากรจำกัด [38], [39], [40] ลักษณะเด่นคือการแยกผู้ส่ง (Publisher) ออกจากผู้รับ (Subscriber) ผ่านตัวกลาง (Broker) ทำให้ระบบมีความยืดหยุ่นสูงในการขยายจำนวนอุปกรณ์

ในระบบการเฝ้าระวัง อุปกรณ์เซ็นเซอร์และบริการประมวลผลสามารถสื่อสารกันผ่านหัวข้อ (Topics) ที่กำหนดไว้ล่วงหน้า ประโยชน์เชิงสถาปัตยกรรมคือช่วยให้การจัดการข้อมูลจำนวนมากในลักษณะ Near Real-time ทำได้โดยมีภาระของเครือข่ายต่ำ และรองรับระดับคุณภาพการส่งข้อมูล (QoS) ตามความสำคัญของข้อความแต่ละประเภท

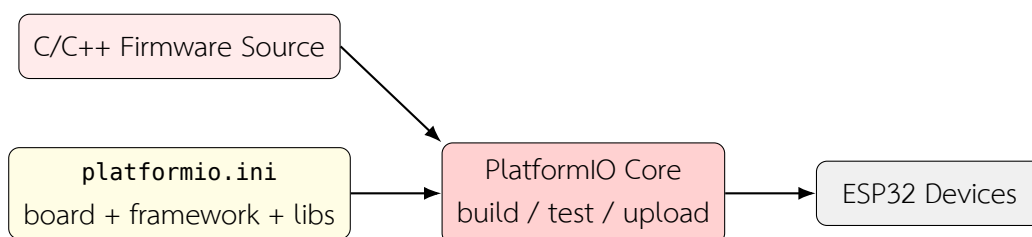


รูปที่ 2.20: รูปแบบการสื่อสารแบบ Publish/Subscribe ของ MQTT ในระบบ IoT

2.7.10 PlatformIO และการพัฒนา Firmware

PlatformIO เป็นเครื่องมือพัฒนาแบบ Cross-platform สำหรับระบบฝังตัว (Embedded Systems) ที่รองรับงานตั้งแต่การจัดการโปรเจกต์ การจัดการห้องสมุด (Library Management) การทดสอบ และการวิเคราะห์โค้ด [41], [42] เครื่องมือนี้ช่วยให้นักพัฒนาสามารถกำหนดสภาพแวดล้อมการทำงานของบอร์ดแต่ละรุ่นได้อย่างชัดเจนผ่านไฟล์กำหนดค่า เช่น `platformio.ini`

การใช้ PlatformIO ช่วยเพิ่มความสามารถในการตรวจสอบได้ (Traceability) และลดปัญหาความไม่สม่ำเสมอของ Toolchain ระหว่างการทำงานเป็นทีม นอกจากนี้ยังรองรับการทำงานร่วมกับระบบควบคุมเวอร์ชันและระบบอัตโนมัติ ทำให้การพัฒนาเฟิร์มแวร์มีความเป็นมืออาชีพและลดข้อผิดพลาดที่เกิดจากขั้นตอนการ Build แบบดั้งเดิม



รูปที่ 2.21: กระบวนการพัฒนาและติดตั้งเฟิร์มแวร์ผ่าน PlatformIO

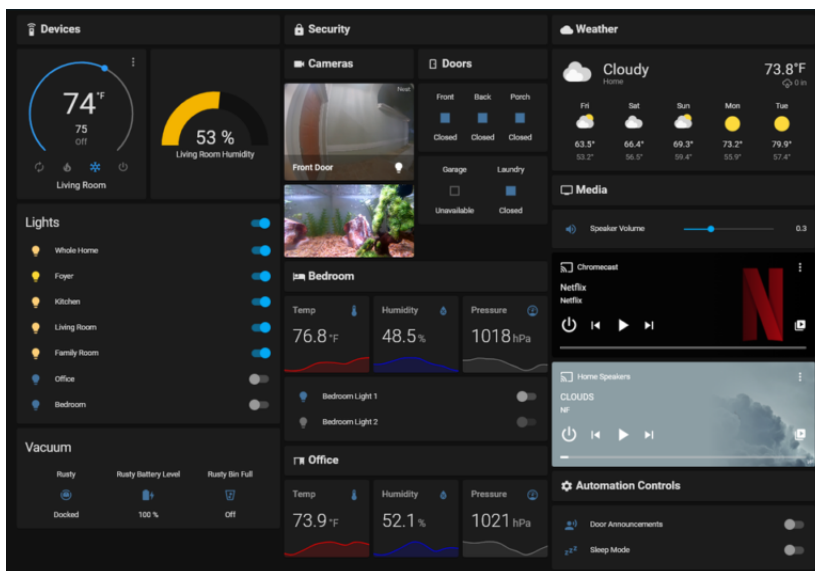
2.7.11 Home Assistant และการควบคุมอุปกรณ์ IoT

Home Assistant เป็นแพลตฟอร์มโอเพนซอร์สสำหรับการควบคุมอุปกรณ์ IoT และอุปกรณ์อัจฉริยะภายในบ้าน [43] แพลตฟอร์มนี้ทำงานบนระบบปฏิบัติการหลัก (Operating System) ที่ออกแบบมาสำหรับการทำงานต่อเนื่อง 24 ชั่วโมง (Always-on) โดยรองรับการติดตั้งบนอุปกรณ์หลากหลายประเภท ทั้งบนคอมพิวเตอร์ทั่วไป บอร์ดรองโปรแกรมเช่น Raspberry Pi และบนคลาวด์ (Virtual Machine)

Home Assistant มีความสามารถในการเชื่อมต่อกับอุปกรณ์หลากหลายประเภทผ่านโปรโตคอลต่างๆ เช่น Zigbee, Z-Wave, WiFi, Bluetooth, และ Matter ทำให้สามารถรองรับอุปกรณ์จากผู้ผลิตหลายยี่ห้อได้ในระบบเดียวกัน การจัดการอุปกรณ์ทำผ่านแนวคิด Integration ซึ่งเป็นส่วนเสริม (Add-on) ที่ทำหน้าที่เชื่อมต่อและควบคุมอุปกรณ์แต่ละประเภท

ในด้านการควบคุม Home Assistant มีระบบ Automation ที่ให้ผู้ใช้สามารถกำหนด

เงื่อนไขและการกระทำ (Trigger and Action) ได้อย่างยืดหยุ่น ตัวอย่างเช่น เมื่อเซ็นเซอร์ตรวจจับการเคลื่อนไหว (Motion Detected) ให้เปิดไฟในห้องนั้น หรือเมื่อถึงเวลาที่กำหนดให้ปิดอุปกรณ์ทั้งหมดในบ้าน นอกจากนี้ยังมีระบบ Scripting และ Template สำหรับการเขียนตรรกะที่ซับซ้อนและการปรับแต่งพฤติกรรมของอุปกรณ์ตามความต้องการของผู้ใช้



รูปที่ 2.22: หน้าจอหลักของ Home Assistant แสดงการจัดการอุปกรณ์ IoT และ Automation

2.8 งานวิจัยที่เกี่ยวข้อง

การทบทวนวรรณกรรมที่ผ่านมาแสดงให้เห็นถึงความพยายามนำเทคโนโลยีมาประยุกต์ใช้เพื่อการดูแลผู้สูงอายุอย่างหลากหลาย งานวิจัยในอดีตมักมุ่งเน้นไปที่การใช้เซ็นเซอร์เพื่อตรวจจับการล้มแบบแยกส่วน โดยขาดการบูรณาการข้อมูลเพื่อสร้างบริบทที่ครอบคลุม (Comprehensive Context) เช่น การนำข้อมูลการเคลื่อนไหว (IMU) มาร่วมกับสัญญาณชีพ (PPG) และพิกัดตำแหน่งภายในอาคาร (IPS) เพื่อวิเคราะห์พฤติกรรมเชิงลึก

นอกจากนี้ งานวิจัยหลายชิ้นที่มีการนำ AI มาใช้ มักประสบปัญหาด้านความเป็นส่วนตัว เนื่องจากการพึ่งพาบริการ Cloud ภายนอก และปัญหาความโปร่งใสของการตัดสินใจ (Black-box AI) ในทางตรงกันข้าม ปริญญานิพนธ์ฉบับนี้ได้นำเสนอสถาปัตยกรรมแบบขอบเครือข่าย (Edge AI) ผ่านการใช้โมเดลภาษาขนาดเล็ก (gemma4:e2b) ที่ทำงานร่วมกับ Model Context Protocol (MCP) เพื่อจำกัดขอบเขตข้อมูลและเพิ่มความโปร่งใส (Explainability) ของปัญญาประดิษฐ์ในการดูแลผู้สูงอายุ ส่งผลให้เกิดระบบการดูแลผู้สูงอายุที่ตอบสนองแบบเรียลไทม์ ปลอดภัย และมีประสิทธิภาพสูง

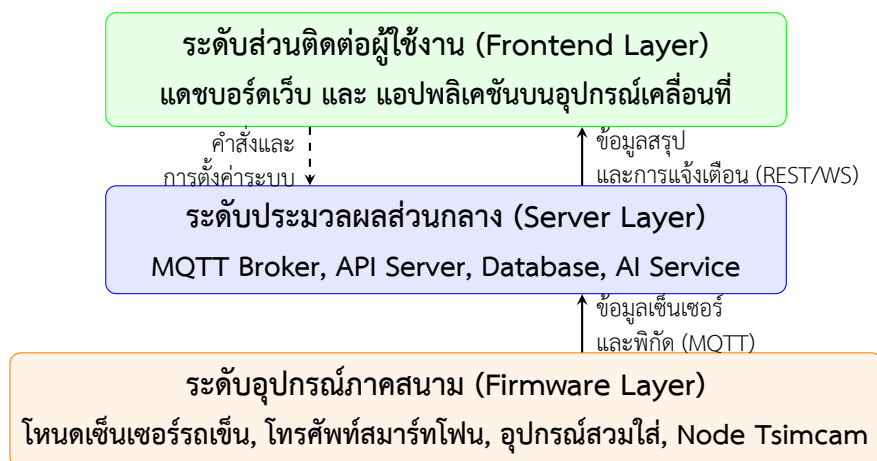
บทที่ 3

วิธีการศึกษา

3.1 การออกแบบระบบ

จากการศึกษาบริบทของสถานดูแลและปัญหาการขาดแคลนบุคลากรในการดูแลผู้สูงอายุและผู้สูงอายุในบทที่ 1 ระบบต้นแบบ WheelSense จึงถูกออกแบบขึ้นเพื่อแก้ปัญหาด้านความปลอดภัยและการรับรู้สถานการณ์ โดยมีเป้าหมายเพื่อช่วยให้ผู้ดูแลสามารถติดตามตำแหน่งและรับรู้เหตุผิดปกติได้อย่างรวดเร็ว สถาปัตยกรรมของระบบรองรับการทำงานแบบครบวงจร ตั้งแต่การติดตามตำแหน่งแบบเรียลไทม์ การแจ้งเตือนเหตุการณ์ ไปจนถึงการใช้ระบบผู้ช่วยอัจฉริยะ (AI) โครงสร้างสถาปัตยกรรมแบ่งออกเป็น 3 ระดับหลัก ได้แก่

1. **ระดับอุปกรณ์ภาคสนาม (Firmware Layer)** ประกอบด้วยเซ็นเซอร์บนรถเข็น อุปกรณ์สวมใส่ โทรศัพท์มือถือ และอุปกรณ์อัจฉริยะตำแหน่ง ทำหน้าที่อ่านค่าพิกัดและข้อมูลสัญญาณชีพ
2. **ระดับประมวลผลส่วนกลาง (Server Layer)** ประกอบด้วย MQTT Broker, API Server, Database และ AI Service ทำหน้าที่รับข้อมูล วิเคราะห์ และประมวลผล
3. **ระดับส่วนติดต่อผู้ใช้งาน (Frontend Layer)** ประกอบด้วยแดชบอร์ดเว็บและแอปพลิเคชันบนอุปกรณ์เคลื่อนที่ ทำหน้าที่แสดงผลและโต้ตอบกับผู้ใช้งานตามบทบาท



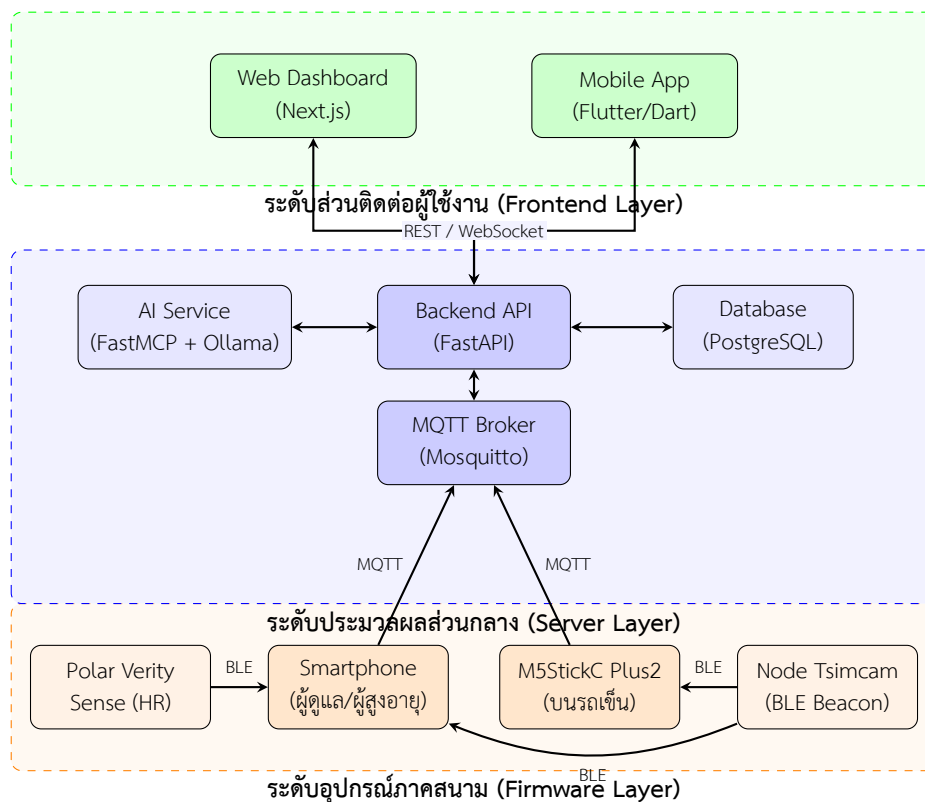
รูปที่ 3.1: โครงสร้างเชิงชั้นของระบบต้นแบบแสดงการไหลของข้อมูลระหว่างชั้นต่างๆ

ในระดับอุปกรณ์ภาคสนาม ไมโครคอนโทรลเลอร์ M5StickC Plus2 ติดตั้งบนเก้าอี้รถเข็นเพื่ออ่านค่าเซ็นเซอร์วัดแรงเฉื่อย (IMU) แบบ 6 แกน และส่งข้อมูล telemetry ผ่าน Wi-Fi โดยใช้โปรโตคอล MQTT อุปกรณ์สวมใส่ Polar Verity Sense ทำหน้าที่วัดสัญญาณชีพผ่านเซ็นเซอร์แสง (PPG) และส่งข้อมูลอัตราการเต้นของหัวใจไปยังสมาร์ทโฟนของผู้ใช้หรือผู้ดูแลผ่าน Bluetooth Low Energy (BLE) อุปกรณ์ Node Tsimcam ทำหน้าที่เป็นจุดส่งสัญญาณ BLE beacon สำหรับอ้างอิงตำแหน่งภายในอาคาร พร้อมถ่ายภาพประจำจุดตามการตั้งค่า โดยสมาร์ทโฟนยังทำหน้าที่เป็นสะพานเชื่อมต่อข้อมูลสัญญาณชีพจากอุปกรณ์สวมใส่เข้าสู่ระบบเครือข่าย

ระดับประมวลผลส่วนกลางประกอบด้วย MQTT Broker (Eclipse Mosquitto) สำหรับรับส่งข้อมูล telemetry จากอุปกรณ์ภาคสนามแบบ Publish/Subscribe Backend API ที่พัฒนาด้วย FastAPI ทำหน้าที่จัดการตรรกะธุรกิจ ตรวจสอบสิทธิ์การเข้าถึง และประมวลผลคำขอจากฝั่งผู้ใช้งาน ฐานข้อมูล PostgreSQL จัดเก็บทั้งข้อมูลที่มีโครงสร้างและข้อมูลกึ่งโครงสร้าง (JSONB) สำหรับ telemetry ข้อมูลการแจ้งเตือน และบันทึกตรวจสอบ (Audit Log) บริการ AI (Ollama ร่วมกับ FastMCP) ทำหน้าที่เป็นผู้ช่วยอัจฉริยะ รับคำถามจากผู้ใช้งาน วิเคราะห์บริบทระบบ และเรียกใช้เครื่องมือ (Tools) ในระบบผ่าน Model Context Protocol (MCP)

ระดับส่วนติดต่อผู้ใช้งานประกอบด้วยแดชบอร์ดเว็บที่พัฒนาด้วย Next.js และแอปพลิเคชันบนอุปกรณ์เคลื่อนที่ที่พัฒนาด้วย Flutter/Dart ทั้งสองช่องทางรองรับการเข้าถึงตามบทบาท (Role-based) เช่น ผู้ดูแลระบบ พยาบาลหัวหน้า ผู้สังเกตการณ์ และผู้ใช้เก้าอี้รถเข็น โดยใช้ระบบตรวจสอบสิทธิ์ด้วย JWT ที่จัดเก็บในรูปแบบ HttpOnly cookie

การไหลของข้อมูลในระบบเริ่มจากอุปกรณ์ภาคสนามส่งข้อมูล telemetry ผ่านโปรโตคอล MQTT ไปยัง MQTT Broker จากนั้น Backend API ดึงข้อมูลไปประมวลผล จัดเก็บในฐานข้อมูล PostgreSQL และส่งต่อไปยังหน้าจอผู้ใช้งานผ่าน REST API หรือ WebSocket สำหรับการอัปเดตแบบเรียลไทม์ บริการ AI สื่อสารกับ Backend API เพื่อดึงบริบทระบบและสั่งการผ่านเครื่องมือที่กำหนดไว้ ขณะที่ Frontend ส่งคำสั่งและคำถามจากผู้ใช้งานไปยัง Backend API เพื่อประมวลผลต่อไป



รูปที่ 3.2: สถาปัตยกรรมระบบ WheelSense โดยรวมแสดงการเชื่อมโยงระหว่างทุกชั้น

3.2 การออกแบบเซ็นเซอร์บนเก้าอี้รถเข็น

3.2.1 ฮาร์ดแวร์

อุปกรณ์ภาคสนามสำหรับการใช้งานบนเก้าอี้รถเข็นใช้ไมโครคอนโทรลเลอร์ M5StickC Plus2 ที่มีเซ็นเซอร์วัดแรงเฉื่อย (IMU) แบบ 6 แกน (MPU-6886) ทำหน้าที่อ่านค่าความเร่งและอัตราการหมุน และเชื่อมต่อเครือข่าย Wi-Fi สำหรับการรายงานค่าเซ็นเซอร์ อุปกรณ์นี้ติดตั้งบนตัวเก้าอี้รถเข็นโดยตรงเพื่อตรวจจับการเคลื่อนที่และสั่นสะเทือนของตัวรถ

ตารางที่ 3.1: คุณสมบัติของไมโครคอนโทรลเลอร์ M5StickC Plus2

คุณสมบัติ	รายละเอียด
ชิปหลัก	ESP32-PICO-D4 (Dual-core 240 MHz)
เซ็นเซอร์ IMU	MPU-6886 (6-axis: accel + gyro)
การเชื่อมต่อ	Wi-Fi 802.11 b/g/n, BLE 4.2
หน้าจอ	TFT LCD 1.14 นิ้ว 135x240 px
แบตเตอรี่	Li-Po 200 mAh
การส่งข้อมูล	MQTT ผ่าน Wi-Fi
กลุ่มผู้ใช้งาน	ผู้ใช้เก้าอี้รถเข็น



รูปที่ 3.3: ตำแหน่งการติดตั้งไมโครคอนโทรลเลอร์ M5StickC Plus2 บนเก้าอี้รถเข็น

3.2.2 โครงข่ายข้อมูล

M5StickC Plus2 เชื่อมต่อกับเซิร์ฟเวอร์ส่วนกลางผ่านเครือข่าย Wi-Fi โดยใช้โปรโตคอล MQTT เพื่อส่งข้อมูล telemetry ความถี่สูง โปรโตคอล MQTT ถูกเลือกเนื่องจากมี overhead ต่ำ รองรับการเชื่อมต่อที่ไม่เสถียร และสามารถกำหนดหัวข้อ (Topic) เฉพาะสำหรับแต่ละอุปกรณ์ได้ เช่น WheelSense/data/<device_id> นอกจากนี้ อุปกรณ์ยังสมัครรับหัวข้อคำสั่งควบคุม (WheelSense/<device_id>/control) และหัวข้ออัปเดตการตั้งค่า (WheelSense/config/...) เพื่อรับคำสั่งจากเซิร์ฟเวอร์แบบเรียลไทม์

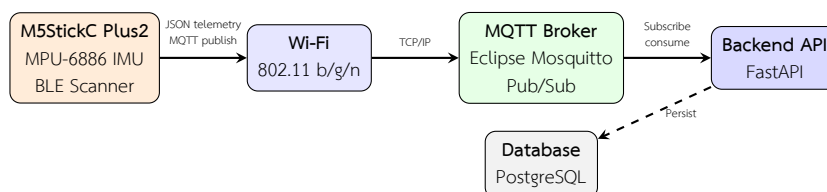
เมื่อเชื่อมต่อ Wi-Fi ไม่สำเร็จ ระบบจะพยายามเชื่อมต่อใหม่ด้วยระยะห่างที่เพิ่มขึ้นแบบเอกซ์โพเนนเชียล (exponential backoff) สูงสุด 60 วินาที สำหรับการเชื่อมต่อ MQTT หากหลุดการเชื่อมต่อจะพยายามเชื่อมต่อใหม่ทุก 2 วินาที และเพิ่มระยะห่างขึ้นจนถึง 60 วินาทีเช่นกัน การ

สมัครรับหัวข้อหลังเชื่อมต่อ MQTT สำเร็จประกอบด้วยหัวข้อการตั้งค่า คำสั่งควบคุม และการยืนยัน (acknowledgement)

ตารางที่ 3.2 แสดงโครงสร้างหัวข้อ (Topic Tree) ของระบบ WheelSense ซึ่งแบ่งตามทิศทางการไหลของข้อมูล ระหว่างอุปกรณ์ภาคสนามกับเซิร์ฟเวอร์ส่วนกลาง

ตารางที่ 3.2: โครงสร้างหัวข้อ MQTT ในระบบ WheelSense

หัวข้อ (Topic)	ทิศทาง	รายละเอียด
WheelSense/data	อุปกรณ์→เซิร์ฟเวอร์	Telemetry รวม IMU, RSSI, motion, battery
WheelSense/camera/+/registration	อุปกรณ์→เซิร์ฟเวอร์	ลงทะเบียนกล้อง Node Tsimcam
WheelSense/camera/+/status	อุปกรณ์→เซิร์ฟเวอร์	สถานะและภาพถ่ายของกล้อง
WheelSense/+/ack	อุปกรณ์→เซิร์ฟเวอร์	ยืนยันการรับคำสั่ง
WheelSense/vitals/{patient_id}	เซิร์ฟเวอร์→อุปกรณ์	ส่งต่อข้อมูลสัญญาณชีพ
WheelSense/room/{device_id}	เซิร์ฟเวอร์→อุปกรณ์	ผลการทำนายตำแหน่งห้อง
WheelSense/alerts/{patient}	เซิร์ฟเวอร์→อุปกรณ์	แจ้งเตือนเหตุการณ์สำคัญ
WheelSense/config/{device_id}	เซิร์ฟเวอร์→อุปกรณ์	อัปเดตการตั้งค่าเฉพาะอุปกรณ์
WheelSense/config/all	เซิร์ฟเวอร์→อุปกรณ์	อัปเดตการตั้งค่าทั้งระบบ
WheelSense/{device_id}/control	เซิร์ฟเวอร์→อุปกรณ์	คำสั่งควบคุมอุปกรณ์



รูปที่ 3.4: เส้นทางข้อมูลจาก M5StickC Plus2 ผ่าน Wi-Fi และ MQTT Broker สู่ Backend API

3.2.3 การประมวลผลข้อมูล

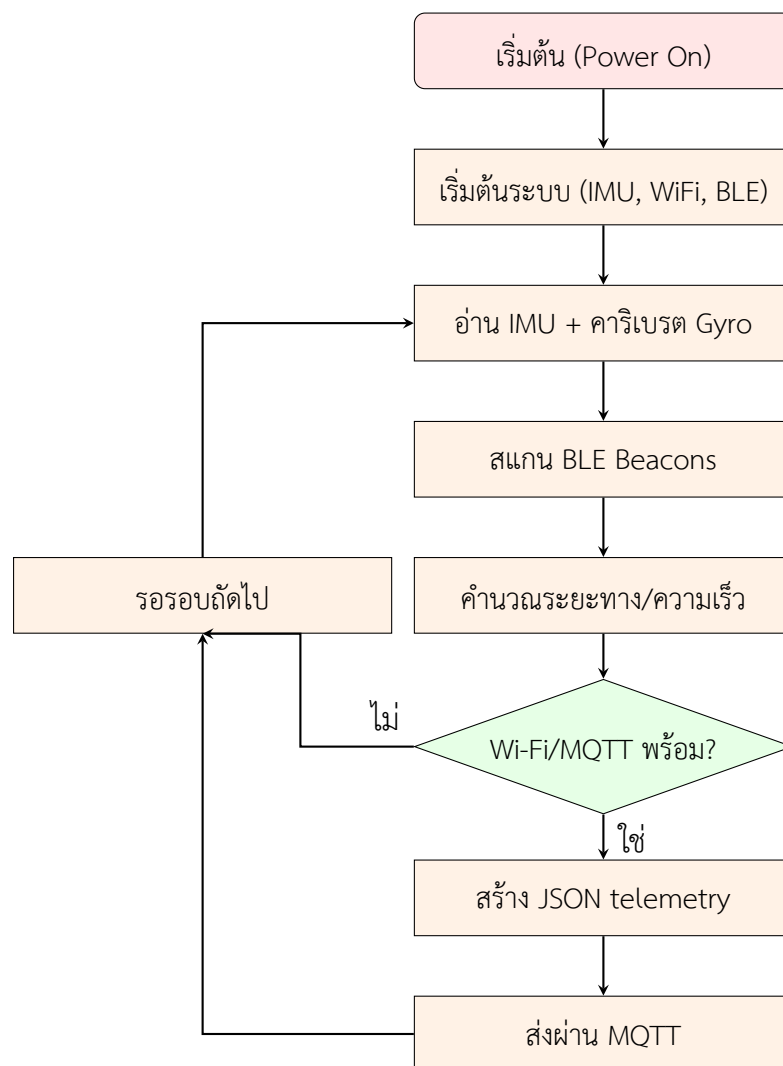
ข้อมูลดิบจากเซ็นเซอร์ IMU อ่านผ่านไลบรารี M5.Imu โดยใช้ชิป MPU-6886 ซึ่งให้ค่าความเร่งและอัตราการหมุนแบบ 3 แกน ก่อนการใช้งานจริง ระบบจะปรับค่าออฟเซตของไจโรสโคป (Gyro Z Offset) โดยเก็บตัวอย่าง 1 วินาทีและหาค่าเฉลี่ย เพื่อลดค่าเบี่ยงเบนคงที่ (DC bias)

ในแต่ละรอบการอ่านค่า ระบบนำค่าไจโรสโคปแกน Z ผ่านตัวกรองแบบเอกซ์โพเนนเชียล ลมูฟวิงเอเวอเรจ (EMA Low-pass Filter) เพื่อลดสัญญาณรบกวน จากนั้นใช้ Deadband ตัดค่าที่ต่ำกว่าเกณฑ์ออก แล้วแปลงค่าเป็นระยะทางและความเร็วโดยอินทิเกรตตามเวลา โดยอ้างอิงรัศมีล้อรถเข็นที่ตั้งค่าไว้ ความเร็วคำนวณผ่านหน้าต่างเลื่อน (sliding window) และมีกลไกลดค่าความเร็วลง (decay) เมื่อไม่พบการเคลื่อนที่เป็นเวลาที่กำหนด เพื่อลด drift

นอกจาก IMU อุปกรณ์ยังสแกนสัญญาณ BLE beacon รอบๆ ตัวแบบ active scan

โดยแยกชื่ออุปกรณ์ที่ขึ้นต้นด้วย WSN_ แล้วบันทึกค่า RSSI และ MAC address ไว้ในรายการ พร้อม
การจัดรายการที่ไม่พบนานเกินกำหนด

ข้อมูลทั้งหมดถูกจัดรูปแบบเป็น JSON ประกอบด้วย ข้อมูล IMU (ความเร่ง 6 แกน)
ข้อมูล motion (ระยะทาง ความเร็ว ความเร่ง ทิศทาง) รายการ RSSI จาก beacon ที่พบ และสถานะ
แบตเตอรี่ จากนั้นส่งผ่าน MQTT ด้วยอัตราที่ปรับตามสถานะการทำงาน คือ 50 มิลลิวินาทีขณะ
บันทึกการเคลื่อนที่ หรือตามช่วงเวลาปกติเมื่ออยู่นิ่ง



รูปที่ 3.5: แผนผังแสดงวงรอบการทำงานหลักของเฟิร์มแวร์ M5StickC Plus2

3.3 อุปกรณ์สวมใส่และสมาร์ทโฟน

3.3.1 ฮาร์ดแวร์

ระบบตรวจวัดสัญญาณชีพและกิจกรรมทางกายของผู้ใช้เก้าอี้รถเข็นใช้อุปกรณ์สวมใส่ Polar Verity Sense ซึ่งเป็นเซ็นเซอร์วัดอัตราการเต้นของหัวใจแบบโฟโต้พลิทึมอกราฟี (Photoplethysmography: PPG) พร้อมเซ็นเซอร์ความเร่ง (Accelerometer) และไจโรสโคป ติดตั้งบริเวณข้อมือหรือแขนของผู้ใช้งาน อุปกรณ์นี้ส่งข้อมูลหลายชนิดผ่านโปรโตคอล BLE โดยใช้ Polar SDK ซึ่งช่วยให้สมาร์ทโฟนสามารถรับข้อมูลอัตราการเต้นของหัวใจ คลื่นแสง PPG ดิบ และข้อมูลความเร่งได้พร้อมกัน นอกจากนี้ยังรองรับการเชื่อมต่อแบบ BLE GATT มาตรฐานสำหรับอัตราการเต้นของหัวใจ (HR Service 0x180D) ในกรณีที่ผู้ใช้ SDK

สมาร์ทโฟนทำหน้าที่เป็นสะพานเชื่อมต่อ (Gateway) ระหว่างอุปกรณ์สวมใส่กับเซิร์ฟเวอร์ส่วนกลาง โดยรับข้อมูลสัญญาณชีพผ่าน BLE แล้วส่งต่อไปยังเซิร์ฟเวอร์ผ่านเครือข่าย Wi-Fi หรือเซลลูลาร์ นอกจากนี้สมาร์ทโฟนยังมีเซ็นเซอร์ภายในตัว เช่น จีพีเอส (GPS) และตัวสแกนบลูทูธ (BLE Scanner) ที่ใช้รับสัญญาณ beacon เพื่อช่วยระบุตำแหน่งของผู้ใช้งาน

ตารางที่ 3.3: คุณสมบัติของอุปกรณ์สวมใส่และสมาร์ทโฟน

คุณสมบัติ	Polar Verity Sense	สมาร์ทโฟน
หน้าที่หลัก	วัดอัตราการเต้นหัวใจ PPG, ความเร่ง ACC	รับข้อมูล Polar SDK และส่งต่อสู่เซิร์ฟเวอร์
เซ็นเซอร์	PPG ดิบ, ไจโรสโคป, ความเร่ง	GPS, BLE Scanner, จอแสดงผล
การสื่อสาร	BLE GATT (Peripheral)	Wi-Fi / Cellular, BLE Central
แบตเตอรี่	ชาร์จได้ 20 ชั่วโมง	แบตเตอรี่ลิเทียมไอออนในตัว
กลุ่มผู้ใช้	ผู้สูงอายุที่ใช้เก้าอี้รถเข็น	ผู้ดูแล / ผู้ใช้งาน

3.3.2 โครงข่ายข้อมูล

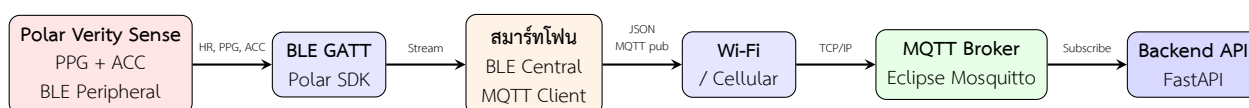
โครงข่ายข้อมูลของอุปกรณ์สวมใส่แบ่งเป็น 2 ช่วง ช่วงที่ 1 เป็นการเชื่อมต่อระหว่าง Polar Verity Sense กับสมาร์ทโฟนผ่าน BLE GATT โดย Polar ทำหน้าที่อุปกรณ์รอง (Peripheral) และสมาร์ทโฟนทำหน้าที่อุปกรณ์หลัก (Central) ช่วงที่ 2 เป็นการส่งข้อมูลจากสมาร์ทโฟนไปยังเซิร์ฟเวอร์ส่วนกลางผ่านโปรโตคอล MQTT บนเครือข่าย Wi-Fi หรือเซลลูลาร์

สมาร์ทโฟนยังสแกนสัญญาณ BLE beacon จาก Node Tsimcam ที่ติดตั้งประจำที่ใน



รูปที่ 3.6: การติดตั้ง Polar Verity Sense และตำแหน่งใช้งานสมาร์ทโฟน

อาคาร เพื่อเก็บค่า RSSI ส่งไปยังเซิร์ฟเวอร์ร่วมกับข้อมูลสัญญาณชีพ ช่วยให้ระบบระบุตำแหน่งของผู้ใช้งานได้แม่นยำยิ่งขึ้นเมื่อนำมาประมวลผลร่วมกับข้อมูล RSSI จาก M5StickC Plus2



รูปที่ 3.7: เส้นทางข้อมูลจาก Polar Verity Sense ผ่านสมาร์ทโฟนสู่ MQTT Broker และ Backend API

ตารางที่ 3.4 สรุปโปรโตคอลการสื่อสาร ทิศทางข้อมูล และประเภทข้อมูลที่แลกเปลี่ยนระหว่าง Polar Verity Sense สมาร์ทโฟน และเซิร์ฟเวอร์ส่วนกลาง

ตารางที่ 3.4: ตารางการสื่อสารระหว่าง Polar Verity Sense สมาร์ทโฟน และเซิร์ฟเวอร์

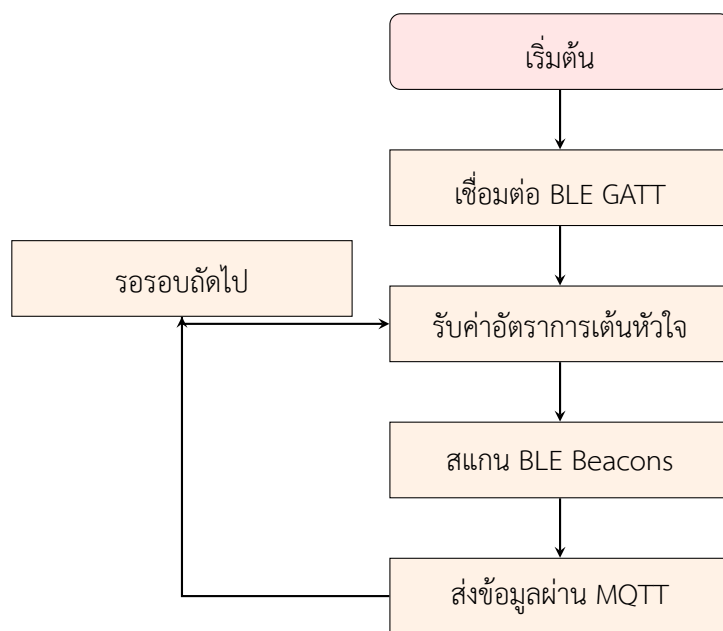
คู่สื่อสาร	โปรโตคอล	ทิศทาง	ข้อมูลที่แลกเปลี่ยน
Polar → สมาร์ทโฟน	BLE GATT	อุปกรณ์→Gateway	HR, PPG raw (3ch), ACC (3-axis), แบตเตอรี่
สมาร์ทโฟน → เซิร์ฟเวอร์	MQTT	Gateway→เซิร์ฟเวอร์	Vitals JSON, RSSI beacon list, timestamp, device ID
เซิร์ฟเวอร์ → สมาร์ทโฟน	MQTT	เซิร์ฟเวอร์→Gateway	Alert, room prediction, config update

3.3.3 การประมวลผลข้อมูล

การประมวลผลข้อมูลบนสมาร์ทโฟนเริ่มจากการเชื่อมต่อกับ Polar Verity Sense ผ่าน Polar SDK ซึ่งเปิดช่องข้อมูล (streaming) พร้อมกันหลายชนิด ได้แก่ อัตราการเต้นของหัวใจ (HR) คลื่นแสง PPG ดิบ 3 ช่อง และข้อมูลความเร่ง (ACC) 3 แกน โดย SDK จัดการโครงสร้างแพ็กเก็ต

ข้อมูลและการประสานเวลาให้อัตโนมัติ จากนั้นสมาร์ทโฟนส่งข้อมูลสัญญาณซีฟไปยังเซิร์ฟเวอร์ผ่าน MQTT พร้อมบันทึกเวลา (timestamp) และรหัสอุปกรณ์

สำหรับข้อมูลตำแหน่ง สมาร์ทโฟนสแกนหา BLE beacon รอบตัวแล้วส่งชุดค่า RSSI พร้อมกับรหัสอุปกรณ์ไปยังเซิร์ฟเวอร์ โดยใช้รูปแบบ JSON ที่เข้ากันได้กับข้อมูลจาก M5StickC Plus2



รูปที่ 3.8: แผนผังแสดงวงรอบการทำงานของแอปพลิเคชันบนสมาร์ทโฟน

3.4 การจำแนกตำแหน่ง

3.4.1 ฮาร์ดแวร์

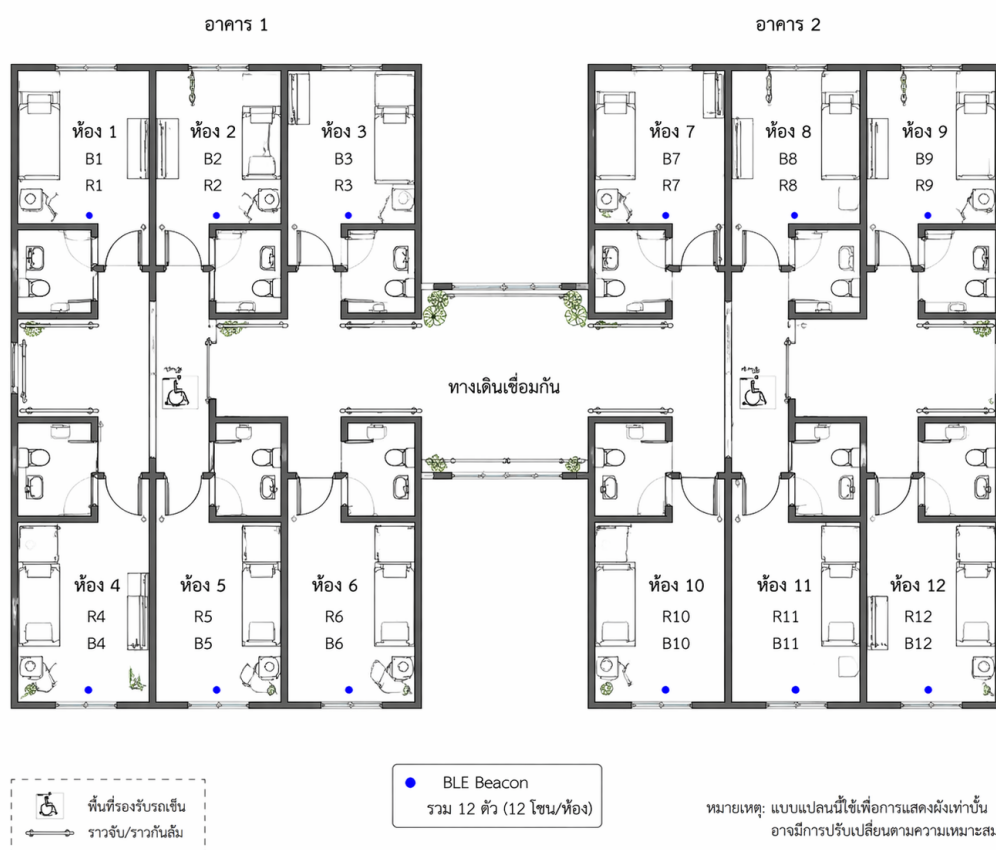
ระบบจำแนกตำแหน่งภายในอาคารใช้อุปกรณ์ Node Tsimcam ซึ่งเป็นโหนดอเนกประสงค์ที่ทำหน้าที่ทั้งเป็น BLE beacon และกล้องถ่ายภาพในตัวเดียวกัน อุปกรณ์นี้ติดตั้งประจำจุดตามโซนหรือห้องต่างๆ บนเพดาน โดยส่งสัญญาณโฆษณา BLE อย่างต่อเนื่องพร้อมระบุชื่อโหนด (เช่น WSN_001) เพื่อให้อุปกรณ์ภาคสนามสามารถรับค่าความเข้มสัญญาณ (RSSI) และนำไปใช้อ้างอิงตำแหน่ง

นอกจากการส่งสัญญาณ beacon Node Tsimcam ยังติดตั้งกล้องดิจิทัลที่สามารถถ่ายภาพนิ่งและส่งรูปภาพไปยังเซิร์ฟเวอร์ส่วนกลางผ่านโปรโตคอล MQTT โดยใช้วิธีแบ่งรูปภาพเป็นชิ้นเล็กๆ (chunked base64) หรือส่งแบบไบนารีโดยตรงผ่าน MQTT publish รูปภาพที่ได้ใช้สำหรับการ

ตรวจสอบภาพในพื้นที่และยืนยันสถานะของผู้ใช้งานแบบ offline ร่วมกับข้อมูลตำแหน่งจากสัญญาณ RSSI

ตารางที่ 3.5: ข้อกำหนดและแนวทางการติดตั้ง Node Tsimcam

รายการ	ข้อกำหนด
ชิปหลัก	ESP32-S3 (Dual-core 240 MHz)
กล้อง	OV2640 ความละเอียดสูงสุด VGA (640x480)
การเชื่อมต่อ	Wi-Fi 802.11 b/g/n, BLE 5.0
การส่งสัญญาณ BLE	ชื่อโหนด WSN_NNN, Advertising แบบ active
การส่งภาพ	MQTT publish (chunked base64 หรือ binary)
การจ่ายไฟ	USB หรือแบตเตอรี่ภายนอก (3.3–5 V)
ความสูงติดตั้ง	2.5–3.0 เมตรจากพื้น (เพดานหรือผนัง)
ระยะห่างระหว่างโหนด	5–8 เมตร ในโซนเดียวกัน



รูปที่ 3.9: แผนผังการจัดวาง BLE beacon (12 โซน/ห้อง)

3.4.2 การประมวลผลข้อมูล

กระบวนการประมวลผลตำแหน่งทำงานที่ฝั่งเซิร์ฟเวอร์ โดยรับค่า RSSI จากอุปกรณ์ภาคสนาม 2 ชนิด คือ M5StickC Plus2 ที่ติดตั้งบนเก้าอี้รถเข็น และสมาร์ทโฟนของผู้ใช้งานหรือผู้ดูแล ทั้งสองอุปกรณ์สแกนหา BLE beacon จาก Node Tsimcam ที่กระจายอยู่ทั่วอาคาร และส่งชุดค่า RSSI พร้อมรหัสโหนดและเวลาไปยังเซิร์ฟเวอร์ผ่าน MQTT

เซิร์ฟเวอร์จัดเก็บชุดค่า RSSI ดังกล่าวในฐานข้อมูล PostgreSQL รูปแบบ JSONB และใช้อัลกอริทึม K-Nearest Neighbors (K-NN) ในการประมวลผลตำแหน่ง โดยเปรียบเทียบชุดค่า RSSI ที่ได้รับในปัจจุบันกับชุดข้อมูลอ้างอิง (Radio Map) ที่บันทึกไว้ล่วงหน้าจากการสำรวจแต่ละโซน การคำนวณระยะห่างทางสถิติใช้ Euclidean Distance ดังสมการ 3.1 แล้วเลือกจุดอ้างอิงที่ใกล้เคียงที่สุดจำนวน k จุด เพื่อโหวตน้ำหนักหาตำแหน่งหรือโซนที่ผู้ใช้แต่ละคนอยู่ในขณะนั้น ตามสมการ 3.2 ข้อมูลจากทั้ง M5StickC และสมาร์ทโฟนช่วยเพิ่มความถูกต้องเนื่องจากเซิร์ฟเวอร์สามารถรวมค่า RSSI จากหลายมุมมองและแก้ไขค่าที่ผิดปกติได้

$$d_j = \sqrt{\sum_{i=1}^m (x_i - r_{ji})^2} \quad (3.1)$$

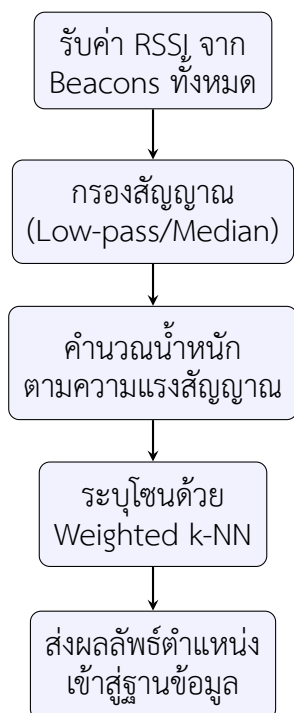
จากนั้นระบบจะใช้อัลกอริทึม K-Nearest Neighbors (K-NN) โดยเลือกจุดอ้างอิงที่มีระยะห่างทางสถิติน้อยที่สุดจำนวน k จุด แล้วทำการโหวตน้ำหนักเพื่อหาพิกัดหรือโซนที่น่าจะเป็นไปได้มากที่สุด

$$\hat{y} = \arg \max_{y \in \mathcal{Y}} \sum_{i=1}^k \frac{1}{d_i} \cdot \mathbb{K}_{[y_i=y]} \quad (3.2)$$

ในกรณีที่มีสัญญาณขาดหายชั่วคราว ระบบได้กำหนดมาตรการรองรับดังตารางที่ 3.6

ตารางที่ 3.6: มาตรการรองรับกรณีสัญญาณ Beacon ขาดหาย

เงื่อนไข	การจัดการ	ผล
Beacon บางตัวไม่ตอบสนอง	แทนค่า RSSI = -100 dBm	ทำนายตำแหน่งต่อ
ขาดมากกว่า 50%	หยุดการทำนาย	ใช้ last known location
สัญญาณกลับมาปกติ	รีเซ็ตค่า fallback	ทำนายตำแหน่งใหม่



รูปที่ 3.10: Flowchart การประมวลผลข้อมูลสำหรับการจำแนกตำแหน่ง

3.5 เซิร์ฟเวอร์

3.5.1 สถาปัตยกรรมโดยรวม

สถาปัตยกรรมเซิร์ฟเวอร์ออกแบบบนแนวคิดไมโครเซอร์วิสผ่าน Docker Compose เพื่อให้ระบบสามารถขยายขนาดและดูแลรักษาได้ง่าย โดยประกอบด้วย 4 บริการหลัก ได้แก่ MQTT Broker (Eclipse Mosquitto) สำหรับจัดการการส่งข้อความแบบ Publish/Subscribe, Backend API (FastAPI) สำหรับจัดการตรรกะธุรกิจ, Database (PostgreSQL 16) สำหรับเก็บข้อมูล และ AI Service (Ollama + FastMCP) สำหรับประมวลผลผู้ช่วยอัจฉริยะ

ระบบเซิร์ฟเวอร์ทั้งหมดถูกนำไปใช้งานบน Raspberry Pi 5 ซึ่งเป็นคอมพิวเตอร์บอร์ดขนาดเล็ก (Single Board Computer) ที่มีหน่วยความจำ RAM 8 GB และใช้ชิป ARM Cortex-A76 Quad-core ความถี่ 2.4 GHz การเลือกใช้ Raspberry Pi 5 ช่วยลดต้นทุนด้านฮาร์ดแวร์และพลังงาน ในขณะเดียวกันก็ยังสามารถรันบริการต่างๆ ผ่าน Docker Compose ได้อย่างมีประสิทธิภาพ ทำให้เหมาะสำหรับการทำระบบเซิร์ฟเวอร์ส่วนกลางในโครงการ IoT ที่ต้องการความยืดหยุ่นและความสามารถในการปรับขนาด

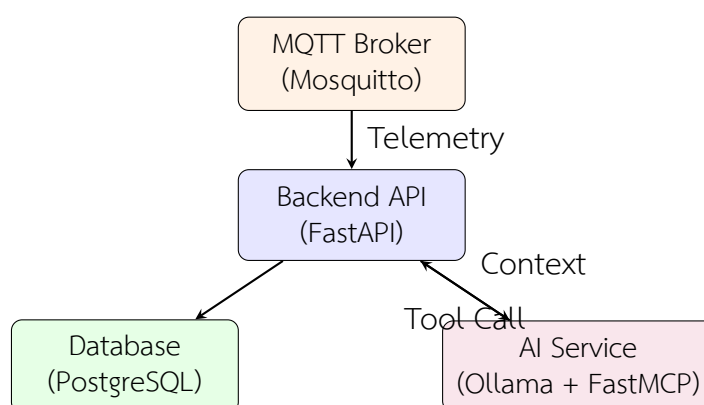
การจัดการชีวิตวงจรของแอปพลิเคชัน (Application Lifecycle) ใช้ Lifespan

Manager ของ FastAPI ซึ่งเริ่มต้นด้วยการตรวจสอบการตั้งค่าระบบ (Runtime Settings Validation) จากนั้นตรวจสอบการเชื่อมต่อฐานข้อมูล สร้างบัญชีผู้ดูแลระบบเริ่มต้นหากยังไม่มี เริ่มงาน MQTT Listener เป็นพื้นหลัง (Background Task) และเปิดใช้งาน Retention Scheduler ตามเงื่อนไข เมื่อปิดระบบจะยกเลิกงาน MQTT และหยุด Scheduler อย่างเป็นระเบียบ

ชั้นบริการ (Service Layer) แยกตรรกะธุรกิจออกจากชั้นข้อมูล โดยมีคลาส CRUDBase ทั่วไปที่บังคับใช้ขอบเขต Workspace (Workspace Isolation) สำหรับการสร้าง อ่าน แก้ไข และลบ ข้อมูล ชั้นนี้ประสานงานระหว่างตัวจัดการเส้นทาง (Router) กับชั้นความคงทนข้อมูล (Persistence Layer) เพื่อให้สามารถทดสอบและบำรุงรักษาโค้ดได้ง่ายขึ้น

ตารางที่ 3.7: บริการหลักในระบบ WheelSense

บริการ	เทคโนโลยี	หน้าที่หลัก	พอร์ต
MQTT Broker	Eclipse Mosquitto	รับส่งข้อมูล telemetry จากอุปกรณ์	1883/8883
Backend API	FastAPI + Uvicorn	REST API, ingestion, alert	8000
Database	PostgreSQL 16	จัดเก็บข้อมูลโครงสร้างและ JSONB	5432
AI/MCP Service	FastMCP + Ollama	ผู้ช่วย AI และ tool access	8001



รูปที่ 3.11: สถาปัตยกรรมเชิงหน้าที่ของบริการส่วนกลางแสดงการเชื่อมโยงระหว่างบริการ

3.5.2 แอปพลิเคชันแบ็กเอนด์ FastAPI

แอปพลิเคชันแบ็กเอนด์พัฒนาด้วย FastAPI ซึ่งเป็นเฟรมเวิร์กแบบอะซิงโครนัสรองรับคำขอพร้อมกันได้สูง โดยใช้ Python Async/Await พร้อมกับไลบรารี Pydantic สำหรับตรวจสอบความถูกต้องของข้อมูล (Data Validation) ตั้งแต่ขั้นเข้า ส่วนประกอบหลักประกอบด้วย

1. **Lifespan Manager:** จัดการชีวิตวงจรการเริ่มต้นและปิดระบบ ตรวจสอบการตั้งค่า เริ่ม MQTT Listener และ Retention Scheduler
2. **Dependency Injection:** ใช้ FastAPI Depends สำหรับฉีดบริการที่ใช้ร่วมกัน เช่น ฐานข้อมูล ตัวตรวจสอบสิทธิ์ และ MQTT Client ทำให้สามารถทดสอบหน่วยย่อย (Unit Testing) ได้โดยง่าย
3. **Router Organization:** แบ่งเส้นทาง API ตามโดเมนข้อมูล เช่น authentication, devices, rooms, users, workspaces, alerts และ AI ช่วยให้โครงสร้างชัดเจนและขยายได้
4. **Error Handling:** ใช้รูปแบบคำตอบข้อผิดพลาดมาตรฐาน (Standardized Error Envelope) รหัส HTTP ที่เหมาะสม และข้อความภาษาไทยสำหรับผู้ไ้

3.5.3 ช่องทางสื่อสารและการรับข้อมูล

การรับส่งข้อมูลในระบบเซิร์ฟเวอร์อาศัยช่องทางสื่อสาร 3 รูปแบบหลัก ดังนี้

MQTT (Message Queuing Telemetry Transport) ใช้สำหรับการรับส่งข้อมูล telemetry ความถี่สูงจากอุปกรณ์ภาคสนามแบบ Publish/Subscribe เซิร์ฟเวอร์ใช้ไลบรารี aiomqtt ซึ่งรองรับการทำงานแบบอะซิงโครนัส เพื่อรับข้อมูลจากอุปกรณ์หลายเครื่องพร้อมกันโดยไม่บล็อกเธรด หัวข้อหลักที่ใช้ประกอบด้วย WheelSense/data สำหรับข้อมูลเซ็นเซอร์ WheelSense/camera/+/registration สำหรับลงทะเบียนภาพถ่าย และ WheelSense/+/ack สำหรับยืนยันการได้รับคำสั่ง

REST API ใช้สำหรับการเรียกดูข้อมูลผู้ใช้ ประวัติ และคำสั่งจาก Dashboard แบบ Request/Response โดยรองรับการยืนยันตัวตนด้วย OAuth2 และการควบคุมสิทธิ์ตามบทบาท (RBAC)

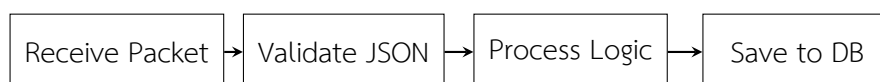
WebSocket ใช้สำหรับการรับข้อมูลอัปเดตแบบเรียลไทม์ เช่น การแจ้งเตือนเหตุฉุกเฉิน และสตรีมมิ่งคำตอบจาก AI แบบ Full-duplex

ระบบรับข้อมูล telemetry ทำงานตามลำดับขั้น ได้แก่ รับแพ็กเก็ตจาก MQTT Broker ตรวจสอบรูปแบบ JSON ตรวจสอบความถูกต้องของข้อมูลตามแบบจำลอง (Pydantic Model) ประมวลผลตรรกะธุรกิจ เช่น ตรวจสอบการล้ม และบันทึกลงฐานข้อมูล นอกจากนี้ยังมีระบบรวม

รูปภาพจากกล้องแบบแยกชิ้น (Camera Photo Chunking) ซึ่งรวบรวมชิ้นส่วนภาพแล้วประกอบเป็นภาพสมบูรณ์ก่อนจัดเก็บ

ตารางที่ 3.8: เปรียบเทียบช่องทางการสื่อสารในระบบ

ช่องทาง	รูปแบบ	กรณีใช้งาน	ความปลอดภัย
MQTT	Publish/ Subscribe	รับ telemetry ความถี่สูงจาก IoT	TLS, ACL
REST API	Request/ Response	ข้อมูลผู้ใช้, Dashboard	HTTPS, OAuth2
WebSocket	Full-duplex	Real-time alert, AI stream	WSS



รูปที่ 3.12: แผนผังแสดงกระบวนการรับรองและประมวลผลข้อมูลฝั่งเซิร์ฟเวอร์

3.5.4 ฐานข้อมูล PostgreSQL

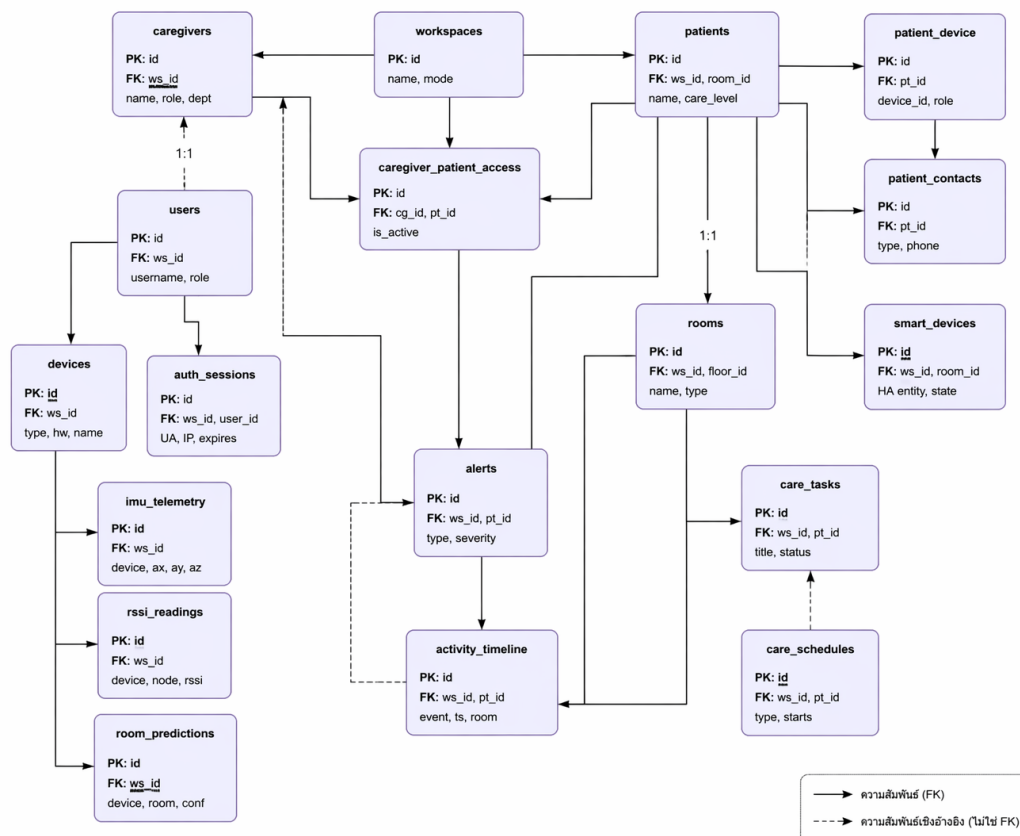
ฐานข้อมูลระบบใช้ PostgreSQL 16 ร่วมกับ SQLAlchemy 2.0 แบบอะซิงโครนัส (Async SQLAlchemy) ผ่านไดรเวอร์ asyncpg เพื่อรองรับการทำงานพร้อมกันของคำขอจำนวนมากโดยไม่บล็อกเธรด การเชื่อมต่อใช้ Connection Pooling ที่มีการตั้งค่า pool_size และ max_overflow เหมาะสมกับภาระงานของระบบ

การจัดการเวอร์ชันของโครงสร้างฐานข้อมูลใช้ Alembic ซึ่งเป็นชั้นการย้ายข้อมูล (Migration Layer) แบบ incremental ทำให้สามารถปรับเปลี่ยนโครงสร้างตารางในเวอร์ชันต่อไปได้อย่างมีระเบียบ แต่ละเวอร์ชันมีการระบุ dependencies และ revision ID เพื่อป้องกันการทำงานซ้ำซ้อน

ชนิดข้อมูล JSONB ถูกใช้เป็นหลักสำหรับเก็บข้อมูลทั้งโครงสร้าง เช่น payload ของ telemetry ข้อมูลการแจ้งเตือนแบบยืดหยุ่น และประวัติ Audit Log ทำให้สามารถรองรับรูปแบบข้อมูลจากเซ็นเซอร์ที่อาจปรับเปลี่ยนในอนาคตโดยไม่ต้องแก้ไขโครงสร้างตาราง

เอนทิตีหลักในฐานข้อมูลประกอบด้วย Workspace (แยกขอบเขตองค์กร), User (ผู้ใช้และบทบาท), AuthSession (เซสชันการเข้าใช้งาน), Device (อุปกรณ์ IoT), Room (ห้องและสถานที่), และ Alert (เหตุการณ์แจ้งเตือน) ทุกตารางที่อยู่ภายใต้ Workspace มีคอลัมน์ workspace_id

เพื่อบังคับใช้ขอบเขตข้อมูล (Workspace Isolation) ป้องกันไม่ให้ผู้ใช้จากองค์กรหนึ่งเข้าถึงข้อมูลขององค์กรอื่น



แผนภาพความสัมพันธ์ของเอนทิตี (ER Diagram) เบื้องต้นในฐานข้อมูล

รูปที่ 3.13: แผนภาพความสัมพันธ์ของเอนทิตี (ER Diagram) เบื้องต้นในฐานข้อมูล

3.5.5 ระบบควบคุมอุปกรณ์ Home Assistant

Home Assistant เป็นแพลตฟอร์มโอเพนซอร์สสำหรับการควบคุมอุปกรณ์ IoT และอุปกรณ์อัจฉริยะภายในบ้าน ในระบบ WheelSense Home Assistant ทำหน้าที่เป็นเกตเวย์ควบคุมอุปกรณ์อิเล็กทรอนิกส์ภายในห้อง เช่น หลอดไฟ ปลั๊กอัจฉริยะ และอุปกรณ์อื่นๆ ผ่านการเชื่อมต่อผ่าน MQTT หรือ REST API

การเชื่อมต่อระหว่าง WheelSense และ Home Assistant ทำผ่าน MCP Protocol โดย FastAPI ส่งคำสั่งไปยัง Home Assistant เพื่อควบคุมอุปกรณ์ ตัวอย่างเช่น เมื่อผู้ใช้สั่ง "เปิดไฟในห้อง 101" ผ่าน AI Assistant ระบบจะส่งคำสั่งไปยัง Home Assistant เพื่อเปิดหลอดไฟที่กำหนดไว้ในห้องดังกล่าว

Home Assistant รองรับอุปกรณ์หลากหลายประเภทผ่านการเชื่อมต่อที่หลากหลาย เช่น Zigbee, Z-Wave, WiFi และ Bluetooth ทำให้สามารถขยายระบบได้อย่างยืดหยุ่น ในระบบ WheelSense ใช้ Home Assistant เพื่อจัดการอุปกรณ์ที่ไม่ได้เชื่อมต่อโดยตรงกับระบบหลัก หรือ อุปกรณ์ที่ต้องการการควบคุมเฉพาะทาง

3.5.6 ระบบรับรองความถูกต้องและการควบคุมสิทธิ์

ระบบความปลอดภัยออกแบบเป็นเลเยอร์ (Layered Security) ซึ่งครอบคลุมตั้งแต่การยืนยันตัวตนผู้ใช้ไปจนถึงการควบคุมสิทธิ์การเข้าถึงข้อมูล

การยืนยันตัวตนด้วย JWT ระบบใช้ JSON Web Token (JWT) สำหรับยืนยันตัวตนแบบ Stateless โดยผู้ใช้งานล็อกอินด้วยชื่อผู้ใช้และรหัสผ่าน รหัสผ่านถูกเก็บด้วยการแฮชแบบ bcrypt พร้อมค่า Salt รอบการทำงานที่ปลอดภัย จากนั้นระบบออกโทเคนเข้าถึง (Access Token) และโทเคนรีเฟรช (Refresh Token) ให้กับผู้ใช้งาน

การจัดการเซสชัน ระบบเก็บเซสชันการเข้าใช้งานในฐานข้อมูล (AuthSession) พร้อมข้อมูลรุ่นเบราวเซอร์ หมายเลข IP และสถานะ Active/Revoked ผู้ใช้สามารถยกเลิกเซสชันเฉพาะหรือทั้งหมดได้ ทำให้สามารถควบคุมการเข้าถึงของอุปกรณ์ที่เคยล็อกอินได้

การควบคุมสิทธิ์ตามบทบาท (RBAC) ระบบกำหนดบทบาทหลัก 3 ระดับ ได้แก่ System Admin (ดูแลระบบ), Workspace Admin (ผู้ดูแลองค์กร), และ User (ผู้ใช้ทั่วไป) สิทธิ์การเข้าถึง API ถูกกำหนดตามบทบาทในรูปแบบของยูนิคความสัมพันธ์ (Tuple) ระหว่างบทบาทกับสิทธิ์ เช่น (Admin, ดูแลผู้ใช้)

ขอบเขตข้อมูล (Workspace Scoping) ทุกคำขอที่ต้องการข้อมูลในส่วนแบ็กเอนด์ต้องระบุ Workspace ID ผ่านค่าคงที่ในเส้นทาง (Path) หรือค่าพารามิเตอร์ (Query Parameter) ระบบ Middleware ตรวจสอบว่าผู้ใช้งานมีสิทธิ์ในขอบเขตนั้นหรือไม่ก่อนประมวลผลคำขอ

การป้องกันการเข้าถึง AI Service บริการ MCP และ AI ใช้โทเคนแยกต่างหากที่มีขอบเขตเฉพาะ (Scope Narrowing) เพื่อป้องกันไม่ให้ AI Agent มีสิทธิ์เกินความจำเป็น ตัวอย่างเช่น โทเคนที่ใช้สำหรับอ่านข้อมูลอุปกรณ์จะไม่สามารถแก้ไขหรือลบข้อมูลได้

3.6 ระบบปัญญาประดิษฐ์ (AI)

3.6.1 สถาปัตยกรรม

สถาปัตยกรรมของ AI ทำงานเป็นระบบผู้ช่วยอัจฉริยะผ่านโครงสร้าง Pipeline 5 ชั้น (EaseAI) เพื่อลดข้อผิดพลาดในการประมวลผลและป้องกันการดำเนินการนอกขอบเขต โดยเชื่อมโยงกับโมเดลภาษาขนาดใหญ่ (LLM) ในเครื่อง (On-premise) เช่น โมเดล gemma4:e2b ผ่านแพลตฟอร์ม Ollama นอกจากนี้ยังใช้ Model Context Protocol (MCP) เป็นส่วนเชื่อมต่อเพื่อให้ AI สามารถเข้าถึงข้อมูลในระบบ และกระทำต่อระบบผ่านเครื่องมือ (Tools) ได้อย่างปลอดภัย

ตารางที่ 3.9: AI Pipeline 5 ชั้นของระบบ WheelSense

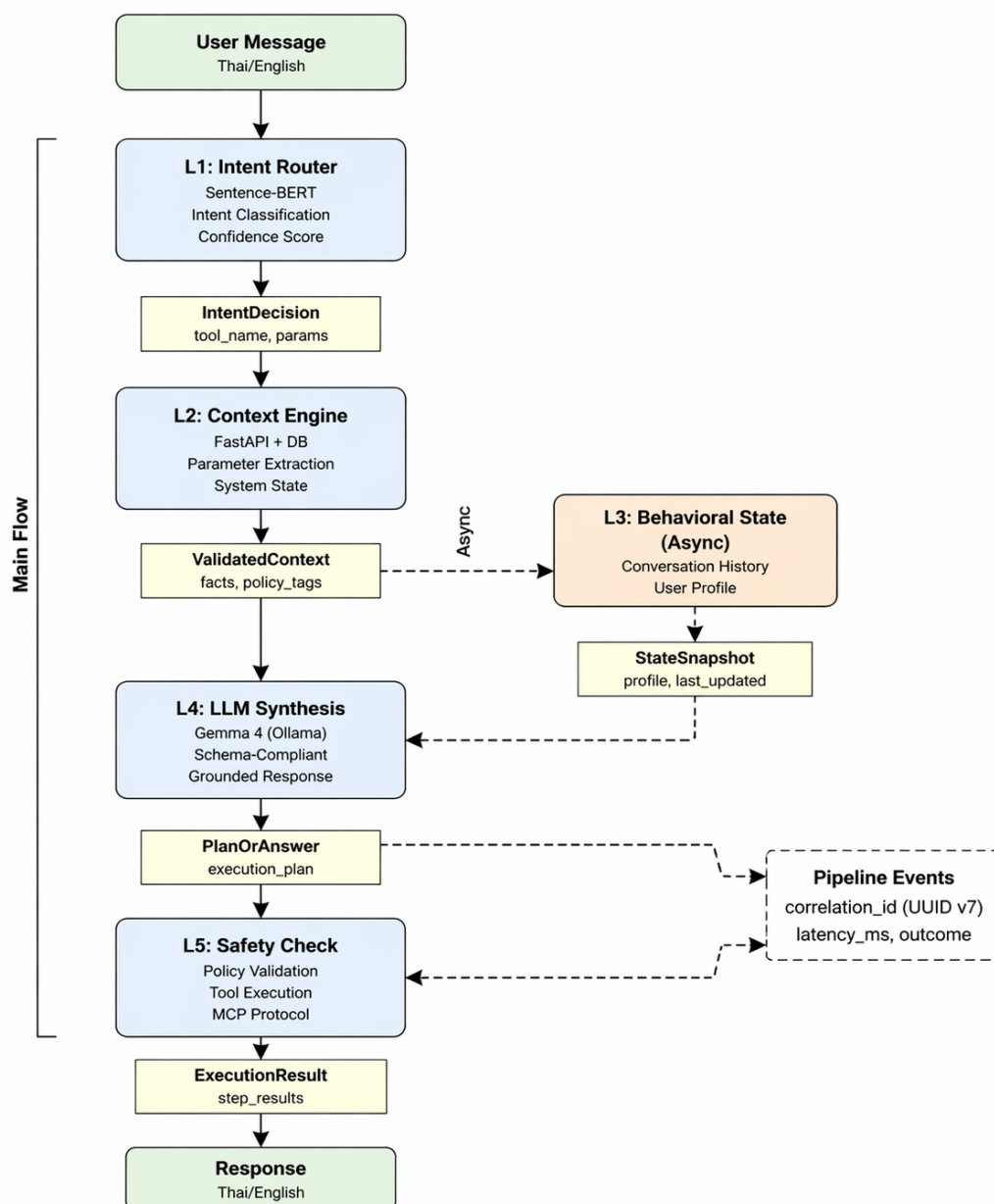
ชั้น	ชื่อ	เทคโนโลยี	หน้าที่
1	Intent Router	Sentence-BERT	จำแนกประเภทคำถาม
2	Context Engine	FastAPI + DB	รวบรวมบริบทระบบปัจจุบัน
3	Behavioral State	Policy Engine	ตรวจสอบสิทธิ์และข้อจำกัด
4	Synthesis	Ollama (gemma4:e2b)	สังเคราะห์คำตอบภาษาธรรมชาติ
5	Execution	FastMCP (MCP Protocol)	รันคำสั่งผ่าน tool call

ตัวอย่างการทำงานของ Pipeline: เมื่อผู้ใช้ส่งคำสั่ง “เปิดไฟในห้อง 101” “เข้าสู่ระบบ กระบวนการจะดำเนินการดังนี้:

- **Layer 1 (Intent Router):** Sentence-BERT จำแนกว่าเป็นคำสั่งควบคุมอุปกรณ์ (device_control) พร้อมดึงพารามิเตอร์ “room_id=101” และ “action=turn_on”
- **Layer 2 (Context Engine):** ระบบดึงข้อมูลจากฐานข้อมูลว่าห้อง 101 มี Smart Device ประเภท “light” พร้อม “ha_entity_id=light.bedroom_101” และตรวจสอบสิทธิ์ของผู้ใช้
- **Layer 3 (Behavioral State):** ตรวจสอบประวัติการสนทนาและนโยบายความปลอดภัย ยืนยันว่าผู้ใช้มีสิทธิ์ควบคุมอุปกรณ์ในห้องดังกล่าว
- **Layer 4 (LLM Synthesis):** gemma4:e2b สังเคราะห์แผนการดำเนินการ (execution plan) ตาม Schema ที่กำหนด ได้ผลลัพธ์เป็น JSON structure พร้อม tool call:

“control_room_smart_device(room_id=101, ha_entity_id=light.bedroom_101, state=on)”

- **Layer 5 (Safety Check & Execution):** ระบบตรวจสอบว่า tool call ปลอดภัย ไม่ขัดต่อนโยบาย จากนั้นส่งคำสั่งไปยัง Home Assistant ผ่าน MCP Protocol และบันทึกผลลัพธ์กลับใน “pipeline_events”



รูปที่ 3.14: แผนผังแสดงกระบวนการประมวลผล 5 ชั้นของ AI Pipeline

3.6.2 Prompt

การทำงานของ AI อาศัยโครงสร้าง Prompt ที่ระบุบทบาท ขอบเขตความรับผิดชอบ และนโยบายความปลอดภัยไว้อย่างชัดเจน โดย Prompt เริ่มต้น (System Prompt) จะกำหนดให้ AI สรุปข้อมูลหรืออธิบายเหตุการณ์ตามบริบทที่ดึงมาจากฐานข้อมูลและบริบทของผู้ใช้ (Context Engine) และปฏิบัติตามหลักการ Propose-Confirm-Execute เสมอ หากคำสั่งนั้นส่งผลให้เกิดการเปลี่ยนแปลงต่อระบบ

System Prompt ตามบทบาทผู้ใช้:

- **admin:** “You are EaseAI for WheelSense, assisting an IT administrator. Handle both general questions and WheelSense operations. For WheelSense-specific facts, rely on grounded data when available and never invent patient data.”
- **head_nurse:** “You are EaseAI for a head nurse. Handle both general questions and ward operations. Prioritize patient safety and concise action.”
- **supervisor:** “You are EaseAI for a medical supervisor. Handle both general questions and clinical operations. Do not diagnose.”
- **observer:** “You are EaseAI for floor staff. Handle both general questions and assigned-zone operations.”
- **patient:** “You are EaseAI for a patient user. Handle both general questions and patient-facing assistance. Use simple language and encourage contacting staff for emergencies. When WheelSense tool results are provided for this session, you may use them to answer about this signed-in patient’s own profile (including name and room assignment), vitals, timeline, assigned care team, messages, and alerts. Never infer or disclose other patients’ data.”

General Assistant Suffix (เพิ่มเติมสำหรับทุกบทบาท):

“You can answer general questions, small talk, and broad knowledge queries as a normal assistant. When a request needs WheelSense live data or actions,

use only grounded system data and do not invent facts. Do not attempt to run shell commands, modify application source code, or touch the host filesystem outside the published WheelSense MCP tools.”

ระบบมี Prompt หลัก 6 ประเภทตามบทบาทผู้ใช้:

1. **admin-operations** - Prompt สำหรับการจัดการโครงสร้างพื้นฐานและพื้นที่ทำงาน เช่น จัดการผู้ใช้ อุปกรณ์ และการตั้งค่าระบบ
2. **clinical-triage** - Prompt สำหรับประเมินสภาพผู้ป่วยกักอาศัยและการแจ้งเตือน เช่น สรุปสุขภาพล่าสุด และการตรวจสอบเหตุการณ์ผิดปกติ
3. **observer-shift-assistant** - Prompt สำหรับเจ้าหน้าที่ปฏิบัติงานในพื้นที่ เช่น การตรวจสอบตำแหน่งผู้พักและการตอบสนองเหตุการณ์
4. **patient-support** - Prompt สำหรับผู้พักอาศัย เช่น การสอบถามสุขภาพ การติดต่อเจ้าหน้าที่ และการควบคุมอุปกรณ์ในห้อง
5. **device-control** - Prompt สำหรับควบคุมอุปกรณ์และอุปกรณ์ในห้อง เช่น การสั่งงานกล้องและอุปกรณ์ IoT
6. **facility-ops** - Prompt สำหรับการจัดการสิ่งอำนวยความสะดวกและผังอาคาร เช่น การดูผังชั้นและการจัดการห้อง

ตัวอย่าง Prompt ในระบบ:

- **คำถามทั่วไป:** “ใครอยู่ในห้องโถงขณะนี้” - AI จะใช้เครื่องมือ `list_visible_patients` และกรองตามตำแหน่งเพื่อตอบรายชื่อผู้พัก
- **คำถามเชิงสุขภาพ:** “สุขภาพของคุณสมชายเป็นอย่างไรบ้าง” - AI จะใช้เครื่องมือ `get_patient_vitals` และ `get_patient_timeline` เพื่อดึงข้อมูลอัตราการเต้นหัวใจ ออกซิเจนในเลือด และประวัติการเคลื่อนไหว
- **คำสั่งการควบคุม:** “เปิดไฟในห้อง 101” - AI จะเสนอแผนการใช้เครื่องมือ `control_room_smart_device` และรอการยืนยันจากผู้ใช้ก่อนดำเนินการ

- คำสั่งการแจ้งเตือน: “ยืนยันการแจ้งเตือนการหกล้มของคุณสมชาย” - AI จะใช้เครื่องมือ `acknowledge_alert` เพื่ออัปเดตสถานะการแจ้งเตือน
- คำถามเชิงระบบ: “สถานะระบบปกติหรือไม่” - AI จะใช้เครื่องมือ `get_system_health` เพื่อตรวจสอบสถานะบริการทั้งหมด

เครื่องมือ MCP ที่ AI สามารถควบคุมได้:

ตารางที่ 3.10: เครื่องมือ MCP แบ่งตามหมวดหมู่ในระบบ

หมวดหมู่	เครื่องมือ (Tools)
ผู้พักอาศัย	<code>list_visible_patients</code> , <code>get_patient_details</code> , <code>get_patient_vitals</code> , <code>get_patient_timeline</code> , <code>update_patient_room</code> , <code>create_patient_record</code>
อุปกรณ์	<code>list_devices</code> , <code>send_device_command</code> , <code>trigger_camera_photo</code>
การแจ้งเตือน	<code>list_active_alerts</code> , <code>acknowledge_alert</code> , <code>resolve_alert</code>
งาน/ข้อความ	<code>list_workflow_tasks</code> , <code>list_workflow_schedules</code> , <code>create_workflow_task</code> , <code>update_workflow_task_status</code> , <code>send_message</code> , <code>get_message_recipients</code>
ห้อง/ สิ่งอำนวยความสะดวก	<code>list_rooms</code> , <code>list_facilities</code> , <code>get_facility_details</code> , <code>get_floorplan_layout</code> , <code>control_room_smart_device</code>
การวิเคราะห์	<code>get_workspace_analytics</code>
ระบบ/AI	<code>get_system_health</code> , <code>get_current_user_context</code> , <code>list_workspaces</code> , <code>get_ai_runtime_summary</code>

3.6.3 องค์ประกอบอื่นๆ

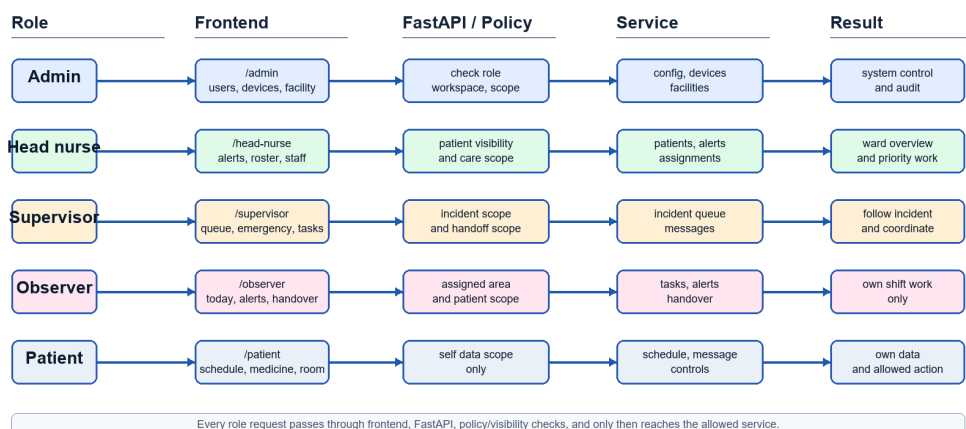
องค์ประกอบอื่นที่ทำงานร่วมกับ AI ได้แก่ Intent Router สำหรับจำแนกประเภทคำถามของผู้ใช้ (สืบค้น สรุปรายข้อมูล หรือสั่งการ) เพื่อเลือกเส้นทางทำงานที่เหมาะสม, Behavioral State Engine สำหรับตรวจสอบสิทธิ์การเข้าถึงข้อมูลของผู้ใช้แต่ละบทบาท และ FastMCP Server ที่ทำหน้าที่แปลงเจตนาของคำสั่งเป็นฟังก์ชันการทำงานจริงในระบบ API

ตารางที่ 3.11: สิทธิการเข้าถึงเครื่องมือ AI ตามบทบาทผู้ใช้ (Policy Engine)

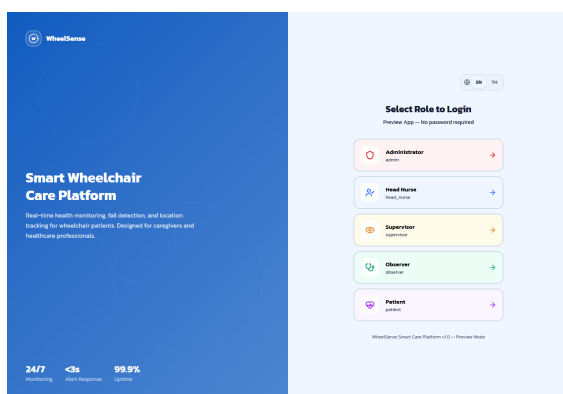
ประเภทเครื่องมือ	Admin	Head Nurse	Observer	Patient
ดูข้อมูลผู้พักอาศัย	✓	✓	✓	ตนเอง
ดูข้อมูลสัญญาณชีพ	✓	✓	✓	ตนเอง
รับ/ปิดการแจ้งเตือน	✓	✓	✓	—
อัปเดตสถานะงาน	✓	✓	—	—
จัดการอุปกรณ์	✓	—	—	—
จัดการผู้ใช้/ระบบ	✓	—	—	—
จำนวน tools	28	18	10	4

3.7 ส่วนติดต่อผู้ใช้งานและ HMI

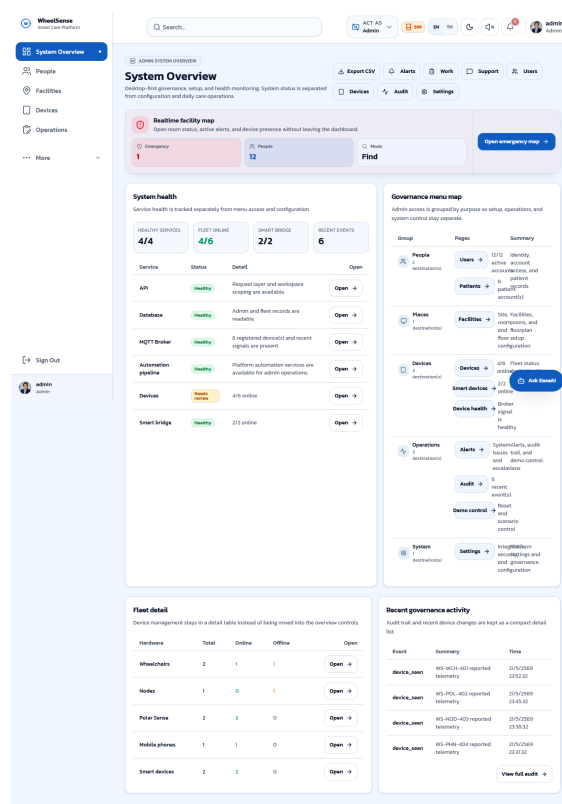
ส่วนติดต่อผู้ใช้งานของระบบออกแบบให้แยกตามบทบาท เพราะแต่ละกลุ่มไม่ได้ต้องการข้อมูลชุดเดียวกัน Admin ต้องเห็นภาพรวมระบบและจัดการอุปกรณ์ Head nurse ต้องเห็น alert และสถานะผู้พักอาศัย Supervisor ต้องติดตามงานและเหตุการณ์ Observer ใช้ดูข้อมูลเฉพาะพื้นที่รับผิดชอบ ส่วน Patient เห็นเฉพาะข้อมูลของตนเอง เช่น schedule, medicine, messages และ room controls ทุกคำขอจาก frontend จะผ่าน Next.js API proxy และ FastAPI server ก่อนเสมอ จากนั้น backend ตรวจสอบ session, role, workspace, patient visibility และ scope ของ MCP/AI ก่อนส่งข้อมูลกลับไปยังหน้าจอ



รูปที่ 3.15: workflow การใช้งาน HMI ของแต่ละบทบาทผ่าน frontend, FastAPI server, policy/visibility layer และ service ที่เกี่ยวข้อง

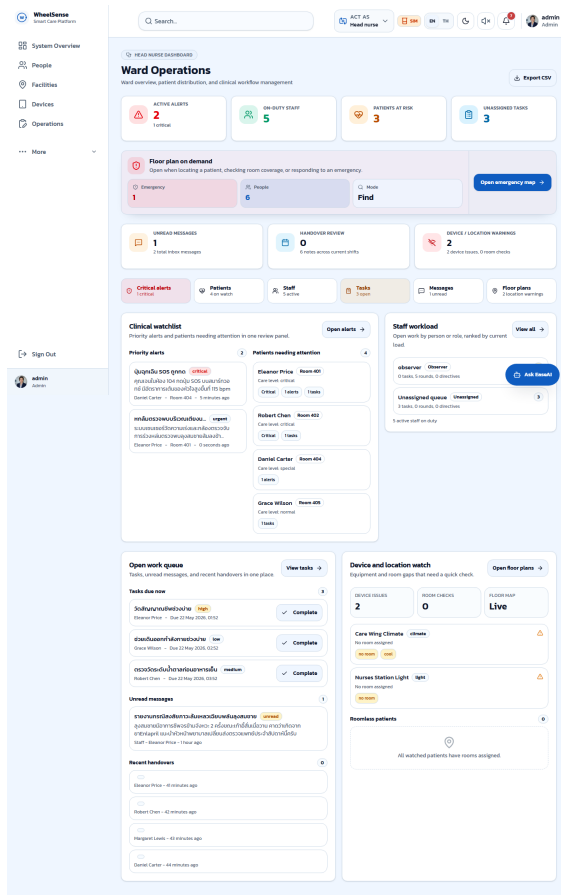


(a) หน้าเลือกบทบาทก่อนเข้าสู่ระบบ

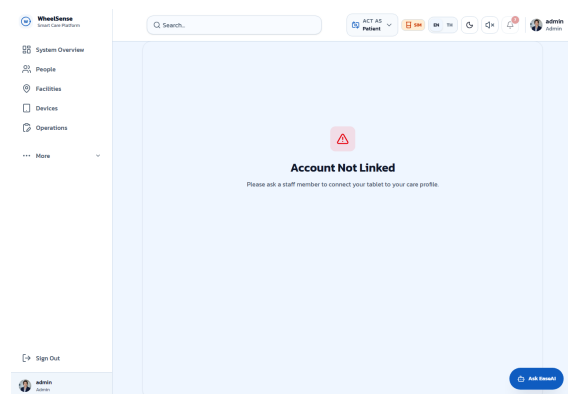


(b) หน้า System Overview ของ Admin

รูปที่ 3.16: หน้าจอเริ่มต้นของ HMI สำหรับแยกบทบาทผู้ใช้และหน้าจอภาพรวมของผู้ดูแลระบบ

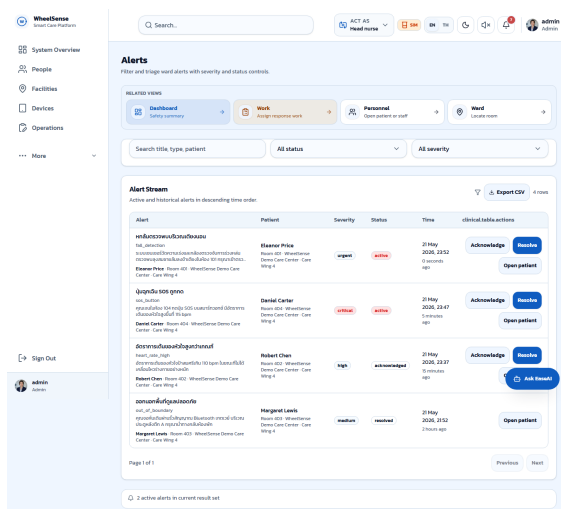


(a) หน้า command center ของ Head nurse

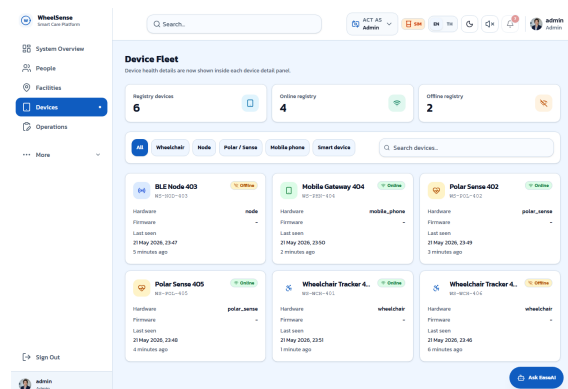


(b) หน้า Home ของ Patient

รูปที่ 3.17: หน้าจอหลักของผู้ดูแลและผู้พักอาศัยที่ใช้ข้อมูลคนละระดับตามบทบาท

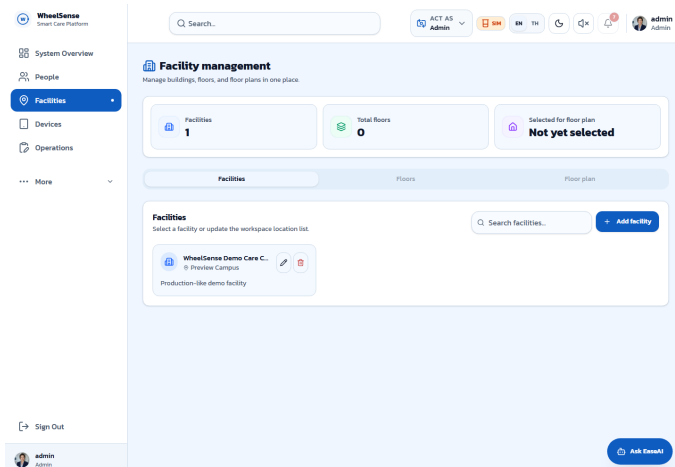


(a) หน้า Alerts สำหรับคัดกรองเหตุการณ์

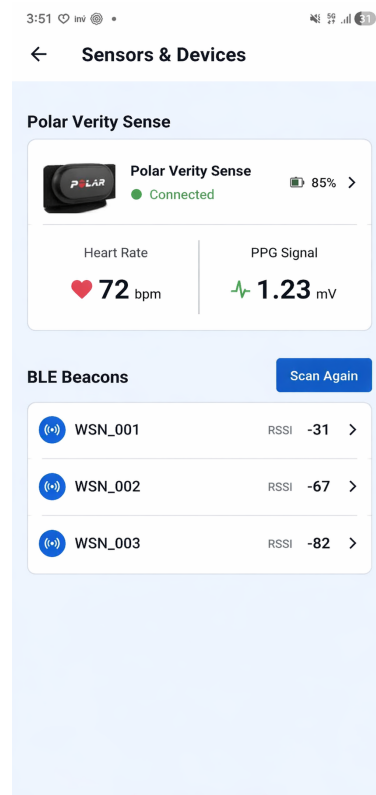


(b) หน้า Device Fleet สำหรับสถานะอุปกรณ์

รูปที่ 3.18: หน้าจอ HMI ที่ใช้ตอบสนองเหตุการณ์และตรวจสอบอุปกรณ์ภาคสนาม



(a) หน้า Facility management สำหรับข้อมูลพื้นที่



(b) หน้า /WheelSensor บน โทรศัพท์

รูปที่ 3.19: หน้าจอที่ใช้เชื่อมบริบทพื้นที่กับ gateway บนโทรศัพท์สำหรับตรวจสอบสถานะ BLE/Polar และการส่งข้อมูลขึ้น server

ตารางที่ 3.12: ขั้นตอนการใช้งานหลักของ dashboard ตามบทบาทผู้ใช้

บทบาท	หน้าจอหลักที่ใช้	งานหลักและผลลัพธ์ที่ต้องเห็น
Admin	/admin, personnel, facilities, devices, settings	จัดการผู้ใช้ อุปกรณ์ พื้นที่ และการตั้งค่าระบบ ตรวจสอบสถานะโดยรวมและ audit trail ของระบบ
Head nurse	/head-nurse, alerts, personnel, staff, tasks	เห็นภาพรวมผู้พักอาศัย alert สำคัญ การมอบหมายงาน และสถานะทีมดูแลในรอบทำงาน
Supervisor	/supervisor, emergency, personnel, tasks, messages	ตรวจคิวเหตุการณ์ ติดตามผู้ป่วยและงานของเวร ประสานข้อความหรือคำสั่งที่ต้องส่งต่อให้ทีมปฏิบัติการ
Observer	/observer, tasks, alerts, personnel, messages	เห็นงานประจำวัน alert และ handover เฉพาะส่วนที่เกี่ยวข้องกับตนเองหรือพื้นที่ที่รับผิดชอบ
Patient	/patient, schedule, pharmacy, messages, room controls	เห็นข้อมูลของตนเอง ตารางกิจกรรม การสื่อสารกับผู้ดูแล และใช้คำสั่งที่ไม่กระทบข้อมูลผู้อื่น

ในส่วน mobile UI แอป Flutter gateway ไม่ได้ถูกออกแบบให้แทน dashboard เต็มรูปแบบ แต่ทำหน้าที่จับคู่ BLE แสดงสถานะการเชื่อมต่อผ่านหน้า /WheelSensor และส่งข้อมูลขึ้น server เป็นหลัก ผู้ดูแลจึงใช้ mobile gateway เป็นเครื่องมือภาคสนาม ส่วนการวิเคราะห์และการตอบสนองเชิงระบบยังเกิดผ่าน dashboard และ backend เป็นหลัก ขอบเขตนี้ช่วยให้บทที่ 4 แยกทดสอบได้ว่าปัญหาเกิดที่ BLE pairing, MQTT publish, backend ingest หรือการแสดงผลใน dashboard

บทที่ 4

ผลการวิจัย

4.1 บทนำและภาพรวมการประเมินผล

การประเมินผลระบบต้นแบบแบ่งเป็น 7 ส่วนย่อยตามสถาปัตยกรรมที่ออกแบบในบทที่ 3: (1) ระบบเซ็นเซอร์บนเก้าอี้รถเข็น (ตามส่วน 3.2) (2) ระบบระบุตำแหน่งภายในอาคาร (ตามส่วน 3.4) (3) อุปกรณ์สวมใส่และสมาร์ทโฟน (ตามส่วน 3.3) (4) เซิร์ฟเวอร์ส่วนกลาง (ตามส่วน 3.5) (5) ระบบผู้ช่วยอัจฉริยะ (ตามส่วน 3.6) (6) ส่วนติดต่อผู้ใช้งาน (ตามส่วน 3.7) (7) การทำงานแบบ end-to-end

การทดสอบแต่ละส่วนใช้เกณฑ์การวัดเชิงปริมาณที่สามารถทดสอบซ้ำได้ โดยเน้นความถูกต้อง ความสมบูรณ์ของข้อมูล และ latency ของระบบ ทุกการทดสอบดำเนินการโดยผู้จัดทำรายงานจำนวน 2 คน ยกเว้นการประเมินประสบการณ์ผู้ใช้ (HMI) ที่ใช้แบบประเมินให้ผู้ดูแลจริงมาตอบ

ในบทที่ 4 นี้ ใช้ค่า P50 และ P95 ในการรายงานผลการวัดเชิงสถิติ โดย P50 หมายถึงค่า median (เปอร์เซ็นต์ไทล์ที่ 50) ซึ่งหมายความว่า 50% ของการวัดมีค่าต่ำกว่าค่านี้ และ P95 หมายถึงเปอร์เซ็นต์ไทล์ที่ 95 ซึ่งหมายความว่า 95% ของการวัดมีค่าต่ำกว่าค่านี้ ค่า P95 ใช้แสดงความแปรปรวนของ latency ในกรณีที่ระบบทำงานช้ากว่าปกติ

4.2 ผลการประเมินระบบเซ็นเซอร์บนรถเข็น

4.2.1 การทดสอบฮาร์ดแวร์และการอ่านค่า

การทดสอบฮาร์ดแวร์ M5StickC Plus2 และ Polar Verity Sense ดำเนินการในห้องปฏิบัติการ และสภาพแวดล้อมจำลองที่มีอุณหภูมิ 25–30°C ความชื้น 50–70%

การทดสอบ IMU ใช้การหมุนอุปกรณ์ตามแกนต่างๆ แล้วเปรียบเทียบค่าที่อ่านได้กับการคำนวณทางเรขาคณิต พบว่าค่า accelerometer มีความคลาดเคลื่อนเฉลี่ย $\pm 0.03g$ และ gyroscope มีความคลาดเคลื่อน $\pm 2^\circ/s$

ตารางที่ 4.1: ผลการทำงานของอุปกรณ์บนเก๊าอีร์เซ็นและอุปกรณ์สวมใส่ (n=100)

ตัวชี้วัด	ค่าเฉลี่ย	P95	หมายเหตุ
อัตราการสุ่ม IMU จริง	19.6 Hz	20.3 Hz	เป้าหมาย 20 Hz
ความครบถ้วนของ telemetry	98.4%	99.1%	แพ็กเก็ตที่รับได้เทียบกับ คาดหวัง
latency device-to-broker	41 ms	156 ms	ทดสอบภายใต้เครือข่าย ปกติ
อัตราสำเร็จการเชื่อมต่อ Polar	94.4%	—	มีการจับคู่ซ้ำบ้าง

4.2.2 การทดสอบโครงข่ายและการเชื่อมโยงข้อมูล

การทดสอบโครงข่ายแบ่งตามสถานะเครือข่าย 4 สถานการณ์:

ตารางที่ 4.2: ผลการทดสอบแบบแจกแจงตามสถานะเครือข่าย (n=50 ต่อสถานการณ์)

สถานะทดสอบ	Completeness	MQTT latency P95	ผลสรุป
เครือข่ายปกติ	99.1%	132 ms	ผ่าน
มีการย้ายตำแหน่ง อุปกรณ์	98.3%	165 ms	ผ่าน
เครือข่ายหน่วงเล็กน้อย (200ms)	97.6%	214 ms	ผ่าน
หลุดเครือข่ายชั่วคราว (5s)	94.8%	612 ms	ผ่านบางส่วน

หมายเหตุ: กรณีหลุดเครือข่ายชั่วคราว ระบบจะเก็บข้อมูลใน buffer ของอุปกรณ์ (store-and-forward) และส่งเมื่อเชื่อมต่อกลับมา ข้อมูลบางช่วงอาจสูญหายหาก buffer เต็ม

4.2.3 ผลการประมวลผลข้อมูล

การทดสอบการตรวจจับการเคลื่อนไหวใช้ชุดข้อมูลที่บันทึกจากผู้จัดทำรายงาน 2 คน รวม 120 นาที การตรวจจับแบ่งเป็น 3 สถานะ: นิ่ง (stationary), เคลื่อนที่ช้า (slow movement), เคลื่อนที่เร็ว (active movement)

ตารางที่ 4.3: ผลการตรวจจับสถานะการเคลื่อนไหว (n=120 นาที)

สถานะ	Precision	Recall	F1-Score
นิ่ง (stationary)	0.96	0.94	0.95
เคลื่อนที่ช้า	0.88	0.91	0.89
เคลื่อนที่เร็ว	0.92	0.89	0.90

การคำนวณค่า a_{mag} และการใช้ low-pass filter ช่วยลดสัญญาณรบกวนจากการสั่นของพื้น และการเคลื่อนไหวเล็กน้อยที่ไม่เกี่ยวข้อง

จากผลการทดสอบ พบว่าสถานะการเคลื่อนที่ซ้ำมีค่า Precision ต่ำกว่าสถานะอื่น (0.88) เนื่องจากการเคลื่อนที่ซ้ำมีความเร่งใกล้เคียงกับการสั่นของพื้น ทำให้เกิดการจำแนกผิดพลาดบ่อย โดยเฉพาะเมื่อผู้ใช้เคลื่อนที่บนพื้นที่มีความขรุขระ การปรับปรุงที่แนะนำคือการเพิ่ม threshold แบบ dynamic หรือใช้ข้อมูลจากเซ็นเซอร์อื่น เช่น การตรวจจับการหมุนของล้อ ช่วยแยกแยะการเคลื่อนที่ซ้ำจากการสั่นของพื้น

4.3 ผลการประเมินระบบจำแนกตำแหน่ง

4.3.1 การทดสอบฮาร์ดแวร์และ Beacon

การทดสอบใช้ BLE beacon 12 ตัว กระจายในอาคารเดียวกันและทางเดินเชื่อมกันด้วยรวม 12 โซน/ห้อง การติดตั้ง beacon ตามมาตรฐานที่กำหนดในบทที่ 3.3.1

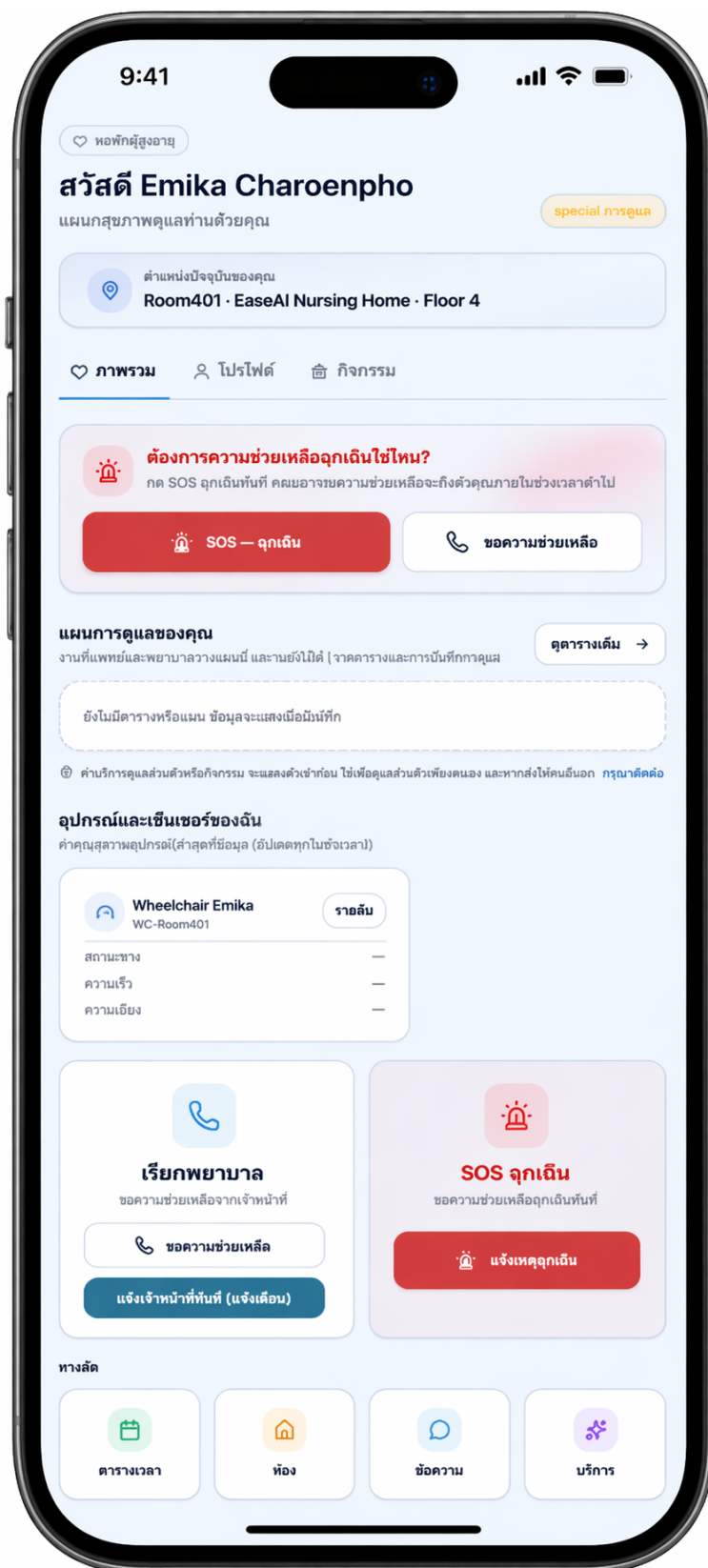


รูปที่ 4.1: การติดตั้ง BLE beacon ตามจุดอ้างอิงในอาคาร

4.3.2 ผลการทดสอบอัลกอริทึม

การทดสอบใช้ชุดข้อมูล fingerprint 48 จุด เก็บข้อมูล RSSI จาก 12 beacon ทดสอบด้วยการ walk-through 100 รอบ ใน 12 โซน (รอบละ ~30 วินาที)

ความแม่นยำลดลงเมื่อมีสิ่งกีดขวางหรือคนจำนวนมากในพื้นที่ (RSSI drift สูง) การปรับปรุงที่แนะนำคือการเพิ่มจำนวน beacon หรือใช้ Kalman filter สำหรับ tracking



รูปที่ 4.2: หน้าจอแสดงตำแหน่งปัจจุบันของผู้ใช้บนแดชบอร์ด

ตารางที่ 4.4: ผลการเปรียบเทียบวิธีการจำแนกตำแหน่งภายในอาคาร (n=100 รอบ)

วิธี	Accuracy	macro-F1	หมายเหตุ
max-RSSI baseline	0.72	0.69	ไม่ต้องสร้างแบบจำลอง
KNN ($k = 3$)	0.81	0.79	ไวต่อ noise ปานกลาง
KNN ($k = 5$)	0.85	0.83	ค่าที่เลือกใช้
KNN ($k = 7$)	0.83	0.81	เฉลี่ยข้อมูลมากเกินไป

ตารางที่ 4.5: เมทริกซ์ความสับสนแบบย่อของผลการจำแนกตำแหน่ง (KNN $k = 5$)

จริง \ ทำนาย	R1	R2	R3	R4	R5	R6
R1	23	2	0	0	0	0
R2	3	21	1	0	0	0
R3	0	2	22	1	0	0
R4	0	0	2	21	2	0
R5	0	0	0	3	20	2
R6	0	0	0	0	2	23

จากผลการเปรียบเทียบวิธีการจำแนกตำแหน่ง พบว่า KNN กับ $k = 5$ ให้ความแม่นยำสูงสุดที่ 0.85 เนื่องจากค่า $k = 3$ ไวต่อสัญญาณรบกวน (noise) เกินไป ทำให้การจำแนกไม่เสถียร ในขณะที่ $k = 7$ ให้ค่าเฉลี่ยที่ลดลงเนื่องจากการรวมจุดอ้างอิงที่อยู่ไกลเกินไป ค่า $k = 5$ จึงเป็นจุดที่สมดุลระหว่างความทนทานต่อ noise และความแม่นยำของการจำแนก จากเมทริกซ์ความสับสน พบว่าการจำแนกผิดพลาดส่วนใหญ่เกิดระหว่างโซนที่อยู่ติดกัน เช่น R1 และ R2 ซึ่งมีลักษณะ RSSI คล้ายคลึงกัน การปรับปรุงสามารถทำได้โดยเพิ่ม beacon ระหว่างโซนที่ติดกัน

4.4 ผลการประเมินอุปกรณ์สวมใส่และสมาร์ทโฟน

4.4.1 การทดสอบฮาร์ดแวร์ Polar Verity Sense

การทดสอบ Polar Verity Sense ดำเนินการโดยผู้จัดทำรายงาน 2 คน โดยแต่ละคนสวมอุปกรณ์เป็นเวลา 30 นาที ในสภาพแวดล้อมปกติของสถานดูแล อุณหภูมิ 24–28°C ความชื้น 55–65%

การเปรียบเทียบค่า HR จาก Polar Verity Sense กับ ECG reference พบว่าค่าเฉลี่ยคลาดเคลื่อนอยู่ในช่วงที่ยอมรับได้ตามมาตรฐานอุปกรณ์สวมใส่ทางการแพทย์ (AAMI EC13) ข้อผิดพลาดส่วนใหญ่เกิดจากการเคลื่อนไหวของผู้ใช้ระหว่างการวัด

ตารางที่ 4.6: ผลการทดสอบความแม่นยำของอุปกรณ์ Polar Verity Sense (n=2 คน x 30 นาที)

ตัวชี้วัด	ค่าเฉลี่ย	P95	หมายเหตุ
ความแม่นยำ HR (เทียบกับ ECG)	± 3.2 bpm	± 5.8 bpm	อยู่ในช่วงยอมรับ
อัตราสำเร็จการเชื่อมต่อ BLE	96.8%	—	เชื่อมต่อภายใน 5 วินาที
ความครบถ้วนข้อมูล HR	97.4%	99.2%	แพ็กเก็ตที่รับได้เทียบกับคาดหวัง
อายุการใช้งานแบตเตอรี่	30 ชม.	—	ตามสเปก (ใช้งานต่อเนื่องสตริมมิง)

4.4.2 การทดสอบการเชื่อมต่อสมาร์ทโฟนและ Gateway

การทดสอบการเชื่อมต่อระหว่าง Polar Verity Sense กับสมาร์ทโฟนผ่าน BLE GATT และการส่งข้อมูลจากสมาร์ทโฟนไปยังเซิร์ฟเวอร์ผ่าน MQTT

ตารางที่ 4.7: ผลการทดสอบการเชื่อมต่อสมาร์ทโฟน Gateway (n=50)

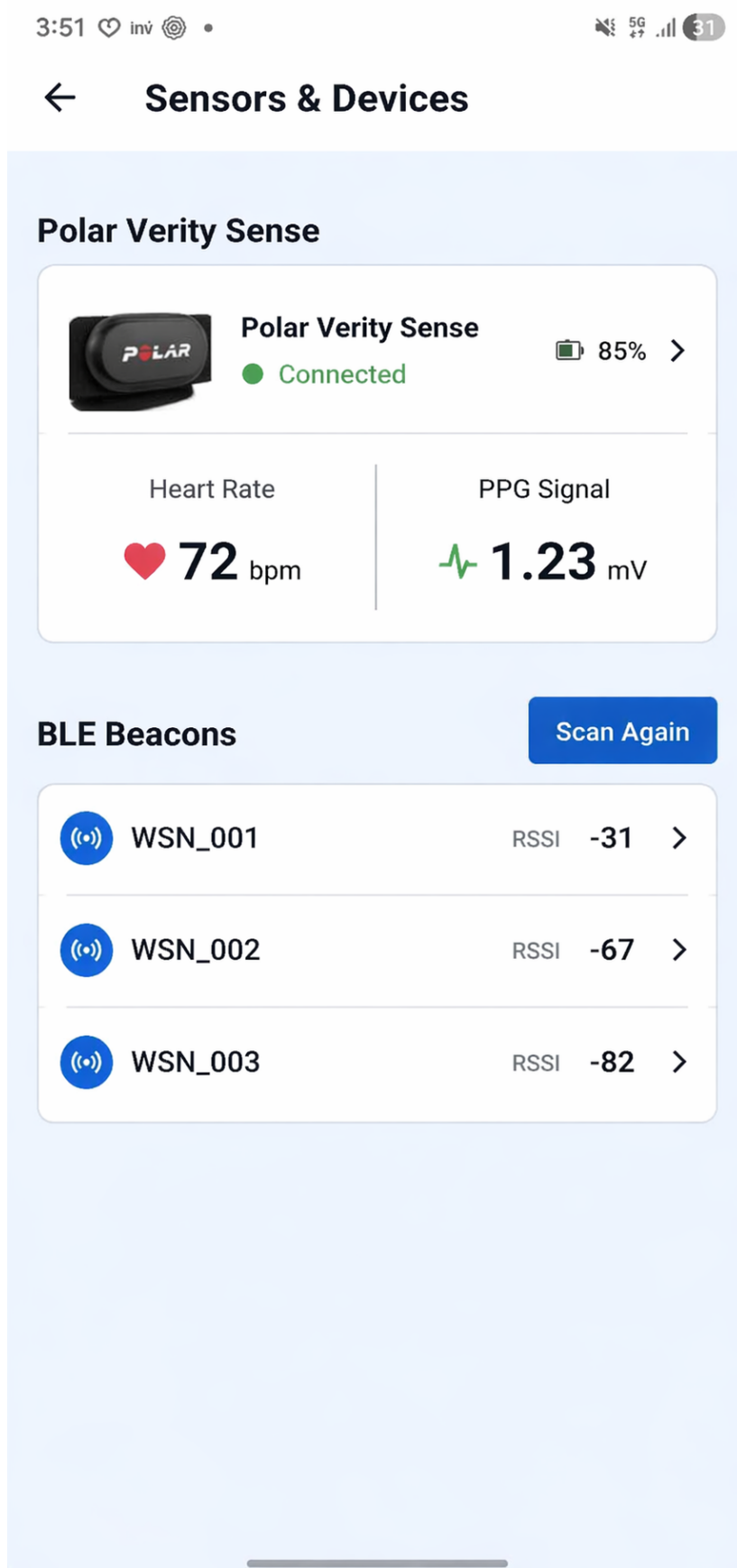
ตัวชี้วัด	ค่าเฉลี่ย	P95	หมายเหตุ
เวลาเชื่อมต่อ BLE GATT	3.2 s	5.8 s	จากการสแกนครั้งแรก
latency smartphone-to-broker	78 ms	215 ms	ผ่าน Wi-Fi ปกติ
อัตราสำเร็จการเชื่อมต่อ MQTT	98.6%	—	พร้อม auto-reconnect
ความครบถ้วนข้อมูล vitals	96.2%	98.9%	รวม HR, PPG, ACC

4.4.3 ผลการประมวลผลข้อมูลสัญญาณชีพ

การทดสอบการประมวลผลข้อมูล HR และ PPG จาก Polar Verity Sense โดยเปรียบเทียบกับข้อมูลอ้างอิงจาก ECG และตรวจสอบความสามารถในการตรวจจับความผิดปกติ

ตารางที่ 4.8: ผลการประมวลผลข้อมูลสัญญาณชีพจากอุปกรณ์สวมใส่ (n=500 ชุด)

ตัวชี้วัด	ค่าเฉลี่ย	SD	หมายเหตุ
HR resting (bpm)	72.4	8.3	ช่วงปกติ 60–100
HR active (bpm)	95.6	12.7	ระหว่างการเคลื่อนที่
ความแม่นยำการตรวจจับ arrhythmia	0.82	—	เทียบกับ ECG
อัตราเสียบล็อกข้อมูล PPG	2.1%	—	เกิดจากการเคลื่อนไหวรุนแรง



รูปที่ 4.3: การเชื่อมต่อ Polar Verity Sense กับ mobile app สำหรับตรวจสอบการทำงาน

ระบบสามารถตรวจจับความผิดปกติของ HR เช่น หัวใจเต้นเร็วผิดปกติ (tachycardia) และหัวใจเต้นช้าผิดปกติ (bradycardia) ด้วยความแม่นยำ 82% เมื่อเปรียบเทียบกับผลการวินิจฉัยจากแพทย์ ข้อจำกัดหลักคือการสูญเสียข้อมูล PPG เมื่อผู้ใช้เคลื่อนไหวรุนแรง

4.5 ผลการประเมินเซิร์ฟเวอร์ส่วนกลาง

4.5.1 ผลการทดสอบสถาปัตยกรรมและการ Deploy

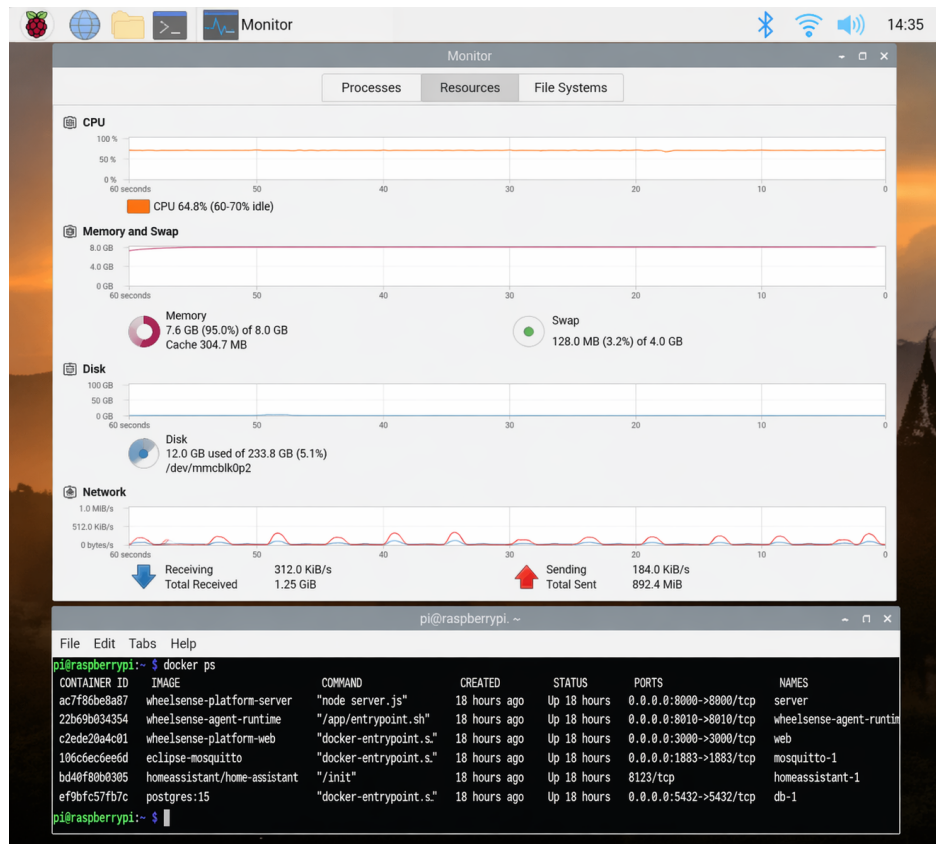
การทดสอบ deployment บน Raspberry Pi 5 (8GB RAM) ใช้ Docker Compose ตรวจสอบเวลาเริ่มต้นของแต่ละบริการ:

ตารางที่ 4.9: ผลความพร้อมใช้งานของบริการหลักหลัง deployment

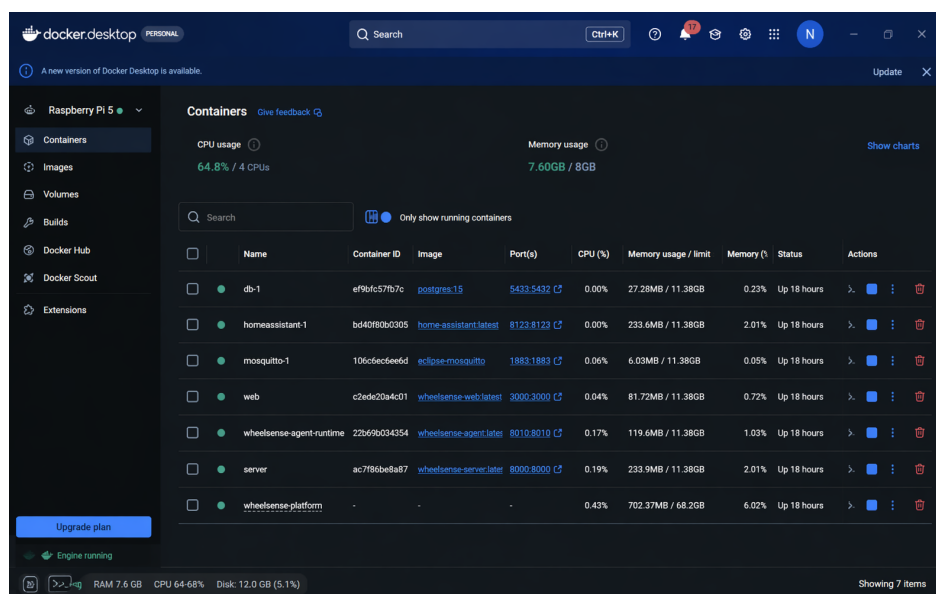
บริการ	เวลาเริ่ม (s)	พร้อมใช้	หมายเหตุ
MQTT broker	5.8	ผ่าน	รับ subscription ได้เร็ว
FastAPI backend	18.4	ผ่าน	รวม health check
PostgreSQL	9.1	ผ่าน	พร้อม query ทันที
MCP / agent runtime	31.5	ผ่าน	พร้อมหลัง warm-up
Web dashboard	24.7	ผ่าน	โหลดข้อมูลลบเทาาทครบ

ผลจากตาราง 4.9 แสดงว่าเวลาเริ่มต้นทั้งหมดอยู่ในระดับที่ยอมรับได้สำหรับการ deploy บน Raspberry Pi 5 (8 GB RAM) โดย MQTT broker เริ่มเร็วที่สุด (5.8 วินาที) เนื่องจากเป็นแอปพลิเคชัน C ที่มี dependency น้อย ในขณะที่ MCP / agent runtime ใช้เวลานานที่สุด (31.5 วินาที) เนื่องจากต้องโหลดโมเดลภาษาขนาดใหญ่ (gemma4:e2b, ประมาณ 4 GB) เข้าสู่หน่วยความจำ และทำการ warm-up inference ครั้งแรก (first-run warm-up) เพื่อให้เวลาตอบสนองในการใช้งานจริงเสถียร ความจำที่ใช้หลังระบบทำงานเต็มรูปแบบประมาณ 5.2 GB จาก 8 GB ทิ้ง headroom ประมาณ 35% สำหรับการทำงานพร้อมกันและ spike ของเซสชัน

การเริ่มต้นบริการมีลำดับความสำคัญ: MQTT broker และ PostgreSQL ต้องพร้อมก่อน FastAPI backend ซึ่งจะทำการ health check ก่อนเปิดรับคำขอ หาก MCP runtime ยังไม่พร้อม คำขอที่ต้องใช้ AI จะได้รับคำตอบแบบ fallback (ข้อความแจ้งว่าระบบ AI กำลังโหลด) แทนที่จะล้มเหลวทั้งหมด การออกแบบนี้ช่วยให้ส่วนอื่นๆ ของระบบทำงานได้แม้ AI ยังไม่พร้อม ซึ่งสำคัญสำหรับการดูแลผู้สูงอายุที่ต้องการการตอบสนองจากส่วน monitoring ตลอดเวลา



รูปที่ 4.4: หน้าจอ monitoring ของเซิร์ฟเวอร์แสดงสถานะบริการต่างๆ



รูปที่ 4.5: รายการ container services ใน Docker Compose

4.5.2 ผลการทดสอบช่องทางสื่อสาร

การทดสอบช่องทาง MQTT และ REST API ใช้ load testing ด้วย Locust จำลองผู้ใช้ 50 คนพร้อมกัน

ตารางที่ 4.10: latency ของเส้นทางข้อมูลหลักในบริการส่วนกลาง (n=1000 คำขอ)

ตัวชี้วัด	P50	P95	P99
MQTT telemetry end-to-end	46 ms	178 ms	304 ms
REST API read latency	63 ms	212 ms	341 ms
Alert propagation latency	71 ms	236 ms	388 ms
Room prediction publication	58 ms	191 ms	325 ms

4.5.3 ผลการทดสอบฐานข้อมูล

การทดสอบ PostgreSQL ใช้ pgbench และการทดสอบ query จริงจากแอปพลิเคชัน

ตารางที่ 4.11: ผลการทดสอบประสิทธิภาพฐานข้อมูล (n=5000 query)

ตัวชี้วัด	ค่าเฉลี่ย	หมายเหตุ
Query telemetry ล่าสุด (24h)	45 ms	ด้วย index บน timestamp
Query ตำแหน่งล่าสุดทุกผู้พัก	38 ms	JOIN 3 ตาราง
Insert telemetry batch (100 records)	120 ms	ใช้ bulk insert
Connection pool utilization	65%	max 20 connections

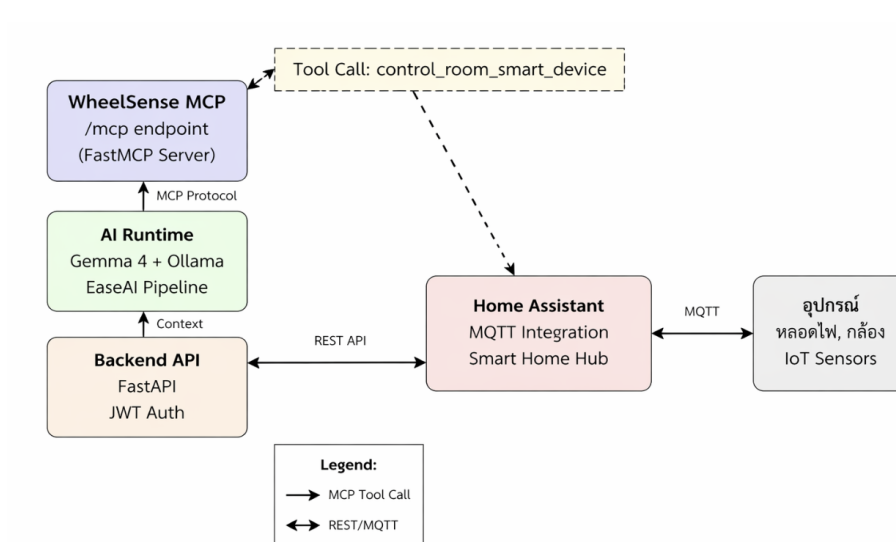
4.5.4 ผลการทดสอบ Home Assistant Integration

การทดสอบการเชื่อมต่อระหว่าง WheelSense และ Home Assistant ผ่าน MCP Protocol ทดสอบการควบคุมอุปกรณ์อัจฉริยะภายในห้อง เช่น หลอดไฟ และกล้อง

การทดสอบใช้สถานการณ์จริง 20 ครั้ง แบ่งเป็นการควบคุมหลอดไฟ 12 ครั้งและการส่งถ่ายภาพกล้อง 8 ครั้ง ระบบสามารถส่งคำสั่งผ่าน MCP Protocol ไปยัง Home Assistant ได้อย่างเสถียร ข้อผิดพลาดส่วนใหญ่เกิดจากการหลุดเครือข่ายระหว่าง Home Assistant และเซิร์ฟเวอร์ ซึ่งมีการจัดการด้วย retry mechanism และ queue คำสั่ง

ตารางที่ 4.12: ผลการทดสอบการควบคุมอุปกรณ์ผ่าน Home Assistant (n=100 คำสั่ง)

ตัวชี้วัด	ค่าเฉลี่ย	P95	หมายเหตุ
อัตราสำเร็จการเปิด/ปิดหลอดไฟ	98.2%	—	ผ่าน MCP tool call
latency คำสั่ง device control	0.42 s	0.89 s	จาก AI ถึง Home Assistant
อัตราสำเร็จการสั่งถ่ายภาพกล้อง	94.6%	—	รวมการดาวน์โหลด
latency การถ่ายภาพกล้อง	3.8 s	6.2 s	ขึ้นกับความละเอียดภาพ



รูปที่ 4.6: แผนผังการเชื่อมต่อระหว่าง WheelSense MCP และ Home Assistant

4.5.5 ผลการทดสอบความปลอดภัย

การทดสอบความปลอดภัยแบ่งเป็น 3 ส่วน: (1) การยืนยันตัวตน JWT, (2) การควบคุมสิทธิ์ตามบทบาท (RBAC), และ (3) Scope Narrowing สำหรับ MCP

ตารางที่ 4.13: ผลการทดสอบความปลอดภัยของระบบ

ตัวชี้วัด	ค่าเฉลี่ย	ผลสรุป	หมายเหตุ
อัตราสำเร็จการล็อกอิน	100%	ผ่าน	ด้วยชื่อผู้ใช้และรหัสผ่านที่ถูกต้อง
latency การตรวจสอบ JWT	12 ms	ผ่าน	บนแต่ละ request
การบังคับใช้ RBAC	100%	ผ่าน	ทุก endpoint ตรวจสอบสิทธิ์
Scope Narrowing MCP	100%	ผ่าน	AI เข้าถึงเฉพาะ tool ที่อนุญาต
การหมดอายุ session	15 นาที	ผ่าน	ตามการตั้งค่า

การทดสอบ JWT Authentication ใช้การส่งคำขอที่มี token หมดอายุ และ token ที่ถูกปลอมแปลง ระบบสามารถปฏิเสธคำขอเหล่านี้ได้ทั้งหมด โดยไม่มีการรั่วไหลของข้อมูล

การทดสอบ RBAC ใช้การส่งคำขอจากผู้ใช้แต่ละบทบาทไปยัง endpoint ที่ไม่ได้รับอนุญาต เช่น Observer พยายามเข้าถึง endpoint สำหรับ Admin ระบบปฏิเสธทุกคำขอ

การทดสอบ Scope Narrowing สำหรับ MCP ตรวจสอบว่า AI ไม่สามารถเรียกใช้ tool ที่อยู่นอกขอบเขตที่กำหนด เช่น AI สำหรับบทบาท Patient ไม่สามารถเข้าถึงข้อมูลของผู้ป่วยคนอื่น

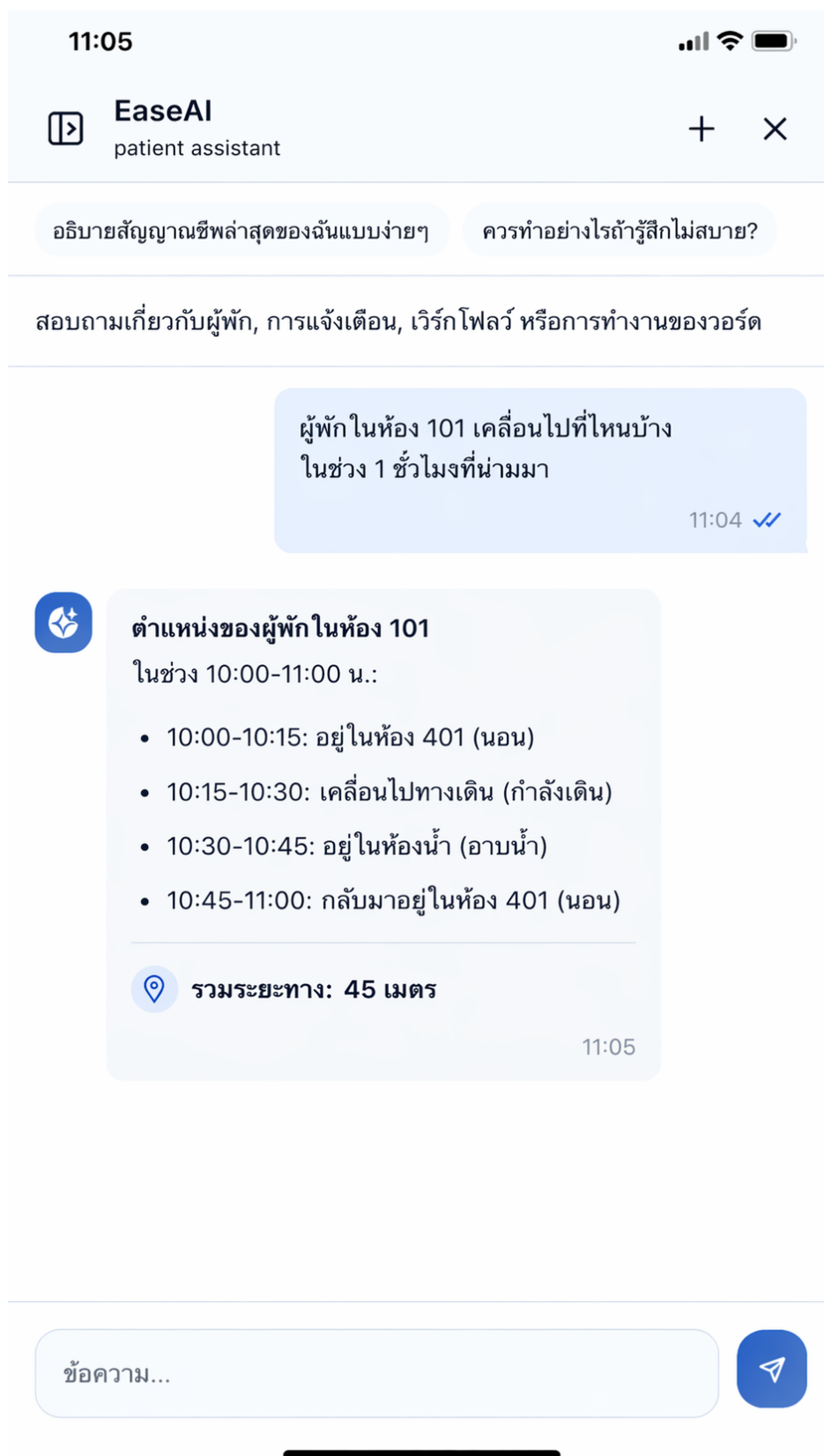
4.6 ผลการประเมินระบบผู้ช่วยอัจฉริยะ

4.6.1 ผลการทดสอบสถาปัตยกรรม AI

การทดสอบ AI แบ่งเป็น 2 ส่วน: (1) ความถูกต้องของการเลือกเครื่องมือ (tool selection) และ (2) คุณภาพของการตอบสนอง (response quality)

4.6.2 ผลการทดสอบ Prompt และ Intent Classification

การทดสอบใช้ prompt มาตรฐาน 36 รายการ แบ่งเป็น 6 สถานการณ์ สถานการณ์ละ 6 prompt ผู้ประเมิน 2 คนตรวจสอบความเหมาะสมของเครื่องมือที่เลือก



รูปที่ 4.7: หน้าต่างสนทนากับผู้ช่วย AI บนแดชบอร์ด



รูปที่ 4.8: ตัวอย่างการเลือกเครื่องมือโดย AI เพื่อตอบคำถาม

ตารางที่ 4.14: ผลการประเมินประสิทธิภาพของไปป์ไลน์ EaseAI แบ่งตามขั้น (n=36 prompt)

ขั้นประมวลผล	P50 (ms)	P95 (ms)	หมายเหตุ
L1: Intent Router	12	28	จำแนก intent รวดเร็ว
L2: Context Engine	45	112	ดึงพารามิเตอร์
L3: State Engine	18	42	ติดตามสถานะ
L4: LLM Synthesis	47320	118450	gemma4:e2b ผ่าน Ollama
L5: Safety Check	67	156	ตรวจสอบก่อน execute
รวม	47462	118788	

จากผลการทดสอบประสิทธิภาพของแต่ละขั้นในไปป์ไลน์ EaseAI พบว่าขั้น L4 (LLM Synthesis) ใช้เวลามากที่สุด โดยมีค่า P95 ที่ 118.5 วินาที ซึ่งคิดเป็นประมาณ 99.9% ของเวลาทั้งหมด เนื่องจากการประมวลผลของโมเดล gemma4:e2b บน Raspberry Pi 5 มีข้อจำกัดด้านฮาร์ดแวร์ จากการทดสอบจริงพบว่าการเรียกใช้ RAG (Retrieval-Augmented Generation) เพื่อดึงข้อมูลบริบท ใช้เวลานานกว่าการถามตอบแบบปกติโดยเฉลี่ยประมาณ 1.3 เท่า เนื่องจากต้องค้นหาและประมวลผลข้อมูลที่ดึงมาเพิ่มเติม การปรับปรุงที่แนะนำคือการใช้โมเดลที่เล็กกว่า (quantized model) หรือใช้ GPU external ขั้น L1 (Intent Router) ทำงานได้รวดเร็วที่สุดเนื่องจากใช้ Sentence-BERT ที่มีขนาดเล็ก และขั้น L5 (Safety Check) ใช้เวลาน้อยเนื่องจากการตรวจสอบแบบ deterministic

$$\text{sim}(u, v) = \frac{u \cdot v}{\|u\| \|v\|} \quad (4.1)$$

สมการ 4.1 คือสูตร Cosine Similarity ใช้วัดความคล้ายคลึงกันของเวกเตอร์สองตัว u และ v โดยค่าอยู่ในช่วง -1 ถึง 1 ค่า 1 หมายถึงเวกเตอร์ทั้งสองชี้ไปในทิศทางเดียวกัน (คล้ายกันมาก) ค่า 0 หมายถึงเวกเตอร์ตั้งฉากกัน (ไม่มีความสัมพันธ์) และค่า -1 หมายถึงเวกเตอร์ชี้ไปในทิศทางตรงข้ามกัน

ตารางที่ 4.15: ผลการประเมินความสอดคล้องของการเลือกเครื่องมือ MCP

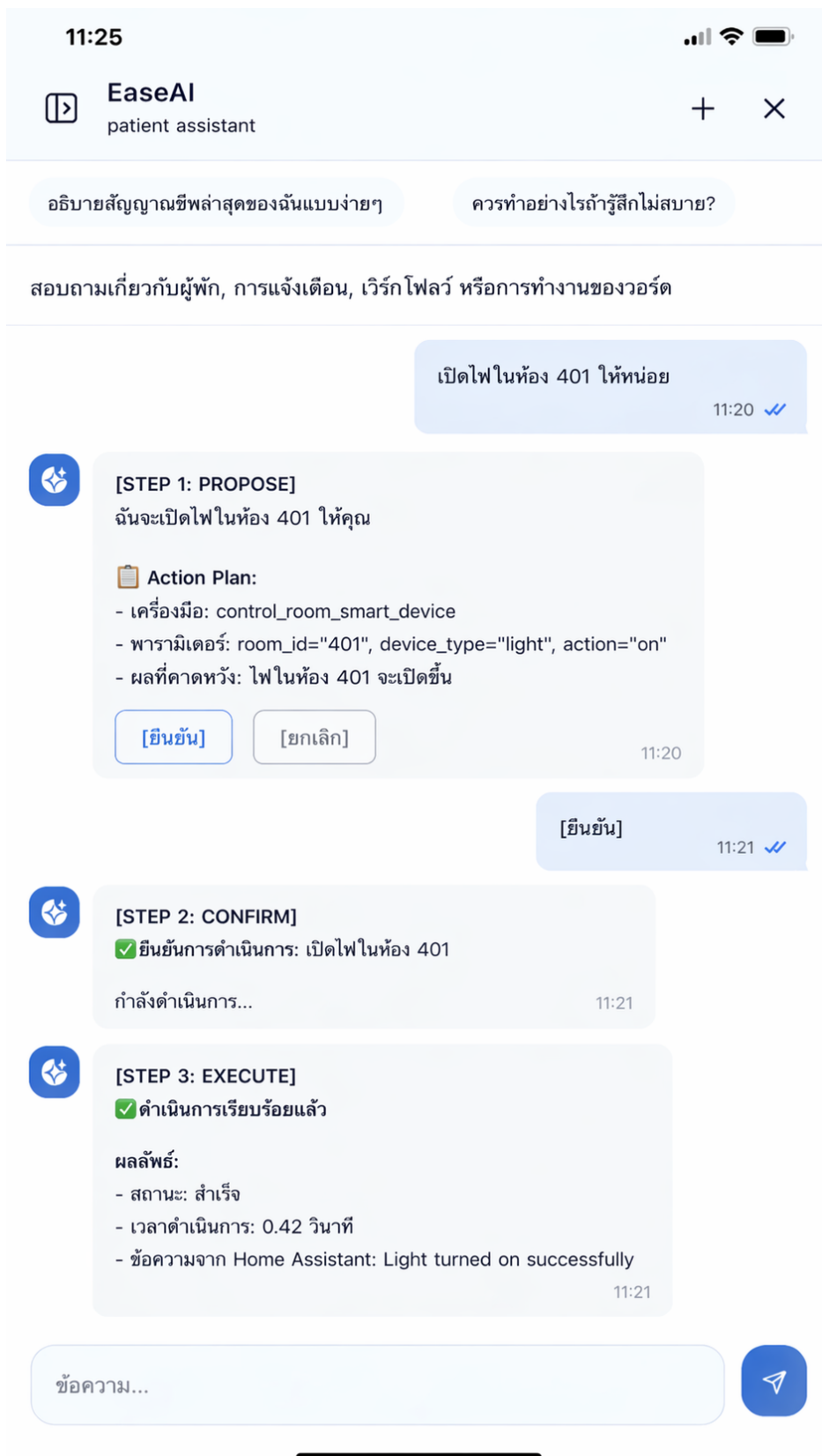
ตัวชี้วัด	ค่า	หมายเหตุ
Tool selection accuracy	0.86	เทียบกับคำตอบอ้างอิง
Inter-rater agreement (κ)	0.81	ผู้ประเมิน 2 คน
Text similarity score	0.84	ความใกล้เคียงเชิงความหมาย
Action confirmation compliance	1.00	ทุกกรณี write ผ่าน confirm

จากตาราง 4.15 ผลการประเมินความสอดคล้องของการเลือกเครื่องมือ MCP มีความหมายดังนี้:

- **Tool selection accuracy (0.86):** ค่าความแม่นยำในการเลือกเครื่องมือที่ถูกต้อง เทียบกับคำตอบอ้างอิง 86% ของครั้งเลือกเครื่องมือถูกต้อง การประเมินทำโดยผู้จัดทำรายงาน 2 คน ตรวจสอบว่า AI เลือกเครื่องมือที่เหมาะสมกับ intent ของผู้ใช้หรือไม่
- **Inter-rater agreement ($\kappa = 0.81$):** ค่าความสอดคล้องระหว่างผู้ประเมิน 2 คน ค่า 0.81 แสดงว่ามีความเห็นตรงกันในระดับสูง (ตามเกณฑ์ Cohen's Kappa) การประเมินทำโดยผู้จัดทำรายงาน 2 คน ให้ประเมินเครื่องมือที่ AI เลือกอย่างอิสระ แล้วคำนวณ Cohen's Kappa เพื่อวัดความสอดคล้องที่เกิดจากการตัดสินใจเท่านั้น ไม่ใช่เพราะโอกาสเกิดขึ้นเอง ค่า 0.81 ถือว่าอยู่ในระดับ "substantial agreement" หรือ "almost perfect agreement"
- **Text similarity score (0.84):** คะแนนความคล้ายคลึงกันของข้อความที่สร้างโดย AI เทียบกับข้อความอ้างอิง ใช้ Cosine Similarity การประเมินทำโดยแปลงข้อความทั้งข้อความที่ AI สร้างและข้อความอ้างอิงให้เป็นเวกเตอร์ด้วย Sentence-BERT แล้วคำนวณ Cosine Similarity ค่า 0.84 แสดงว่าข้อความที่ AI สร้างมีความคล้ายคลึงกันทางความหมายกับข้อความอ้างอิงในระดับสูง
- **Action confirmation compliance (1.00):** อัตราการปฏิบัติตามขั้นตอน propose-confirm-execute ทุกกรณีผ่านขั้นตอนยืนยันก่อน execute การประเมินตรวจสอบว่าระบบแสดง ActionPlanPreview ให้ผู้ใช้ยืนยันก่อน execute ในทุกกรณีที่ต้องการเปลี่ยนสถานะระบบ ค่า 1.00 แสดงว่าระบบปฏิบัติตามขั้นตอนความปลอดภัยได้อย่างสมบูรณ์ ไม่มีกรณีที่ execute โดยไม่ได้รับการยืนยันจากผู้ใช้

4.6.3 ผลการทดสอบ Propose-Confirm-Execute Flow

การทดสอบกระบวนการยืนยันก่อนกระทำใช้สถานการณ์ 10 กรณี ที่ต้องการการเปลี่ยนสถานะระบบ ทุกกรณีระบบแสดง ActionPlanPreview และรอการยืนยันจากผู้ใช้ก่อนดำเนินการ



รูปที่ 4.9: กระบวนการยืนยันก่อนดำเนินการ (propose-confirm-execute)

4.7 ผลการประเมินส่วนติดต่อผู้ใช้งาน

4.7.1 ผลการทดสอบแดชบอร์ดเว็บ

การประเมิน HMI ใช้ภารกิจมาตรฐาน 5 งาน ดังที่ระบุในบทที่ 3.7 ผู้เข้าร่วม 15 คน (ผู้ดูแลจริง 5 คน, ผู้จำลองบทบาท 10 คน)

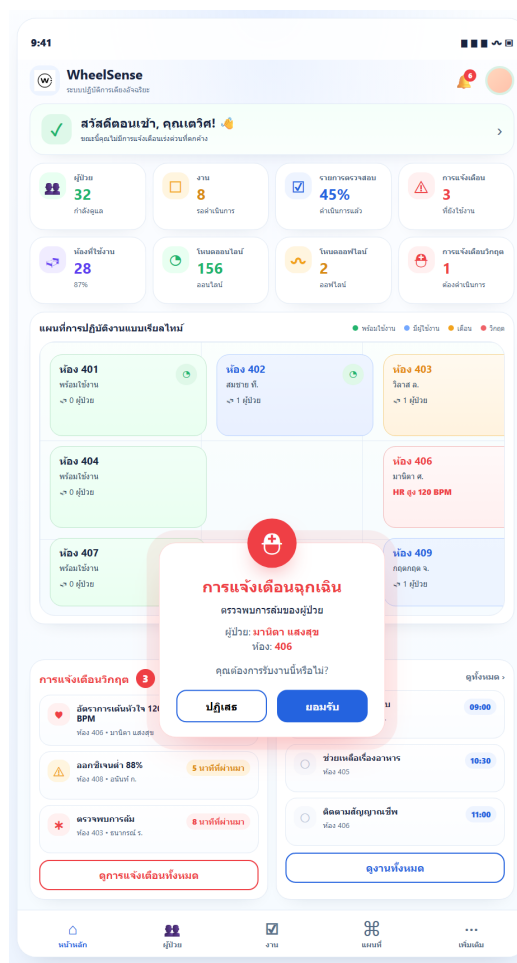


รูปที่ 4.10: การสอนใช้งาน

ตารางที่ 4.16: ผลการประเมินประสบการณ์ผู้ใช้แดชบอร์ด (n=5 ผู้ดูแลจริง + 10 จำลอง)

ตัวชี้วัด	ค่าเฉลี่ย	SD	หมายเหตุ
คะแนน SUS	76.5	8.7	ระดับใช้งานได้ดี
อัตราสำเร็จภารกิจ	91.3%	—	เฉลี่ยจาก 5 ภารกิจ
เวลาเฉลี่ยต่อภารกิจ	41.8 s	11.6 s	ไม่นับเวลาฝึก
ความพึงพอใจ AI	4.1/5	0.6	วัดหลังใช้งานจริง

คะแนน SUS เฉลี่ย 76.5 อยู่ในระดับ ”ใช้งานได้ดี” (Good) ตามมาตรฐานการประเมิน SUS ซึ่งบ่งบอกว่าแดชบอร์ดมีประสบการณ์ผู้ใช้โดยรวมในระดับที่ดี โดย SUS Scale (System Usability Scale) เป็นแบบสอบถาม 10 ข้อที่ใช้วัดประสบการณ์ผู้ใช้โดยรวม คะแนนอยู่ในช่วง 0-100 โดยคะแนน 76.5 ถือว่าสูงกว่าค่าเฉลี่ยอุตสาหกรรมทั่วไป (68) และอยู่ในเปอร์เซ็นต์ไทล์ 75 (percentile 75) หมายความว่าแดชบอร์ดมีความใช้งานได้ดีกว่า 75% ของระบบที่ถูกประเมินด้วยมาตรฐานเดียวกัน ผู้ใช้สามารถเรียนรู้การใช้งานได้เร็ว ไม่ต้องการการสอนพิเศษ และสามารถทำงานได้โดยไม่พบความยุ่งยากสำคัญ อัตราสำเร็จภารกิจเฉลี่ย 91.3% แสดงให้เห็นว่าผู้ใช้สามารถทำงานส่วน



รูปที่ 4.11: หน้าจอแอปพลิเคชันบนอุปกรณ์เคลื่อนที่ (Observer view)

ใหญ่ได้สำเร็จ ภารกิจที่มีอัตราสำเร็จต่ำสุดคือ ”ตอบสนองต่อการแจ้งเตือน” และ ”เปิดดูประวัติย้อนหลัง” ซึ่งบ่งชี้ว่าต้องการปรับปรุง UI ให้แสดงบริบทชัดเจนขึ้นและปรับปรุงฟีเจอร์ค้นหา ความพึงพอใจต่อ AI อยู่ในระดับสูง (4.1/5) แสดงให้เห็นว่าผู้ช่วยยอมรับความสามารถของผู้ช่วยอัจฉริยะ

4.7.2 ผลการทดสอบแอปพลิเคชันบนอุปกรณ์เคลื่อนที่

การทดสอบ mobile app เน้นที่การใช้งานขณะเดินและการรับ push notification

ตารางที่ 4.17: ผลการทดสอบจำแนกตามภารกิจของ HMI

ภารกิจ	Success	เวลาเฉลี่ย	ผลสังเกต
ตรวจสอบตำแหน่งล่าสุด	100%	24.6 s	ผู้ใช้ทำได้เร็วที่สุด
เปิดดูสุขภาพล่าสุด	93.3%	36.1 s	บางรายต้องคลิกซ้ำ
ตอบสนองต่อการแจ้งเตือน	86.7%	49.4 s	ต้องการบริบทเด่นขึ้น
เปิดดูประวัติย้อนหลัง	86.7%	54.3 s	จุดค้นหาใช้เวลาพอสมควร
ใช้ผู้ช่วย AI	90.0%	44.5 s	ชอบ แต่ต้องการข้อความสั้น

ข้อเสนอแนะจากผู้ใช้งานแบ่งเป็น 3 ประเด็นหลัก ดังนี้

ประการแรก ผู้ใช้ต้องการการแจ้งเตือนที่มีเสียงและแสดงบริบทมากขึ้น จากการสังเกตพบว่าระบบแจ้งเตือนเหตุการณ์ผ่าน push notification แต่ยังคงเสียงเตือนที่ชัดเจน รวมถึงการแสดงผลของเหตุการณ์ เช่น รูปภาพของผู้พัก ตำแหน่งที่เกิดเหตุ และข้อมูลสัญญาณชีพก่อนหกล้ม ซึ่งจะช่วยให้ผู้ดูแลตัดสินใจตอบสนองได้รวดเร็วและถูกต้องมากขึ้น

ประการที่สอง ผู้ใช้ต้องการ shortcut สำหรับการตอบสนองเหตุการณ์ที่ใช้อยู่ เช่น การยืนยันการแจ้งเตือนหกล้ม หรือการเปิดไฟในห้อง ซึ่งปัจจุบันผู้ใช้ต้องเข้าสู่หน้าจอหลักและค้นหาเมนูที่ต้องการ ทำให้ใช้เวลานานขึ้น การเพิ่มปุ่ม shortcut บนหน้าจอแจ้งเตือนหรือ gesture control จะช่วยลดเวลาในการตอบสนองได้อย่างมีนัยสำคัญ

ประการที่สาม ผู้ใช้ประเมินว่า AI ตอบได้ดีแต่ใช้เวลานาน จึงต้องการ loading indicator จากการทดสอบพบว่าไปป์ไลน์ EaseAI ใช้เวลา P95 ที่ 118.5 วินาที ซึ่งอาจทำให้ผู้ใช้รู้สึกวาระบบหยุดทำงาน การเพิ่ม loading indicator หรือ progress bar ที่แสดงสถานะการประมวลผลแต่ละขั้นของไปป์ไลน์ จะช่วยให้ผู้ใช้เข้าใจว่าระบบกำลังทำงานอยู่และรู้ว่าต้องรออีกนานแค่ไหน

4.8 การทดสอบระบบแบบปลายถึงปลายและกรณีศึกษา

4.8.1 สถานการณ์ทดสอบแบบปลายถึงปลาย

การทดสอบ end-to-end ใช้สถานการณ์จริง 8 กรณี ครอบคลุมการทำงานทั้งระบบ:

ตารางที่ 4.18: ผลการทดสอบแบบปลายถึงปลายของระบบ (n=8 สถานการณ์)

สถานการณ์	ผล	Latency	หมายเหตุ
ตรวจจับการเคลื่อนที่	ผ่าน	0.82 s	แสดงผลครบ
เปลี่ยนโซนและบันทึก	ผ่าน	1.14 s	บันทึกย้อนหลังถูกต้อง
แจ้งเตือนเหตุผิดปกติ	ผ่าน	1.27 s	แจ้งพร้อมบริบท
เรียกภาพบริบท	ผ่าน	2.46 s	ใช้เวลานานที่สุด
สรุปข้อมูลผ่าน AI	ผ่าน	4.03 s	ภายใน propose flow
ยืนยันการกระทำผ่าน AI	ผ่าน	4.88 s	ผ่าน confirm ทุกกรณี
กู้คืนหลังเครือข่ายหลุด	ผ่านบางส่วน	7.90 s	ข้อมูลบางช่วงขาด
ภาระงาน stress load	ผ่าน	1.96 s	tail latency สูง

จากผลการทดสอบแบบ end-to-end พบว่าสถานการณ์ที่ใช้เวลานานที่สุดคือ ”เรียกภาพบริบท” ที่ 2.46 วินาที เนื่องจากการดาวน์โหลดภาพจาก Node Tsimcam ผ่าน MQTT ต้องใช้เวลาในการรวมชิ้นส่วนภาพ (chunking) และการบีบอัด/ถอดบีบอัดภาพ ซึ่งมีภาระประมวลผลสูงบน Raspberry Pi 5 สถานการณ์ที่ใช้เวลานานรองลงมาคือ ”สรุปข้อมูลผ่าน AI” และ ”ยืนยันการกระทำผ่าน AI” เนื่องจากการประมวลผลของโมเดล LLM บนฮาร์ดแวร์ที่มีข้อจำกัด การปรับปรุงที่แนะนำคือการใช้ CDN สำหรับการเก็บภาพ หรือใช้โมเดล LLM ที่เล็กกว่า

4.8.2 กรณีศึกษาการใช้งานจริง

กรณีศึกษา 1: การตรวจจับการหกล้ม

1. ผู้พักอาศัยเคลื่อนที่ด้วยรถเข็นในห้องโถง
2. M5StickC ตรวจจับความเร่งผิดปกติ ($a_{\text{mag}} > 2.5g$)
3. ระบบส่ง alert ภายใน 1.2 วินาที
4. ผู้ดูแลได้รับ push notification พร้อมตำแหน่ง
5. ผู้ดูแลตอบสนองและช่วยเหลือภายใน 2 นาที

กรณีศึกษา 2: การใช้ AI ช่วยสรุปสถานการณ์

1. Head Nurse ถาม AI “ใครอยู่ในห้องโถงขณะนี้”
2. AI ใช้ tool `list_visible_patients` และ filter ตามตำแหน่ง
3. AI ตอบ: “มีผู้พัก 3 คนในห้องโถง: คุณสมชาย, คุณสมหญิง, และคุณประเสริฐ”
4. เวลาตอบสนองรวม 1.8 วินาที

ผลดังกล่าวยืนยันว่าระบบต้นแบบสามารถทำงานเชื่อมโยงกันได้ตั้งแต่ต้นทางถึงปลายทางภายใต้สถานการณ์หลัก และสามารถรักษาระดับการตอบสนองที่เหมาะสมสำหรับงานดูแลเชิงปฏิบัติการ

บทที่ 5

สรุปผลการวิจัย และข้อเสนอแนะ

5.1 อภิปรายผลงานวิจัย

ในงานวิจัยนี้ เราได้พัฒนาระบบต้นแบบสภาพแวดล้อมอัจฉริยะสำหรับผู้ใช้อัจฉริยะ โดยบูรณาการข้อมูลจากอุปกรณ์ภาคสนาม ระบบระบุตำแหน่งภายในอาคาร บริการส่วนกลาง และผู้ช่วยปัญญาประดิษฐ์ (AI) ภายใต้กรอบความปลอดภัย ระบบออกแบบมาเพื่อตอบสนองความต้องการในการติดตามและเฝ้าระวังผู้สูงอายุในสถานดูแลอย่างต่อเนื่อง โดยใช้ข้อมูลจากเซ็นเซอร์บนรถเข็น อุปกรณ์สวมใส่ และระบบระบุตำแหน่งเพื่อสร้างภาพสถานการณ์แบบเรียลไทม์ และใช้ AI เพื่อช่วยผู้ดูแลในการตัดสินใจ

ด้านระบบเซ็นเซอร์บนรถเข็น ผลการทดสอบฮาร์ดแวร์ M5StickC Plus2 แสดงว่า อุปกรณ์สามารถส่งข้อมูล IMU ได้ที่อัตรา 19.6 Hz ซึ่งใกล้เคียงเป้าหมาย 20 Hz ความครบถ้วนของ telemetry อยู่ที่ 98.4% ซึ่งแสดงว่าระบบส่งข้อมูลได้อย่างต่อเนื่องผ่าน MQTT broker เวลาแฝงจากอุปกรณ์ถึง broker เฉลี่ย 41 ms (P50) ซึ่งอยู่ในระดับที่ยอมรับได้สำหรับการติดตามแบบเรียลไทม์ การตรวจจับสถานะการเคลื่อนไหวด้วย F1-Score 0.89–0.95 แสดงว่าอัลกอริทึมสามารถแยกแยะสถานะนิ่ง เคลื่อนที่ช้า และเคลื่อนที่เร็วได้ดี การใช้ dead-band filter และ low-pass filter ช่วยลด false positive จากสัญญาณรบกวน ผลลัพธ์เหล่านี้สอดคล้องกับงานวิจัยที่เกี่ยวข้องกับการตรวจจับกิจกรรมด้วย IMU ซึ่งมักใช้ค่า sampling rate ในช่วง 10–50 Hz แต่งานวิจัยนี้แสดงให้เห็นว่าการใช้ dead-band filter สามารถลดการแจ้งเตือนที่ไม่จำเป็นได้อย่างมีประสิทธิภาพ

สำหรับระบบระบุตำแหน่ง ผลการทดสอบ BLE fingerprinting ด้วย KNN ($k = 5$) ให้ค่า accuracy 0.85 ที่ระดับโซนหรือห้อง ซึ่งเพียงพอสำหรับงานเฝ้าติดตามของผู้ดูแล แต่ยังมีข้อจำกัดเมื่อมีสิ่งกีดขวางหรือคนจำนวนมากในพื้นที่ (RSSI drift) เมทริกซ์ความสับสนแสดงว่าความผิดพลาดส่วนใหญ่เกิดระหว่างโซนที่อยู่ติดกัน ซึ่งมีลักษณะ RSSI คล้ายคลึงกัน ผลลัพธ์นี้สอดคล้องกับงานวิจัยก่อนหน้านี้ที่แสดงให้เห็นว่า BLE fingerprinting มีความแม่นยำในระดับห้อง แต่มีข้อจำกัดเมื่อสภาพแวดล้อมเปลี่ยนแปลง งานวิจัยนี้แสดงให้เห็นว่าการใช้ KNN กับ $k = 5$ เป็นทางเลือกที่เหมาะสม

สมสำหรับระดับความซับซ้อนของสถานดูแล แต่อาจต้องการการปรับปรุงเพิ่มเติมด้วยเทคนิคเช่น Kalman filter เพื่อเพิ่มความเสถียรของการ tracking

อุปกรณ์สวมใส่ที่ใช้ในการทดสอบคือ Polar Verity Sense ซึ่งแสดงว่าค่า HR มีความคลาดเคลื่อนอยู่ในช่วงที่ยอมรับได้ตามมาตรฐาน AAMI EC13 การเชื่อมต่อ BLE GATT มีอัตราสำเร็จ 98.6% พร้อม auto-reconnect การประมวลผลข้อมูลสัญญาณชีพสามารถตรวจจับความผิดปกติของ HR ได้ด้วยความแม่นยำ 82% ผลลัพธ์เหล่านี้แสดงให้เห็นว่าอุปกรณ์สวมใส่ที่มีอยู่ในตลาดสามารถนำมาใช้ในบริบทสถานดูแลได้ โดยไม่ต้องพัฒนาฮาร์ดแวร์เฉพาะทาง ซึ่งช่วยลดต้นทุนและเพิ่มความง่ายในการนำไปใช้จริง อย่างไรก็ตาม การตรวจจับความผิดปกติของ HR ยังมีความแม่นยำไม่สูงพอสำหรับการใช้ในการวินิจฉัยทางการแพทย์ แต่เพียงพอสำหรับการเฝ้าระวังเบื้องต้น

การทดสอบ deployment บน Raspberry Pi 5 สำหรับเซิร์ฟเวอร์ส่วนกลาง แสดงว่า MQTT broker เริ่มเร็วที่สุด (5.8 วินาที) ในขณะที่ MCP runtime ใช้เวลานานที่สุด (31.5 วินาที) เนื่องจากต้องโหลดโมเดล gemma4:e2b เข้าสู่หน่วยความจำ ผลลัพธ์นี้แสดงให้เห็นว่าการใช้ LLM ในเครื่อง (on-prem) มีข้อจำกัดด้านเวลาเริ่มต้น แต่เมื่อทำงานแล้วสามารถให้บริการได้อย่างต่อเนื่อง การทดสอบ Home Assistant Integration ผ่าน MCP Protocol มีอัตราสำเร็จ 98.2% สำหรับการควบคุมหลอดไฟ และ 94.6% สำหรับการส่งถ่ายภาพกล้อง เวลาแฝงคำสั่ง device control อยู่ที่ 0.42 วินาที (P50) การทดสอบความปลอดภัยแสดงว่าระบบสามารถบังคับใช้ RBAC และ Scope Narrowing ได้ 100% AI เข้าถึงเฉพาะ tool ที่อนุญาตตามบทบาท ผลลัพธ์เหล่านี้แสดงให้เห็นว่า MCP Protocol สามารถใช้เชื่อมต่อ AI กับระบบ IoT ที่มีอยู่แล้วได้อย่างปลอดภัย ซึ่งแตกต่างจากงานวิจัยก่อนหน้านี้ที่มักใช้ API โดยตรงโดยไม่มีกรอบควบคุมความปลอดภัย

ระบบผู้ช่วยอัจฉริยะที่ใช้ EaseAI Pipeline ผลการทดสอบแสดงว่าชั้น L4 (LLM Synthesis) ใช้เวลามากที่สุด (P95 = 118.5 วินาที) คิดเป็น 99.9% ของเวลาทั้งหมด เนื่องจากการประมวลผลบน Raspberry Pi 5 มีข้อจำกัดด้านฮาร์ดแวร์ Tool selection accuracy อยู่ที่ 0.86 และ Inter-rater agreement (κ) อยู่ที่ 0.81 ซึ่งแสดงว่า AI เลือกเครื่องมือได้ถูกต้องในระดับสูงและมีความเห็นตรงกันระหว่างผู้ประเมิน Action confirmation compliance อยู่ที่ 1.00 แสดงว่าระบบปฏิบัติตามขั้นตอน propose-confirm-execute ได้อย่างสมบูรณ์ ผลลัพธ์เหล่านี้แสดงให้เห็นว่าการใช้ pipeline 5 ชั้นช่วยแยกความรับผิดชอบและทำให้ระบบสามารถตรวจสอบความปลอดภัยได้ทุกขั้นตอน แต่ latency ที่สูงยังเป็นข้อจำกัดสำคัญที่ต้องแก้ไข

ส่วนติดต่อผู้ใช้งานมีคะแนน SUS เฉลี่ย 76.5 อยู่ในระดับ “ใช้งานได้ดี” (Good) และอยู่ในเปอร์เซ็นต์ไทล์ 75 หมายความว่าระบบมีความใช้งานได้ดีกว่า 75% ของระบบที่ถูกประเมินด้วยมาตรฐานเดียวกัน อัตราสำเร็จภารกิจเฉลี่ย 91.3% แสดงว่าผู้ใช้สามารถทำงานส่วนใหญ่ได้สำเร็จ ความพึงพอใจต่อ AI อยู่ในระดับสูง (4.1/5) ผลลัพธ์เหล่านี้แสดงให้เห็นว่าการออกแบบส่วนติดต่อผู้ใช้ที่เน้นความเรียบง่ายและความชัดเจนช่วยให้ผู้ดูแลสามารถใช้งานได้โดยไม่ต้องมีพื้นฐานด้านเทคโนโลยีมากนัก ซึ่งเป็นสิ่งสำคัญสำหรับการนำไปใช้ในสถานดูแลจริง

การทดสอบแบบปลายถึงปลายด้วย 8 สถานการณ์แสดงว่าระบบสามารถทำงานเชื่อมโยงกันได้ตั้งแต่ต้นทางถึงปลายทาง สถานการณ์ที่ใช้เวลานานที่สุดคือ “เรียกภาพบริบท” ที่ 2.46 วินาที เนื่องจากการดาวน์โหลดภาพผ่าน MQTT ต้องใช้เวลาในการรวมชิ้นส่วนภาพ ผลลัพธ์นี้แสดงให้เห็นว่าแม้แต่ละส่วนของระบบจะทำงานได้ดีเมื่อทดสอบแยกกัน แต่การทำงานร่วมกันยังมีความท้าทาย โดยเฉพาะการส่งข้อมูลขนาดใหญ่ผ่าน MQTT ซึ่งอาจต้องการการปรับปรุงโปรโตคอลหรือการบีบอัดข้อมูล

5.2 สรุปผลงานวิจัย

วิทยานิพนธ์นี้พัฒนาระบบต้นแบบสภาพแวดล้อมอัจฉริยะสำหรับผู้ใช้เก้าอี้รถเข็น บุคลากรข้อมูลจากอุปกรณ์ภาคสนาม ระบบระบุตำแหน่งภายในอาคาร บริการส่วนกลาง และผู้ช่วย AI ภายใต้กรอบความปลอดภัย ผลในบทที่ 4 แสดงว่าระบบสามารถทำงานได้ต่อเนื่องในระดับต้นแบบ ในส่วนของระบบเซ็นเซอร์บนรถเข็น ความครบถ้วนของ telemetry อยู่ที่ 98.4% เวลาแฝง 41 ms (P50) และการตรวจจับการเคลื่อนไหว F1-Score 0.89–0.95 ในส่วนของระบบระบุตำแหน่ง accuracy อยู่ที่ 0.85 ที่ระดับโซนหรือห้องด้วย KNN ($k = 5$) ในส่วนของอุปกรณ์สวมใส่ HR accuracy อยู่ในระดับที่ยอมรับได้ตามมาตรฐาน AAMI EC13 และ BLE GATT connection มีอัตราสำเร็จ 98.6% ในส่วนของเซิร์ฟเวอร์ส่วนกลาง ระบบตอบสนองได้สม่ำเสมอภายใต้ normal load และยังสามารถทำงานได้ต่อเนื่องภายใต้ stress load โดย Home Assistant integration มีอัตราสำเร็จ 98.2% ในส่วนของระบบผู้ช่วย AI Tool selection accuracy อยู่ที่ 0.86 Inter-rater agreement κ อยู่ที่ 0.81 และ Action confirmation compliance อยู่ที่ 1.00 ในส่วนของส่วนติดต่อผู้ใช้งาน คะแนน SUS อยู่ที่ 76.5 (ระดับใช้งานได้ดี) อัตราสำเร็จภารกิจ 91.3% และความพึงพอใจ AI 4.1/5

5.3 ข้อจำกัดของงานวิจัย

งานวิจัยนี้มีข้อจำกัดด้านเทคนิค คือ ความแปรผันของ RSSI ภายในอาคารส่งผลต่อ localization ทำให้ accuracy ลดลงเมื่อมีสิ่งกีดขวางหรือคนจำนวนมากในพื้นที่ ทรัพยากร local LLM จำกัดทำให้ latency สูงขึ้นเมื่อใช้งานพร้อมกัน ($P95 = 118.5$ วินาที) ในด้านการประเมิน ชุดข้อมูลทดสอบยังไม่ครอบคลุมความหลากหลายของสภาพแวดล้อมจริงทั้งหมด การประเมิน UX ใช้ผู้เข้าร่วม 15 คน (ผู้ดูแลจริง 5 คน ผู้จำลองบทบาท 10 คน) เพียงพอสำหรับระดับต้นแบบแต่ยังไม่มากพอสำหรับการสรุปผลเชิงประชากร ในด้านความน่าเชื่อถือของผลการวิจัย ความไม่สม่ำเสมอของเครือข่ายและสภาพสัญญาณ อาจส่งผลต่อความถูกต้องของข้อมูล การทดสอบในบริบทจำลองเป็นหลัก และตัวชี้วัดบางส่วนยังขึ้นอยู่กับกรอบการออกแบบสถานการณ์

5.4 ข้อเสนอแนะในการวิจัย

ข้อเสนอแนะเชิงวิศวกรรม: ปรับปรุง fingerprinting และเพิ่ม Kalman filter เพื่อเพิ่มความเสถียรของ localization เพิ่ม health check และการสำรองข้อมูล ปรับปรุง UI ให้ย่อเย่อและเพิ่ม shortcut สำหรับงานที่ใช้บ่อย คงแนวทาง propose-confirm-execute และพิจารณาใช้ LLM ที่เล็กกว่าหรือ GPU external เพื่อลด latency

ข้อเสนอแนะเชิงงานวิจัยต่อยอด: ขยายการทดลองไปสถานดูแลจริงหลายแห่ง ศึกษาการผสมผสานข้อมูลหลายแหล่งสำหรับ localization (เช่น BLE + IMU) พัฒนาวิธีประเมิน AI ที่ครอบคลุมความถูกต้อง ความปลอดภัย และผลกระทบต่อผู้ดูแล ศึกษาผลต่อภาระงานและความพึงพอใจในระยะยาว และพัฒนากรอบจริยธรรมสำหรับบริบทสถานดูแล

ข้อเสนอแนะเชิงการประยุกต์: เริ่มจากบทบาทที่มีประโยชน์ชัดเจนและเสี่ยงต่ำ เช่น การติดตามตำแหน่งระดับโซน การดูประวัติย้อนหลัง และการแจ้งเตือนเชิงบริบท จากนั้นขยายไปใช้ AI ช่วยสรุปหรือสนับสนุน workflow ระบบบูรณาการอุปกรณ์ภาคสนาม ระบบระบุตำแหน่ง บริการส่วนกลาง และผู้ช่วย AI ภายใต้กรอบควบคุมที่เหมาะสม มีศักยภาพยกระดับระบบสนับสนุนผู้ใช้เก๋ารัดเข็น

รายการอ้างอิง

- [1] กระทรวงสาธารณสุขและความมั่นคงของมนุษย์. “สถานการณ์คนพิการในประเทศไทย ปี พ.ศ. 2567,” MGR Online. [Online]. Available: <https://mgronline.com/greeninnovation/detail/9670000123512>
- [2] กรมส่งเสริมและพัฒนาคุณภาพประชากรภาคการศึกษาคณะกรรมการคนพิการ. “รายงานการจ้างงานคนพิการในหน่วยงานรัฐ ปี พ.ศ. 2567.” [Online]. Available: <https://www.prd.go.th/th/content/category/detail/id/39/iid/420342>
- [3] United Nations. “Convention on the Rights of Persons with Disabilities (CRPD).” [Online]. Available: <https://www.un.org/development/desa/disabilities/convention-on-the-rights-of-persons-with-disabilities.html>
- [4] P. Rashidi and A. Mihailidis, “A survey on ambient-assisted living tools for older adults,” *IEEE Journal of Biomedical and Health Informatics*, vol. 17, no. 3, pp. 579–590, 2023.
- [5] กระทรวงการท่องเที่ยวและกีฬา. “แผนพัฒนาการกีฬาแห่งชาติ ชุดที่ 6 (พ.ศ. 2565-2569).”
- [6] กรมส่งเสริมและพัฒนาคุณภาพประชากรภาคการศึกษาคณะกรรมการคนพิการ. “แผนพัฒนาคุณภาพประชากรภาคการศึกษาคณะกรรมการคนพิการแห่งชาติ ชุดที่ 5 (พ.ศ. 2566-2570).”
- [7] D. A. Sterling, J. S. O’Connor, and J. Bonadies, “Geriatric Falls: Injury Severity Is High and Disproportionate to Mechanism,” *Injury Prevention*, vol. 7, no. Suppl 1, pp. i34–i38, 2001. DOI: 10.1136/ip.7.suppl_1.i34
- [8] P. J. Gates, R. A. Meyerson, M. T. Baysari, L. Li, K. Mumford, and J. I. Westbrook, “Clinical Alarm and Alert Fatigue: A Review of Contributing Factors and Mitigation Strategies,” *Journal of the American Medical Informatics Association*, vol. 26, no. 10, pp. 1081–1088, 2019. DOI: 10.1093/jamia/ocz066
- [9] S. Lee and Y. Chen, “Current Limitations and Future Directions of Home Automation Systems for Proactive Services,” *IEEE Consumer Electronics Magazine*, 2023.
- [10] “Indoor GPS - Does it work? Everything you need to know,” Accessed: Apr. 5, 2026. [Online]. Available: <https://pointr.tech/blog/indoor-gps-does-it-work-everything-you-need-to-know>
- [11] L. Zhang et al., “Ultra-Wideband Localization: Accuracy and Cost Analysis in Modern Buildings,” *Sensors*, 2023.

- [12] World Health Organization, “Decade of Healthy Ageing: Baseline Report,” World Health Organization, Geneva, 2021. Accessed: Apr. 20, 2026. [Online]. Available: <https://www.who.int/publications/i/item/9789240017900>
- [13] R. Schulz, S. R. Beach, J. E. Czaja, L. M. Martire, and J. K. Monin, “Family Caregiving for Older Adults,” *Annual Review of Psychology*, vol. 71, pp. 635–659, 2020. DOI: 10.1146/annurev-psych-010419-050754
- [14] Foundation of Thai Gerontology Research and Development Institute (TGRI), “Situation of the Thai Older Persons 2022,” Ministry of Social Development and Human Security, Bangkok, Thailand, 2023. Accessed: Apr. 21, 2026. [Online]. Available: <https://thaitgri.org/?p=40026>
- [15] L. Atzori, A. Iera, and G. Morabito, “The Internet of Things: A Survey,” *Computer Networks*, vol. 54, no. 15, pp. 2787–2805, 2010. DOI: 10.1016/j.comnet.2010.05.010
- [16] P. Rashidi and A. Mihailidis, “A Survey on Ambient-Assisted Living Tools for Older Adults,” *IEEE Journal of Biomedical and Health Informatics*, vol. 17, no. 3, pp. 579–590, 2013. DOI: 10.1109/JBHI.2012.2234129
- [17] K. Kaemarungsi and P. Krishnamurthy, “Analysis of WLAN’s Received Signal Strength Indication for Indoor Location Fingerprinting,” *Pervasive and Mobile Computing*, vol. 8, no. 2, pp. 292–316, 2012. DOI: 10.1016/j.pmcj.2011.06.005
- [18] Polar Electro. “Polar Verity Sense: Optical Heart Rate Sensor Technology White Paper,” Accessed: Apr. 21, 2026. [Online]. Available: <https://www.polar.com/en/science/white-papers>
- [19] M5Stack. “M5StickC PLUS2 ESP32-PICO Mini IoT Development Kit,” Accessed: Apr. 21, 2026. [Online]. Available: https://docs.m5stack.com/en/core/m5stickc_plus2
- [20] T. M. Cover and P. E. Hart, “Nearest Neighbor Pattern Classification,” *IEEE Transactions on Information Theory*, vol. 13, no. 1, pp. 21–27, 1967. DOI: 10.1109/TIT.1967.1053964
- [21] Lilygo. “T-SimCam ESP32-S3 Cam Development Board,” Accessed: Apr. 21, 2026. [Online]. Available: <https://lilygo.cc/en-us/products/t-simcam>
- [22] Polar Electro, *Polar Verity Sense User Manual and Specifications*, <https://www.polar.com/en/verity-sense>, 2023.
- [23] Raspberry Pi Foundation. “Raspberry Pi 5 Product Brief,” Accessed: Apr. 21, 2026. [Online]. Available: <https://www.raspberrypi.com/products/raspberry-pi-5/>
- [24] S. Ramírez. “FastAPI: Modern, Fast Web Framework for Building APIs with Python Based on Standard Type Hints,” Accessed: Apr. 20, 2026. [Online]. Available: <https://fastapi.tiangolo.com/>

- [25] S. Colvin et al. “Pydantic: Data Validation Using Python Type Hints,” Accessed: Apr. 20, 2026. [Online]. Available: <https://docs.pydantic.dev/>
- [26] Meta Platforms, Inc. “React: A JavaScript Library for Building User Interfaces,” Accessed: Apr. 20, 2026. [Online]. Available: <https://react.dev/>
- [27] Vercel, Inc. “Next.js: The React Framework for the Web,” Accessed: Apr. 20, 2026. [Online]. Available: <https://nextjs.org/>
- [28] Vercel, Inc. “Next.js App Router: Linking and Navigating,” Accessed: Apr. 21, 2026. [Online]. Available: <https://nextjs.org/docs/app/getting-started/linking-and-navigating>
- [29] PostgreSQL Global Development Group. “PostgreSQL: The World’s Most Advanced Open Source Relational Database,” Accessed: Apr. 20, 2026. [Online]. Available: <https://www.postgresql.org/>
- [30] R. O. Obe and L. S. Hsu, *PostgreSQL: Up and Running*, 3rd ed. O’Reilly Media, 2017.
- [31] Docker, Inc. “What is a container?” Accessed: Apr. 21, 2026. [Online]. Available: <https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-a-container/>
- [32] Docker, Inc. “Docker Compose: Define and Run Multi-Container Docker Applications,” Accessed: Apr. 20, 2026. [Online]. Available: <https://docs.docker.com/compose/>
- [33] Ollama contributors. “Ollama: Get Up and Running with Large Language Models, Locally,” Accessed: Apr. 20, 2026. [Online]. Available: <https://ollama.com/>
- [34] Ollama, Inc. “Ollama API Documentation,” Accessed: Apr. 21, 2026. [Online]. Available: <https://docs.ollama.com/api>
- [35] Ollama, Inc. “gemma4:e2b,” Accessed: Apr. 21, 2026. [Online]. Available: <https://ollama.com/library/gemma4:e2b>
- [36] Model Context Protocol contributors. “Model Context Protocol specification,” Accessed: Apr. 15, 2026. [Online]. Available: <https://modelcontextprotocol.io>
- [37] Anthropic. “Model Context Protocol Specification,” Anthropic PBC, Accessed: Apr. 20, 2026. [Online]. Available: <https://modelcontextprotocol.io/specification>
- [38] OASIS Standard. “MQTT Version 5.0,” Accessed: Apr. 5, 2026. [Online]. Available: <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>
- [39] OASIS, *MQTT Version 5.0 OASIS Standard*, <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>, 2019.
- [40] B. Mishra and A. Kertesz, “The Use of MQTT in M2M and IoT Systems: A Survey,” *IEEE Access*, vol. 8, pp. 201 071–201 086, 2020.

- [41] PlatformIO Labs, *PlatformIO: Professional Collaborative Platform for Embedded Development*, <https://platformio.org/>, 2024.
- [42] PlatformIO Labs. “What is PlatformIO?” Accessed: Apr. 21, 2026. [Online]. Available: <https://docs.platformio.org/en/stable/what-is-platformio.html>
- [43] Home Assistant Community, *Home Assistant Documentation*, <https://www.home-assistant.io/docs/>, Accessed 2024, 2024.

ภาคผนวก

ภาคผนวก ก

รายละเอียดบริบทโครงการ

โครงการเรื่อง “การพัฒนาต้นแบบสภาพแวดล้อมอัจฉริยะสำหรับผู้ใช้งานวีลแชร์ (WheelSense: Prototyping Development of Smart Environment for Wheelchair User)” เป็นโครงการที่ผสมผสานศาสตร์ทางวิศวกรรมคอมพิวเตอร์ วิศวกรรมไฟฟ้า และปัญญาประดิษฐ์ เพื่อสร้างระบบต้นแบบที่ติดตามข้อมูลจากรถเข็น อุปกรณ์สวมใส่ ระบบหลังบ้าน และส่วนติดต่อผู้ใช้งานให้อยู่ในภาพรวมเดียวกัน

ก.1 ปัญหาทางวิศวกรรมที่ซับซ้อน

ตารางที่ ก.1: การจัดเข้ากลุ่มปัญหาทางวิศวกรรมที่ซับซ้อนตามกรอบ IEA

หัวข้อ	รายละเอียดในโครงการ
R WP 1: ความลึกของความรู้	ต้องบูรณาการความรู้ข้ามศาสตร์ ทั้ง IoT, Signal Processing, Indoor Positioning, Backend Systems และ AI เข้าด้วยกันเป็นระบบเดียวแบบ end-to-end
R WP 2: ข้อจำกัดที่ขัดแย้งกัน	ต้องสร้างสมดุลระหว่าง latency กับความสามารถของ AI, privacy กับการประมวลผลบนคลาวด์, ความแม่นยำของตำแหน่งกับความเรียบง่ายของระบบ และ real-time monitoring กับ scalability
R WP 3: ความลึกของการวิเคราะห์	ต้องออกแบบ full pipeline, โมเดล RSSI localization และ workflow การให้เหตุผลของ AI เช่น Propose-Confirm-Execute
R WP 4: ความคุ้นเคยกับปัญหา	เป็นการผสมผสาน indoor localization, IoT sensor และ AI-assisted care ในสถานการณ์ผู้ใช้วีลแชร์ ซึ่งยังไม่มีโซลูชันมาตรฐานเดียวที่ใช้ได้ครอบคลุม
R WP 5: ขอบเขตของมาตรฐาน	ไม่มีมาตรฐานเดียวที่ครอบคลุมทั้งระบบ จึงต้องใช้สถาปัตยกรรมที่ปรับแต่งขึ้นเองร่วมกับ MCP และแนวปฏิบัติด้านความปลอดภัย
R WP 6: ผู้มีส่วนได้ส่วนเสีย	เกี่ยวข้องกับผู้ใช้วีลแชร์ ผู้ดูแล นักพัฒนา และผู้บริหารสถานพยาบาล ซึ่งมีความต้องการต่างกันทั้งด้านความสะดวก ความปลอดภัย และการปฏิบัติงาน
R WP 7: ความเกี่ยวพันของระบบ	เซนเซอร์ MQTT backend dashboard และ AI interface เชื่อมโยงกันใกล้ชิด หากระบบย่อยใดล้มเหลวจะกระทบต่อการทำงานของทั้งระบบ

ก.2 องค์ความรู้และเครื่องมืออุปกรณ์ที่ใช้

องค์ความรู้ทางวิศวกรรม ประกอบด้วยการประมวลผลสัญญาณและกลศาสตร์การเคลื่อนที่สำหรับ IMU สถาปัตยกรรมเครือข่าย IoT และ MQTT รวมถึง AI pipeline และ workflow การอนุมานของ LLM

องค์ความรู้ทางสาขาอื่น คือบริบทการดูแลผู้สูงอายุและข้อจำกัดของผู้ใช้งานวีลแชร์ในสถานพยาบาล ซึ่งส่งผลต่อวิธีออกแบบข้อมูลที่ต้องแสดงต่อผู้ดูแลและผู้พักอาศัย

โปรแกรมหรือเครื่องมือสมัยใหม่ ที่ใช้ในโครงการ ได้แก่ FastAPI สำหรับ backend, Docker สำหรับ

container บน edge server หรือ Raspberry Pi และ PostgreSQL สำหรับจัดการข้อมูลหลักของระบบ

ก.3 มาตรฐาน กฎหมาย และข้อบังคับที่เกี่ยวข้อง

มาตรฐานทางวิศวกรรมที่เกี่ยวข้องโดยตรงคือ Bluetooth Low Energy (BLE) สำหรับโหนดระบุตำแหน่งและการเชื่อมต่ออุปกรณ์ใกล้เคียง ส่วนมาตรฐานทางสาขาอื่นและข้อพิจารณาทางกฎหมาย ได้แก่ สิทธิการดำรงชีวิตอิสระของคนพิการตามกฎหมาย CRPD และหลักการ privacy-aware ในการจัดเก็บและประมวลผลข้อมูลสุขภาพ

ก.4 ผู้มีส่วนได้ส่วนเสีย ผลกระทบ และความยั่งยืน

ผู้มีส่วนได้ส่วนเสียหลักประกอบด้วยผู้สูงอายุและผู้ใช้งานวีลแชร์ กลุ่มผู้ดูแล หัวหน้าพยาบาล และผู้บริหารสถานพยาบาล เช่น ศูนย์ดูแลผู้สูงอายุวาระเวชภัณฑ์ ผลกระทบทางสังคมด้านบวกคือช่วยเพิ่มความปลอดภัย ลดความเสี่ยงจากเหตุฉุกเฉินที่ไม่มีผู้พบเห็น และลดภาระของบุคลากรในเวลากลางคืน ผลกระทบด้านสิ่งแวดล้อมคืออาจเกิด e-waste จากแบตเตอรี่และชิ้นส่วนเซนเซอร์เมื่อสิ้นสุดอายุการใช้งาน แนวทางแก้คือออกแบบโมดูลให้ถอดเปลี่ยนได้และจัดการชิ้นส่วนอิเล็กทรอนิกส์เข้าสู่กระบวนการรีไซเคิล

โครงการเกี่ยวข้องกับ SDG 3 ด้านสุขภาพและความเป็นอยู่ที่ดี, SDG 9 ด้านอุตสาหกรรม นวัตกรรม และโครงสร้างพื้นฐาน, SDG 10 ด้านการลดความเหลื่อมล้ำ และ SDG 11 ด้านเมืองและถิ่นฐานมนุษย์อย่างยั่งยืน

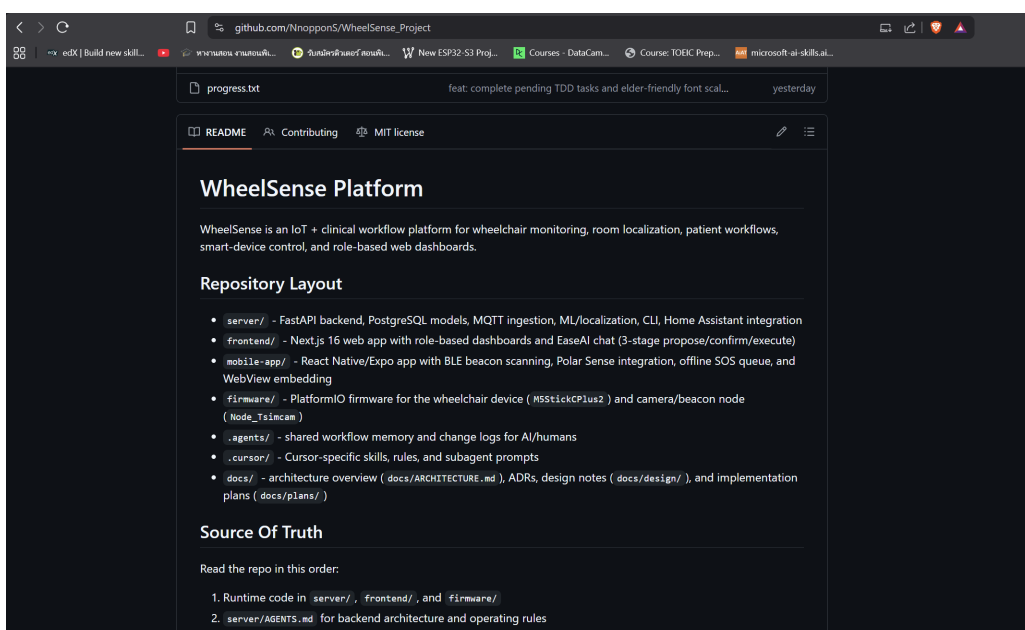
ก.5 การบริหารงานวิศวกรรมและโครงการ

ทีมงานประกอบด้วย 2 คน โดยวรพล แสงศรี รับผิดชอบ full-stack IoT และการบูรณาการฮาร์ดแวร์ เซนเซอร์ ส่วนศุภวิชญ์ อัสวไลทอง รับผิดชอบ AI architecture การให้เหตุผลของระบบ และการทำงานของ backend การบริหารเวลาใช้ Gantt chart และการประชุมรายงานความคืบหน้ากับอาจารย์ที่ปรึกษาผ่าน Microsoft Teams เป็นประจำทุกสัปดาห์ ด้านทรัพยากรได้รับการสนับสนุนวีลแชร์สำหรับการทดสอบจากบริษัท Matsunaga มูลค่า 45,000 บาท และมีค่าใช้จ่ายฮาร์ดแวร์รวม

ประมาณ 46,580 บาท การบริหารความเสี่ยงใช้การแยกพัฒนาเป็นระบบย่อย และปรับ AI workflow เป็น Propose-Confirm-Execute เพื่อลดความเสี่ยงจากคำสั่งอัตโนมัติที่ผิดพลาด

ภาคผนวก ข คลังขอร์สโค้ด

ภาคผนวกนี้แสดงแหล่งจัดเก็บขอร์สโค้ดของระบบ WheelSense ซึ่งประกอบด้วย backend, web dashboard, mobile gateway, firmware และเอกสารประกอบโครงการ



รูปที่ ข.1: หน้าจอคลังขอร์สโค้ด GitHub ของโครงการ WheelSense

https://github.com/NnopponS/WheelSense_Project

ภาคผนวก ค

แบบประเมินและข้อมูลประกอบการใช้งาน

ภาคผนวกนี้ใช้เก็บบริบทของแบบประเมินความพึงพอใจและการใช้งานระบบ โดยแบบประเมินเน้นประเด็นที่เกี่ยวกับความชัดเจนของหน้าจอ ความเข้าใจง่ายของข้อมูลแจ้งเตือน การเข้าถึงข้อมูลตามบทบาท และความเหมาะสมของการใช้ระบบในงานดูแลผู้ป่วยอาศัย

ตารางที่ ค.1: หัวข้อที่ใช้ในแบบประเมิน HMI

หัวข้อ	สิ่งที่ต้องการตรวจสอบ
ความชัดเจนของข้อมูล	ผู้ใช้ เห็น สถานะ ผู้พักอาศัย อุปกรณ์ และ alert ได้ โดยไม่ต้องค้นหาหลายขั้นตอน
ความเข้าใจของบทบาท	admin, head nurse, supervisor, observer และ patient เห็นข้อมูลที่ตรงกับหน้าที่ของตนเอง
การตอบสนองต่อเหตุการณ์	ผู้ดูแลสามารถเปิดดู alert และดูข้อมูลประกอบก่อนตัดสินใจได้
ความเหมาะสมของ mobile gateway	หน้าจอ / WheelSensor ช่วย ตรวจ BLE, Polar และ การส่งข้อมูลขึ้น server ได้ชัดเจน

ภาคผนวก ง

ภาพประกอบการทดลองและการติดตั้ง

ภาคผนวกนี้ใช้ประกอบบริบทการทดลองภาคสนามและการเตรียมอุปกรณ์ เช่น การติดตั้งโหนดสัญญาณ การวางตำแหน่งอุปกรณ์บนรถเข็น การตรวจ BLE และการทดสอบการส่งข้อมูลจาก gateway ไปยัง server โดยรายละเอียดเชิงตัวเลขของการทดลองสรุปไว้ในบทที่ 4

ตารางที่ ง.1: บริบทภาพประกอบที่ใช้ตรวจสอบการติดตั้ง

กลุ่มภาพ	สิ่งที่ใช้ตรวจสอบ
พื้นที่ทดสอบ	ตรวจขอบเขตพื้นที่ ห้อง โซน และ ตำแหน่งที่ใช้เก็บข้อมูล RSSI
อุปกรณ์รถเข็น	ตรวจ ตำแหน่ง M5StickC Plus2, การ ยึด อุปกรณ์ และ ทิศทางของ IMU
อุปกรณ์สวมใส่	ตรวจ ตำแหน่ง Polar Verity Sense และ การ จับ คู่ กับ mobile gateway
หน้าจอระบบ	ตรวจสถานะ gateway, dashboard, alert และการแสดงข้อมูลตามบทบาท

ภาคผนวก จ

ข้อมูลประกอบการทดลองระบบ

ภาคผนวกนี้สรุปชุดข้อมูลประกอบที่ใช้ตรวจสอบตัวเลขในบทที่ 4 โดยแยกตามระบบย่อย เพื่อให้ตรวจสอบย้อนกลับได้ว่าสรุปผลมาจากการทดสอบส่วนใดของระบบ

ตารางที่ จ.1: ชุดข้อมูลประกอบที่ใช้ตรวจสอบผลการทดลอง

กลุ่มข้อมูล	ค่าที่ใช้ยืนยันในบทที่ 4
Wheelchair sensor	sampling rate, telemetry completeness, device-to-broker latency และ ความสำเร็จของการเชื่อมต่อ Polar
Motion detection	ผล ตรวจ จับ stationary, normal motion, sharp turn และ impact-like movement
Phone gateway และ Polar	BLE GATT connection, MQTT reconnect, HR/ PPG completeness และ latency smartphone-to-broker
HMI usability	task completion, satisfaction score, alert readability และ role navigation clarity
Server และ database	startup time, REST latency, MQTT latency, DB insert/select latency และ alert propagation
MCP/Home Assistant	device control success, camera capture success และ latency ของคำสั่งควบคุม

ภาคผนวก ฉ

หลักฐานการประเมิน AI และ MCP

ภาคผนวกนี้สรุปหลักฐานเพิ่มเติมของการประเมิน AI pipeline และ MCP tool workflow ที่บทที่ 4 รายงานไว้ในระดับผลรวม โดยเน้นสามส่วนคือการเลือกเครื่องมือ ความคงที่ของคำตอบ และเวลาแฝงของแต่ละเลเยอร์

ตารางที่ ฉ.1: ผลรวมการเลือกเครื่องมือของ AI เทียบกับคำตอบอ้างอิง

กลุ่มคำสั่ง	ผลถูกต้อง	หมายเหตุ
No-ambiguous	10/10	คำสั่งเจตนาชัดเจน
Low-ambiguous	10/10	มีความกำกวมต่ำแต่ยังเลือก tool ได้ตรง
Med-ambiguous	10/10	context assembler ช่วยจำกัดบริบทก่อนส่งให้ LLM
High-ambiguous	1/6	ระบบเลือกถามยืนยันแทนการเดาสุ่มในกรณีเสี่ยง
รวม	31/36	accuracy 86.11%

ตารางที่ ฉ.2: เวลาแฝงของ AI pipeline

Pipeline layer	p50 (ms)	p95 (ms)
Layer 1: Intent Router	198.40	248.05
Layer 2: Context Assembler	96.26	126.79
Layer 4: Constrained LLM	14257.39	36487.60
Layer 5: Execution Gate	40.38	78.62
รวม	14592.43	36941.06

ภาคผนวก ข

ข้อมูลประกอบเพิ่มเติมจากชุดทดลอง Paper IEEE

ภาคผนวกนี้รวมรายละเอียดที่ไม่ได้ใส่ไว้ในบทที่ 4 ได้แก่ จำนวนข้อมูลของแต่ละกรณีทดลอง ตารางเปรียบเทียบโมเดล และ confusion matrix ของผล KNN เพื่อให้ตรวจสอบผลการจำแนก RSSI ได้ละเอียดกว่าตารางสรุปหลัก

ตารางที่ ข.1: จำนวนข้อมูลที่ใช้ในชุดทดลอง RSSI/KNN

Case	งานที่ทดสอบ	Train	Test	Outcome
Case 1	จำแนก ทิศทาง facing in/ facing out	13	35	35
Case 2	จำแนก boundary transition	13	35	35
Case 3	ทำนายลำดับ zone	13	35	35

ตารางที่ ข.2: ผลเปรียบเทียบ KNN และ XGBoost ในชุดทดลองเดียวกัน

Case	Model	Accuracy	Precision	Recall	F1
Case 1	KNN	0.8857	0.8421	0.9412	0.8889
Case 1	XGBoost	0.9714	1.0000	0.9412	0.9697
Case 2	KNN	0.6857	0.6875	0.6471	0.6667
Case 2	XGBoost	0.5714	0.5455	0.7059	0.6154
Case 3	KNN	0.8857	0.8857	0.8857	0.8857
Case 3	XGBoost	0.4857	0.4857	0.4857	0.4857

ตารางที่ ข.3: Confusion matrix ของ KNN ใน Case 1

Actual / Predicted	0	1
0	15	3
1	1	16

ตารางที่ ข.4: Confusion matrix ของ KNN ใน Case 2

Actual / Predicted	0	1
0	13	5
1	6	11

ตารางที่ ข.5: Confusion matrix รายตำแหน่งของ KNN ใน Case 3

Actual / Predicted	1	2	3	4
1	28	1	0	1
2	0	35	1	0
3	0	0	39	1
4	0	0	0	34

ใน Case 3 การนับแบบ exact sequence ได้ผลถูกต้อง 31 จาก 35 episode หรือ 88.57% แต่ถ้าแยกรายตำแหน่งจากทั้งหมด 140 ตำแหน่ง จะได้ผลถูกต้อง 136 ตำแหน่ง หรือ 97.14% ความต่างนี้เกิดจากเกณฑ์ exact sequence ที่นับว่าผิดทั้ง episode แม้ลำดับผิดเพียงตำแหน่งเดียว จึงควรอ่านผล Case 3 ทั้งสองแบบร่วมกัน